

人工智能前沿技术丛书

Broadview®
www.broadview.com.cn

深入浅出 AI 算法

基础概览

Head First Artificial Intelligence:
Elementary Overview

吕磊◎著

循序渐进迈入人工智能算法大门
阐述AI算法三大研究与应用方向



中国工信出版集团



电子工业出版社
http://www.phei.com.cn

随时、随地、随心阅读

人工智能前沿技术丛书

Broadview®
www.broadview.com.cn

深入浅出 AI 算法

基础概览 Head First Artificial Intelligence:
Elementary Overview

吕磊◎著

循序渐进迈入人工智能算法大门
阐述AI算法三大研究与应用方向

中国工信出版集团 电子工业出版社
http://www.phei.com.cn

版权信息

COPYRIGHT INFORMATION

书名：深入浅出AI算法:基础概览

作者：吕磊

出版社：电子工业出版社

出版时间：2021-07-01

ISBN：9787121415517

本书由电子工业出版社有限公司授权北京当当科文电子商务有限公司制作与发行。

— · 版权所有 侵权必究 · —

作者简介



吕 磊

微软（中国）软件工程师。本硕毕业于山东大学计算机科学与技术系，具有多年算法从业经验，曾先后在京东广告部、蚂蚁金服人工智能部、亚马逊AWS（中国）的AI产品相关部门从事算法工作，在工业界有着丰富的AI算法落地经验。

内容简介

本书从理论到实践，循序渐进地介绍了人工智能算法的基础知识，帮助读者敲开人工智能算法之门。本书内容共8章，包括算法的历史背景与基本概念、算法相关的数学基础知识、信息学算法与数据结构相关的概念与知识，以及业界常用的机器学习算法。同时，本书还介绍了算法工程的组成部分，以及一个典型的算法工程实践项目，手把手带领读者体验算法的魅力。本书还介绍了人工智能算法的三大研究方向，帮助读者迈向AI算法的进阶学习之路。

本书每一章的内容都采用了“总分总”形式，并且在章节末尾提炼出该章的核心关键词，方便读者进一步查询回顾。其中，第2章～第7章配有若干代表性的思考题，帮助读者巩固章节所学知识。

本书适合从事与人工智能相关的工程技术人员和高等院校相关专业的学生，以及在AI领域就业一两年以内的职场人士阅读。

推荐序

自进入21世纪以来，得益于大数据和计算机软硬件技术的迅猛发展，在全球范围内出现了机器学习研究与应用的热潮。机器学习大大提高了语音识别、图像识别、自动驾驶、虚拟现实和机器人等高新技术的发展进程。与前三次工业革命一样，机器学习也是一种能够产生巨大生产力的革命性技术。机器学习是专门研究如何让计算机模拟或实现人类行为的过程，通过调整特定的参数和配置，使对应的机器学习算法模型不断改善自身在某个方面的性能，如对话、翻译、游戏、驾驶等。它是当前人工智能（AI）发展的主要方面，是使计算机具有智能的重要途径。现阶段所讨论的AI算法，通常指的就是机器学习算法。

作为一门交叉学科，机器学习涉及概率论、统计学、组合数学、最优化理论等多门学科，因此学习门槛很高。如何降低机器学习算法的学习门槛，让更多的人可以理解、掌握这些算法并加以改进，从而应用于实际工作中，是一个亟须解决的问题。而本书是在这方面所做的有益尝试。

本书共8章，内容丰富，对常用的机器学习算法进行了深入浅出的介绍。围绕机器学习算法，本书囊括理论和实践，在保证完整性的前提下，重点探讨各个环节中的核心问题。从广度上看，本书讨论了算法的两种类别，即偏逻辑类型的信息学算法与偏数据类型的机器学习算法。从深度上看，本书讨论了相关的数学原理、算法原理及工程实战。

本书作者吕磊曾于山东大学师从于我，攻读硕士学位，毕业后先后就职于京东、阿里巴巴、亚马逊与微软的AI相关部门，在工业界有较为丰富的机器学习算法应用经历。

本书可以作为对人工智能与机器学习感兴趣的本科生、研究生，以及程序开发人员的算法入门参考书。

禹晓辉

加拿大约克大学信息技术学院副教授

山东大学计算机科学与技术学院兼职教授、博士生导师

前言

为什么会有这本书

在笔者从事算法相关的工作后，身边有不少朋友问笔者“算法到底是什么”“如何理解人工智能（Artificial Intelligence，AI）的概念”等问题。他们平时在各类电视节目、网络自媒体及新闻媒体中经常接触AI算法的相关信息，因此对AI算法表现出比较浓厚的兴趣。

经过进一步的沟通了解，笔者发现可以将这些朋友划分为两类人群。第一类人群，他们希望从事AI算法职业，或者希望转型做AI算法相关的工作；第二类人群，他们不想学习AI算法，但是想了解AI算法背后的原理，以及整个AI项目的运作机制、流程和规律等。无论是第一类人群的“入门”需求，还是第二类人群的“鸟瞰”需求，市面上已经有很多相关资料，如吴恩达在Coursera上发布的机器学习课程、斯坦福大学的CS在线课程系列、周志华的《机器学习》及网络上的各种算法学习教程等。这些都是非常优质的AI算法学习资源。但可惜的是，对大部分人来说，一方面，Coursera和斯坦福大学的课程门槛太高，零基础入门不易；另一方面，虽然大部分学习资源在某个方向上基本都能做到比较高质量的讲解，但很少有学习资源是使用算法串联整体内容的，即很少有学习资源可以同时满足“入门”和“鸟瞰”需求。学习者往往需要通过整合多个信息源，才能对AI算法形成一个较为完整的总体印象。

此外，在和朋友讨论算法的过程中，笔者发现一件比较有意思的事情：大家通常会聊到两类算法，一类是信息学算法，另一类是AI算法。虽然都叫算法，但没有一个比较权威的划分方式将二者区别开。比较常见的区分方法是，信息学算法是有确定答案的、结果准确率为

100%的精确算法；而AI算法的答案是不确定的，其结果的准确率通常也不可能达到100%。从这个角度来看，信息学算法的运算过程其实和简单的数学运算的运算过程没有本质区别，二者在每一步计算环节都有明确的结果，最后都会得到一个确定答案。AI算法在每次训练后的准确率都不固定，这体现出了“智能”的高低差异。虽然，当下“人工智能”要摆脱“人工”的日子看起来比较久远，但至少已经出现了一些让人为之惊叹的“人工”奇迹，如AI围棋程序、AI语音助手和AI自动驾驶等。

关于这两类算法的区别，笔者从《樊登读书》中获得了一种新的思考角度。书中提到，这个世界上有一种学问叫作简单科学，还有一种学问叫作复杂科学。简单科学可分为很多个步骤，每个步骤都有明确的输入和输出，各个步骤之间的因果关系清晰、明确、可追溯。例如，建造一栋大楼，或者建造一艘飞船，虽然工程复杂，但完全符合简单科学的范畴。复杂科学是混沌的，因果关系并不明确，如蝴蝶效应、群智涌现等。笔者借助从樊登老师那里得到的灵感发现，信息学算法的步骤和结果明确，AI算法的因果关系混沌，二者恰好可以分别归于简单科学和复杂科学的范畴。这种分法或许比前面的分法更有趣一些。

为了满足读者“入门”和“鸟瞰”的不同需求，一方面，本书在内容筛选上必须大刀阔斧，又不能破坏内容结构的完整性；另一方面，本书将上述两类算法囊括在一起呈现，使读者能清晰、明确地认识二者之间的区别与联系。

本书是笔者温故知新的结果。俗话说，好记性不如烂笔头。通过归纳、总结与再学习，笔者在AI算法方面也有不少新的认识和收获。

本书的读者定位

AI算法博大精深，所含各部分内容都足以各自形成一本书。本书作为一本算法概览书，其内容尚不足以形成完整而系统的算法学习体

系。此外，考虑到笔者存在的信息采样误差，本书挑选的常用知识点不一定具备绝对的普适性。

本书的目标受众包括但不限于行业属性或工作职能与AI算法有关的人士、从事传统行业的工程师、面临毕业的学生、初入职场的人士等。当然，无论是哪一类人群，前提都需要对AI算法感兴趣。爱因斯坦说过，兴趣是最好的老师。笔者希望，读者在读完本书后，可以对AI算法有一个较为清晰的整体认识，并且针对自己感兴趣的环节进行更深入的学习。

本书的内容及风格

本书从介绍算法是什么开始，包括发展简史和目前与算法有关的职位和领域，循序渐进地帮助读者敲开算法的大门。首先，本书甄选了与算法有关的数学基础知识，包括线性代数、排列组合、高等数学、概率与统计等，为读者正式接触算法奠定一定的数学基础。其次，本书从信息学算法与数据结构入手，讲解了二者最常用的概念与知识点，帮助读者培养一定的算法逻辑思维能力和数据结构思想。然后，切入AI算法正题，从总体视角将AI算法划分为两种类别，并且甄选业界常见的几类机器学习算法模型，帮助读者在总体把控的前提下，掌握不同机器学习算法模型中的具体算法，以及它们的实现原理。接着，本书会介绍算法工程的组成部分，并且手把手带领读者完成一个简易的算法工程项目，通过实践加深其对AI算法的理解，使其能够学以致用。最后，本书挑选出AI算法的三大研究方向，帮助读者铺垫AI算法学习的进阶之路，并且总结了笔者对算法行业的思考和展望。全书内容从基础到进阶，从理论到实践，层层递进。

学习算法对数学基础和逻辑思维的要求比较高。因此，本书尽量通过列举与生活息息相关的例子，对其中的一些概念进行直观讲述，如向量、矩阵、数组、链表。不过，并不是所有的知识点都可以通过直观举例来讲解，如高等数学中的导数和梯度、概率与统计中的各种名词等。所以，对于真正想入门AI算法的读者，该下的功夫、该花的时间是少不了的；而对于只想对AI算法有所了解的读者，在遇到较难

理解的概念和公式时，只需了解它们在AI算法中的作用，可以跳过理论细节。

本书的章节标题运用了武侠小说中的“内力”“招式”“武功秘籍”，分别对应数学基础、信息学算法和机器学习算法。有人说，信息学算法和机器学习算法没什么关系，其实不然。看过武侠作品的读者应该知道，“内力”和“招式”作为“武功秘籍”修炼的前提，是一种基础能力。相应地，数学基础和信息学算法对学习机器学习算法而言，不仅是基础的铺垫，而且有助于思维模式的培养。

本书每章的内容都采用“总分总”形式，并且在章节末尾提炼出核心关键词，方便读者进一步查询回顾。其中在第2章~第7章末尾，针对章节内容留有若干思考题，帮助读者巩固和理解相关知识。关于动脑时刻的答案，请关注“对半独白”微信公众号并回复“动脑时刻”获取。

我与本书的局限性

本书的局限性。与其他同类型的机器学习入门书籍相比，本书关于AI算法的内容要少很多。本书作为一本AI算法图书，主要目的在于帮助读者构建一个较为完整的AI算法基础知识体系。作为一本只有200多页的书，既然选择了整体性，就必须舍弃部分性，这便是本书最大的局限性。但反过来想，无论哪一门学问，其知识体系都在持续不断地迭代更新。从这个角度来看，真正做到内容大而全的书，其实是不存在的。所谓“万变不离其宗”，授人以名目齐全之鱼，不如授人以追本溯源之渔。因此，本书在内容层面的局限性，未尝不会成为目标读者群体的优先选择性。

笔者的局限性。有一个词可以充分表达笔者的局限性，这个词或许也可以描述所有作者的局限性，那就是“偏见”。数据有偏见，写作也有偏见。在AI算法模型中，我们需要做的非常重要的一件事就是减少数据偏见，以及降低数据偏见带来的不良结果。所谓写作偏见，就是笔者在写作之前的各种经历和所处环境，对本书写作所带来的影

响，包括但不限于长期积累所形成的个人认知水平、表达水平、专业水平、职业惯性，乃至个人素养和品质。因此，书中如有表达不当或有失偏颇之处，望读者海涵。

吕磊

致谢

首先，我想感谢阅读本书的读者朋友。你们的青睐和支持，给了我坚持写作的原动力。如果本书能给大家带来收获，是我莫大的荣幸。

然后，我想感谢本书责任编辑——电子工业出版社的宋亚东先生。您从“对半独白”公众号第一次联系到我直到现在，已近3年时光。从决定开始写书到最终完稿，这之间的过程真的十分不容易，感谢您一路对我的帮助。如果没有您的支持与鼓励，我不会顺利完成本书的写作，您是我人生旅程中的伯乐。

最后，我想感谢我的妻子——魏金璐。从校园恋爱开始，谢谢你一路走来对我的付出，不管遇到什么困难，你总是默默地在我身边倾听与陪伴，无论是工作还是生活，都给了我无穷的动力。

吕磊

第1章 算法入门

在学习一门技能之前，先了解这门技能的需求和功能，可以让学习目标更加明确，让学习更有动力。随着计算机硬件技术的飞速发展，我们可以收集到各种类型的互联网数据、交通数据、气象数据等并将其保存下来。根据收集到的不同场景中的数据，可以总结出相应的规律，从而制定、调整和优化针对不同场景的策略。总结这些规律的具体方法，就是我们要讲的算法。

在各大互联网公司中，算法相关的岗位招聘越来越多。除了比较典型的算法工程师（外企中相应的岗位为数据科学家），还有数据分析工程师、数据挖掘工程师和机器学习工程师。这几个岗位的工作内容既有相同之处，又有不同之处。那么，算法与数据分析、数据挖掘、机器学习的区别与联系是什么？算法与人工智能之间有什么关系？带着这些问题，本章为大家打开算法之门，揭开算法的神秘面纱。

1.1 打开算法之门

1.1.1 算法简史

算法存在的前提是数据，有数据才能有算法。最早关于人类记录数据的例子是公元前18 000年前出现的符木。生活在旧石器时代的部落人民学会了在树枝上刻下凹痕来记录日常的物品供应和一些简单的交易活动，并且能够通过比较树枝的凹痕进行基本的计算，进而对一些事情做出简单的预测，如食物可以储存多久不变质。

到了公元前4000年左右，在发源于亚洲西部亚美尼亚高原的幼发拉底河与底格里斯河河畔（两河流域），居住着一群人类最早的居民——苏美尔人，他们创造了璀璨的苏美尔文明，也就是美索不达米亚文明的开端。

在像美索不达米亚那样极其恶劣的气候下，知道播种和收获的准确时间是必不可少的，因此有必要找到某种确定时间的可靠途径。最简单的方法是利用月亮的盈亏循环。月亮由最初的蛾眉月到下一个最初的蛾眉月共需要29天半的时间，人们就考虑将这样一个循环视为一个基本的计时单位（我们将其称为一个月），然后在月亮累计运行12个这样的计时单位（6个29天，6个30天）后，一“年”就过去了，又到了开始播种的时候。不幸的是，他们不知道一年实际上是地球绕太阳公转一周的时间，月亮的12次循环或12个月比一个太阳年少11天。在900年后，苏美尔人才了解到，每隔几年他们就要在其年历上另加一个闰月，才能准确地预测季节的循环。

通过感受季节的周期性更替，结合对太阳和月亮的观察，苏美尔人花了900年的时间收集数据，完成了人类的第一个算法模型——太阳（月）历。随后，苏美尔人发明了度量，包括12单位度量和60单位度量，这便是人类最早的十二进制和六十进制。现在世界各国通行的时间度量方法，每天分为24小时，每小时包含60分钟，每分钟包含60秒，钟表刻度盘上走一圈为12小时，还有以12为基本计量单位的“打”，都是在十二进制和六十进制的基础上发展而来的。在公元前2500年左右，苏美尔人就已经掌握了加、减、乘、除四则运算的计算方法，形成了早期比较简单的几何与代数方法论。

时间线走到公元前300年左右，一种耳熟能详的算法——**辗转相除法**，由古希腊数学家欧几里得提出，因此这个算法又称为欧几里得算

法。辗转相除法的意思是两个正整数 a 和 b ($a > b$) 的最大公约数，等于 a 除以 b 的余数与 b 的最大公约数。该算法的具体步骤如下。

- (1) 输入两个正整数 a 和 b ；
- (2) 计算 a 除以 b 的余数 r ；
- (3) 如果 r 等于0，则最大公约数为当前的 b 值，算法结束；
- (4) 如果 r 不等于0，则 $a=b$ ， $b=r$ ，跳转至步骤(2)。

在公元100年左右，比较全面的算法研究出现在由我国古代的张苍和耿寿昌撰写的《九章算术》中。作为我国古代的数学名著，《九章算术》的内容十分丰富，全书采用问题集的形式，总结了自春秋战国至秦汉以来的数学成就，对246个与生产、生活有联系的应用问题逐一提供了与之相应的解决之法。不过，《九章算术》存在着不可忽视的缺点，那就是全书没有任何数学概念的定义，在各个问题的解决方法中，也没有给出相关的理论推导与证明。直到三国时期，数学家刘徽给《九章算术》作注，才弥补了这个缺陷。

总之，早期的算法研究倾向于计算和解决与日常生活息息相关的数学问题。随着算法理论的发展和人类科技的进步，算法研究逐渐转向研究数据本身的规律和价值，尤其是在生产与生活中产生的大量数据所带来的预测价值和决策价值。

到了20世纪上半叶，数理统计学蓬勃发展。作为数学的一个重要分支学科，数理统计学主要研究随机数据的收集、汇总与分析之法，然后对相应的实际问题做出预测和推断。这期间先后诞生了影响至今的两大统计学派——频率学派和贝叶斯学派。从20世纪初开始，以Fisher和Pearson为主要代表人物的频率学派，推动数理统计学发展成为一门成熟的学科。频率学派从自然的角度出发，试图直接给事物本身建模，认为概率就是频率，描述数据分布的参数是固定的，可以使用估计方法得到数据分布的参数值。然而，频率学派这种类似上帝视角的问题切入法，并不能很好地解决所有的实际问题。在20世纪中叶

以后，曾一度被频率学派扼杀的贝叶斯统计迅速发展壮大起来，和频率学派分庭抗礼。贝叶斯学派否定了频率学派的概率就是频率的观点，并且不从描述事物本身入手，而是从观察者角度出发，构造出了一套在贝叶斯概率体系下，对事物不确定性做出推断的框架和方法。当然，无论是频率学派，还是贝叶斯学派，都贡献了一系列非常实用的算法并沿用至今。例如，频率学派的**最大似然估计**（Maximum Likelihood Estimate，MLE）在神经网络模型中应用广泛；贝叶斯学派的**蒙特卡罗方法**（Monte Carlo Method）在强化学习的探索 and 发现中起到了非常关键的作用。总之，两大学派的理论都很重要，缺一不可，并无高下之分。

从20世纪下半叶开始，在两大统计学派的激烈博弈中，感知机学习、统计学习、归纳学习、浅层学习等陆续萌芽发展，这类研究数据本身规律和价值的算法，称为**机器学习算法**。到了20世纪80年代，机器学习算法逐渐成了一个独立的研究方向。在这段时间，诞生了大量关于机器学习的算法，其中一些比较典型的、具有代表性的算法将在后续的章节中介绍。

注意：为了避免概念混淆，本书有关特定数学问题的算法（前言中提到的信息学算法，详见第3章）统称为基础算法，有关研究数据规律和价值的算法（详见第4章）统称为机器学习算法（或AI算法）。

1.1.2 算法与人工智能

相信有不少读者是抱着对人工智能的兴趣阅读本书的。在比算法世界更大的世界中，人们谈论更多的也是人工智能，而非算法。那么，二者到底有什么关系？

人工智能的概念从提出到现在已经过去了半个多世纪，但是关于人工智能的定义却依然存在争议。我们不妨仔细审视一下“人工智能”，它由“人工”和“智能”两个词组成。

先看“智能”一词，它的字面意思是智慧和能力。在自然界，智能可以分为个体智能和群体智能两种。除了人类，被广泛认为具备个体智能的生物还有大猩猩、海豚和章鱼等，具备群体智能的生物有蜂群、蚁群和沙丁鱼群等。在个体智能中，只能表现兴奋和抑制两种状态的人类大脑神经元，形成了复杂的思维与智慧。在群体智能中，通过分泌信息素完成交流的蚂蚁，构成了分工明确、井然有序的蚁群。从这个角度来看，无论是个体智能，还是群体智能，都是由大量简单行为连接转化而成的。在20世纪中期，冯·诺依曼受此启发，提出了**元胞自动机**（Cellular Automata，CA）。元胞自动机中单个个体的算法非常简单，其状态改变只受周围直接相连的其他个体状态影响。例如，假设个体只有0和1两种状态，算法设定个体本身的状态跟随周围个体中多数状态值的变化而变化。令人难以置信的是，这些由简单算法构成的元胞自动机，居然成功模拟出了蚂蚁、大雁、洄游鱼类等动物的一些群体智能行为。

再看“人工”一词，字面理解为由人创造。所谓人工智能，就是由人创造的智能。上文提到的由算法驱动的元胞自动机体现出的群体智能行为，就是一种由人创造的智能。

所以，人工智能离不开算法，算法是产生人工智能的直接工具。

1.1.3 什么是数据分析

顾名思义，**数据分析**（Data Analysis）就是对数据进行分析，即对收集到的数据进行汇总与统计，并且从中提取和归纳出有价值的信息。数据分析一般会预先确定一个明确的统计指标，如总和、均值等；然后采用相应的计算方法和工具，得到预定统计指标的结果；最后采用合适的形式将结果展示出来。

数据分析作为算法研究的第一步，从古至今从未改变，如古代关于天文、历法研究的第一步就是长期收集和汇总数据。因此，从工作

职能上区分，与算法工程师相比，数据分析工程师一般只需专注于数据分析模块。

我们通过一个实例表明数据分析到底在做什么。例如，“关于大学生手机使用情况调研”课题，数据分析的目标之一是统计各种数量总和与均值，包括但不限于不同年级、不同性别的大学生购买和使用不同款式品牌的手机数量总和与均值。然后将这些统计值使用合适的图表展示出来。例如，使用条形图展示“最受男生欢迎的手机型号排序”“销量最高的大学生手机型号排序”“最受大学生欢迎的手机品牌排序”的统计结果，使用折线图展示“大学生每月新购的手机数量”“大学生一周中每天的平均通话时间”的统计结果。

1.1.4 什么是数据挖掘

顾名思义，**数据挖掘**（Data Mining）就是对数据进行深入研究，其目的不仅是获得数据统计分析的表层结论，更多的是挖掘数据与数据之间的内在关系。从这个层面而言，**数据挖掘的精髓在于探索**，探索数据潜在的模式、规律和规则，是一种深层次的数据分析。数据挖掘既有基于统计的算法，又有基于模型的算法。虽然基于模型的算法和机器学习算法存在不少重合之处，但它们的侧重点不同。

数据挖掘作为机器学习算法研究中承上启下的一步，比数据分析更能深入数据，并且不完全依赖于算法模型。因此，与算法工程师相比，数据挖掘工程师对数据模式与规律、规则等数据方面的“嗅觉”要更敏锐。

在“关于大学生手机使用情况调研”的课题中，基于统计的数据挖掘，可以研究一些特征的共现情况。例如，根据“视频”“游戏”“聊天”“拍摄”等标签的分析统计结果，结合固有人群标签（性别、年级、专业等）的统计结果，可以得出各个标签之间的关联度分析结果。基于模型的数据挖掘，可以研究相似群体。例如，“具备相似手机使用习

惯的大学生”有哪几类，每个类别下的大学生分别具备什么样的特质，等等。

1.1.5 什么是机器学习

顾名思义，**机器学习**（Machine Learning）就是让机器自己去学习。既然让机器自己学习，就需要人工预先指定一套学习规则，这套既定的学习规则就是机器学习的算法模型。从这个层面来说，**机器学习的精髓在于优化**，即根据收集到的数据样本，优化指定机器学习算法模型中的各项参数。

机器学习作为算法研究的最后一个环节，从狭义上来说，其重点强调的是模型的选择与参数优化的方法。因此，与算法工程师相比，机器学习工程师专注于模型层面的研究。

在“关于大学生手机使用情况调研”的课题中，机器学习可以利用所有能提取到的大学生特征，基于模型完成某些特定场景中的预测或决策任务。例如，机器学习可以通过模型预测帮助电商平台推荐更适合大学生用户群体的手机品牌与型号，通过模型决策帮助手机厂商提高面向大学生手机市场的投资收益率（Return on Investment，ROI），等等。

1.2 如何学习算法

前面介绍了数据分析、数据挖掘和机器学习三个概念，并且阐述了三者对应的工程师在工作职能上与算法工程师之间的区别与联系。总体而言，算法工程师的要求比较全面，既要求懂数据分析，又要求懂算法模型。

算法是一个很大的概念，AI算法的相关知识至少涵盖了数据分析、数据挖掘、机器学习的相关知识。无论是哪方面的内容，所涉及

的理论、方法和工具，都能形成一本书的内容。而相关书籍之多，加上缺乏较为清晰和完整的学习路线，往往让大部分初学者望而却步，面对庞杂的知识体系无从下手。此外，将算法的所有相关知识浓缩到一本只有200多页的书中，也是很难办到的。

那么，究竟如何入门算法比较合适？要学习一个未知领域的知识，一种比较直观的学习方式是，先了解这个未知领域的核心是什么，把握整个领域的重要组成部分与大致脉络，再对各个局部领域的延伸与拓展填充学习，丰富各个部分的内容细节。这种学习方法的好处有二：其一，尽早对未知领域有一个整体的认识，可以降低学习过程中的倦怠性，使学习更有动力；其二，这种学习方式能帮助读者尽早具备未知领域知识的完整性，有助于在学习过程中宏观把控学习进度，使所学内容更有针对性。

基于上述考量，本书想做的正是这样一件帮助读者构建算法认知骨架的事情。因为无论是基础数学、基础算法，还是模型算法、工程实战，各自展开都远不止本书所讲内容。本书的主旨中心思想很简单，就是给大家展示AI算法世界的全貌。希望本书在读者入门算法的道路上，可以起到抛砖引玉的作用。本书要论述的知识面非常广，对于各个环节之中未能提及的内容和细节，读者可以根据自己的兴趣自行查阅相关资料，进行延伸学习。

1.3 本书结构

本书从算法相关的数学基础开始讲起，到信息学竞赛常用的基础算法，以及工业界常用的机器学习算法，再到算法工程的全貌，手把手教大家构建自己的第一个算法工程，然后介绍三大热门研究方向的算法进阶内容，最后对机器学习算法行业进行一定的思考和展望。

第1章，算法入门。本章主要介绍算法的发展简史、算法与人工智能的关系、算法的相关概念，以及相关职位之间的区别和联系，帮助读者建立对算法的第一印象。

第2章，算法之内力。本章主要介绍算法中的几个重要的数学知识点。

第3章，算法之招式。本章主要介绍常见的几种基础算法。

第4章，算法之武功秘籍。本章主要介绍常见的几种机器学习算法。

第5章，算法工程的组成部分。本章主要介绍算法工程的组成部分。

第6章，算法工程实战。本章主要介绍算法工程具体应该如何开展，并且附带实际操作过程。

第7章，进阶学习。本章主要介绍AI算法的三大进阶内容，包括深度学习、强化学习和迁移学习。

第8章，思考与展望。本章内容是作者关于算法行业的思考与展望。

关键词回顾

算法

人工智能

数据分析

数据挖掘

机器学习

元胞自动机

辗转相除法

第2章 算法之内力

数学涵盖的知识面非常广，本章会根据重要程度筛选出数学与算法之间的相关知识点，包括线性代数、排列组合、高等数学、概率与统计、最优化原理；然后将每个知识点细分为名词解释和具体用法两部分，按照从易到难的顺序，尽量采用通俗易懂的语言进行讲解，帮助读者修炼算法世界的“内力”。

算法是一个比较抽象的概念。往大了说，所有涉及运算的方法都可以称为算法；往小了说，在信息技术（Information Technology, IT）领域，算法可以是解决数学问题的方法与技巧，也可以是描述机器学习或数据挖掘的相关方法，本书会从这两个方面展开剖析。

讲清楚什么是算法并不是一件容易的事情，完整且浅显易懂地讲明白更是难上加难。算法涵盖了很多相关学科的基础概念与知识点，涉及的名词解释与概念介绍较为生涩，难免会降低读者的学习兴趣。作为一本算法知识普及图书，本书涵盖的每部分内容虽然不必太深，但必须全面。因此，如何保证本书的阅读趣味性、内容的完整性与连贯性，在提笔写书之前的很长一段时间里，困扰了我很久。直到后来重看古装电视剧《天龙八部》才猛然发现，本书所要讲的算法，和金庸先生笔下的武功体系，其实有着异曲同工之妙。

金庸武侠，闻名遐迩。说到金庸先生笔下的武功体系，无外乎两大方面：内力和招式。看过《天龙八部》的读者应该知道，聚贤庄一役，被逐出丐帮的萧峰义字当头，对战中并没有使用半招降龙十八掌，而是只用了一套普普通通的太祖长拳，就将围攻他的武林高手打

得落花流水，节节败退。为什么？凭的就是他深厚雄浑的内力。由此可见，一旦具备了深厚的内力，普通招式也能发挥出强大的威力。而在算法的世界中，数学功底即“内力”，只有将数学这门“内力”修炼好，才能让算法在实际应用中发挥出强大的作用。

2.1 线性代数

线性代数是数学的一门分支学科，研究的是线性空间中的函数问题。顾名思义，“线性”是指要研究的代数之间的关系是简单的一次关系，研究过程中没有复杂的数学运算（如平方和开方）；“代数”是指用符号代替数字，方便研究。

对于数学关系 $f(x+y)=f(x)+f(y)$ ，线性代数不关心其中 x 和 y 的具体含义是什么，其研究的是拥有这种线性映射关系的函数 f 所具备的性质。

函数是指输入一个数，在经过某些计算后输出另一个数。随着研究的问题越来越复杂，有时需要输入多个数，然后经过运算输出多个数。从这个角度来说，线性代数解决的就是多输入或多输出的函数问题。

2.1.1 名词解释

在线性代数中，单个数称为标量，用中括号括起来的多个输入的数或多个输出的数（多个标量）称为向量和矩阵。标量、向量和矩阵是算法工程中经常用到的线性代数核心概念。下面从数学角度给出标量、向量和矩阵的概念，以便加深读者对线性代数基本概念的理解。

- 标量：表示一个简单的数，如29、31等。
- 向量：表示一组有序的标量，能够写成一行或一列的形式。

• 矩阵：表示一组有序的向量（一个或多个），可以用二维数组的形式表示。

下面来看一个例子。假设某家包子铺周一能卖出50个肉包，这里的“50”就是一个标量；如果这家包子铺某周每天卖出的包子数量依次是[50, 51, 52, 53, 52, 48, 45]，则“[50, 51, 52, 53, 52, 48, 45]”是一个向量；加上这周每天卖出的豆沙包、咸菜包的数量，组成如下数据形式，就是一个矩阵。

$$\begin{bmatrix} 50 & 51 & 52 & 53 & 52 & 48 & 45 \\ 49 & 48 & 53 & 51 & 50 & 45 & 47 \\ 52 & 52 & 54 & 51 & 56 & 50 & 49 \end{bmatrix}$$

2.1.2 向量和矩阵

标量的概念比较简单，因此我们重点讲解向量和矩阵。向量和矩阵在算法工程中经常用到。例如，一条样本的所有特征值构成一个向量，一批样本的所有特征值构成一个矩阵。几乎所有的算法公式都可以写成向量或矩阵的形式，机器学习中的算法模型通常所说的批（batch）训练，其底层的计算逻辑就是利用向量和矩阵的运算。由此可见，掌握向量和矩阵的相关知识，对于算法“内力”的修炼至关重要。既然涉及运算，自然会有相应的运算法则，那么向量和矩阵的运算法则是怎样的呢？

向量可以分为行向量和列向量，二者在本质上并无区别，只在进行运算时有差别。下面通过生活中的具体例子，逐一介绍与向量有关的运算法则。

某公司有n名员工，月工资用向量[a1, a2, ..., an-1, an]表示，其中ai表示第i个员工的月工资数额。

情形一：公司业绩不错，公司老板决定给每位员工涨薪20%，可以用**向量与标量之间的乘法**来计算涨薪后的工资，如表2-1所示，其中k为原工资倍数，即1.2。

表2-1 员工涨薪情况（一）

姓名 (a _i)	原工资/元	涨薪 (k)	现工资/元
小王	8 000	×(1+20%)	9 600
小李	12 000	×(1+20%)	14 400
.....		×(1+20%)	
小张	10 000	×(1+20%)	12 000

具体计算方法用数学符号表示如下，得到的结果仍然是向量。向量与标量之间的除法类似，可以理解为向量与标量的倒数相乘。

$$k \bullet \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} ka_1 \\ ka_2 \\ \vdots \\ ka_n \end{bmatrix}$$

情形二：公司业绩不错，根据员工完成业绩的不同，老板决定对不同的员工给出相应的工资涨幅。以绝对值涨幅为例（也可以是不同的比例涨幅），给第*i*位员工每月增加*b_i*元工资，可以用行（列）向量与行（列）向量之间的加法来计算涨薪后的工资，如表2-2所示。

表2-2 员工涨薪情况（二）

姓名 (<i>a_i</i>)	原工资/元	涨薪 (<i>b_i</i>)	现工资/元
小王	8 000	+1 000	9 000
小李	12 000	+800	12 800
.....			
小张	10 000	+900	10 900

具体计算方法用数学符号表示如下，计算结果仍然是向量。

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_n \end{bmatrix} \pm \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix} = \begin{bmatrix} a_1 \pm b_1 \\ a_2 \pm b_2 \\ a_3 \pm b_3 \\ \vdots \\ a_n \pm b_n \end{bmatrix}$$

向量与向量之间的减法与加法类似，计算结果仍然是向量。

情形三：老板在按照不同的涨幅比例给员工涨薪之后，想知道工资开支的总预算是多少，可以用行向量与列向量之间的乘法来计算。具体计算方法如下。

$$\begin{bmatrix} a_1 & a_2 & a_3 & \cdots & a_n \end{bmatrix} \cdot \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3 + \cdots + a_n b_n$$

由此可知，行向量与列向量相乘的结果是一个标量。

根据行向量与列向量相乘的例子可知，使运算成立的一个隐含条件是，行向量的列数必须与列向量的行数相等，如例子中的向量长度均为n，这有助于理解后面的矩阵运算（向量是一种特殊的矩阵）。列向量的列数与行向量的行数都等于1。一个列向量与一个行向量相乘，即可得到一个矩阵，具体计算方法如下。

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_n \end{bmatrix} \cdot [b_1 \ b_2 \ b_3 \ \cdots \ b_n] = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 & \cdots & a_1 b_n \\ a_2 b_1 & a_2 b_2 & a_2 b_3 & \cdots & a_2 b_n \\ \vdots & \vdots & \vdots & & \vdots \\ a_n b_1 & a_n b_2 & a_n b_3 & \cdots & a_n b_n \end{bmatrix}$$

在机器学习中，向量通常用于表示特定维度空间中不同维度的特征值，因此又称为特征向量。例如，一个二维特征向量[1, 2]表示两个特征，第一个特征维度的值为1，第二个特征维度的值为2。既然是空间，那就存在距离的概念，即同一个特征空间中不同特征向量之间存在着某种关系，如相似的特征表现。假设有另一个二维特征向量[2, 4]，其在两个特征维度上的值分别为2和4，可以发现，这个二维特征向量和前面的二维特征向量，在各个维度上的特征值大小恰好相差一倍，这说明二者的特征表现相似，只是程度不同。当然，在实际算法工程中，特征向量之间的关系会有更复杂的度量方式，如相似度、相关系数等。

矩阵运算和向量运算的运算法则类似。标量与矩阵的加法、减法、乘法，只需将标量与向量的每一个值分别相加、相减、相乘，得到的结果仍是矩阵。以标量与矩阵的乘法为例，计算方法如下。

$$k \cdot \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,n} \end{bmatrix} = \begin{bmatrix} ka_{1,1} & ka_{1,2} & ka_{1,3} & \cdots & ka_{1,n} \\ ka_{2,1} & ka_{2,2} & ka_{2,3} & \cdots & ka_{2,n} \\ \vdots & \vdots & \vdots & & \vdots \\ ka_{m,1} & ka_{m,2} & ka_{m,3} & \cdots & ka_{m,n} \end{bmatrix}$$

矩阵与向量之间只有乘法。如果矩阵与行向量相乘，那么行向量的列数必须与矩阵的行数相等，并且行向量必须位于乘号的左边，计算方法如下。

$$\begin{bmatrix} b_1 & b_2 & \cdots & b_m \end{bmatrix} \cdot \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,n} \end{bmatrix} \\ = \left[\sum b_i a_{i,1} \quad \sum b_i a_{i,2} \quad \sum b_i a_{i,3} \quad \cdots \quad \sum b_i a_{i,n} \right]$$

如果矩阵与列向量相乘，那么列向量的行数必须与矩阵的列数相等，并且列向量必须位于乘号的右边，计算方法如下。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,n} \end{bmatrix} \cdot \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix} = \begin{bmatrix} \sum a_{1,i}b_i \\ \sum a_{2,i}b_i \\ \vdots \\ \sum a_{m,i}b_i \end{bmatrix}$$

矩阵与矩阵相乘，乘号左边矩阵的列数必须与乘号右边矩阵的行数相等，计算方法如下。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix} \cdot \begin{bmatrix} b_{1,1} & b_{1,2} & \cdots & b_{1,m} \\ b_{2,1} & b_{2,2} & \cdots & b_{2,m} \\ \vdots & \vdots & & \vdots \\ b_{n,1} & b_{n,2} & \cdots & b_{n,m} \end{bmatrix} \\
 = \begin{bmatrix} \sum a_{1,i}b_{i,1} & \sum a_{1,i}b_{i,2} & \cdots & \sum a_{1,i}b_{i,m} \\ \sum a_{2,i}b_{i,1} & \sum a_{2,i}b_{i,2} & \cdots & \sum a_{2,i}b_{i,m} \\ \vdots & \vdots & & \vdots \\ \sum a_{m,i}b_{i,1} & \sum a_{m,i}b_{i,2} & \cdots & \sum a_{m,i}b_{i,m} \end{bmatrix}$$

假设A、B、C表示矩阵。

矩阵之间的加法满足交换律，计算公式如下。

$$A_{i \times j} + B_{i \times j} = B_{i \times j} + A_{i \times j}$$

矩阵之间的乘法满足结合律，计算公式如下。

$$A_{i \times j} \cdot B_{j \times i} \cdot C_{i \times k} = A_{i \times j} \cdot (B_{j \times i} \cdot C_{i \times k})$$

在线性代数中，关于向量和矩阵的知识点还有很多，包括单位矩阵、对角矩阵、对称矩阵等，这些知识点在PCA降维、协同过滤和矩阵分解等算法中都有所运用，下文只对这些概念进行简单介绍，感兴趣的读者可以自行深入拓展学习。

单位矩阵是一个方阵，这个方阵从左上角到右下角的对角线（称为主对角线）上的元素均为1，其他元素均为0，示例如下。

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

对角矩阵也是一个方阵，这个方阵除主对角线上的元素外，其他元素均为0，主对角线上的元素可以不为1，示例如下。由此可知，单位矩阵属于对角矩阵。

$$\begin{bmatrix} a_{1,1} & 0 & 0 & 0 \\ 0 & a_{2,2} & 0 & 0 \\ 0 & 0 & a_{3,3} & 0 \\ 0 & 0 & 0 & a_{4,4} \end{bmatrix}$$

对称矩阵也是一个方阵，这个方阵只需满足以主对角线为对称轴，两边对称位置的元素相等，示例如下。由此可知，单位矩阵和对角矩阵都属于对称矩阵。

$$\begin{bmatrix} a_{1,1} & b_1 & b_2 & b_4 \\ b_1 & a_{2,2} & b_3 & b_5 \\ b_2 & b_3 & a_{3,3} & b_6 \\ b_4 & b_5 & b_6 & a_{4,4} \end{bmatrix}$$

2.2 排列组合

大部分读者应该对排列组合比较熟悉，因为它在高中的数学课上出现过。与解析几何相比，排列组合的公式简单，并且理解起来比较容易、直观。

排列组合包含两个概念：排列和组合。举个例子，假设某小组一共有5个人，有以下两个问题。

① 从该小组中选出3个人站成一排，一共有多少种排法？

② 从该小组中选出3个人打扫卫生，一共有多少种选法？

对比问题①和问题②可以发现，问题①需要考虑顺序，问题②不需要考虑顺序。这便是排列与组合的区别。问题①要计算排列种类数，排列用字母P（Permutation）或A（Arrangement）表示；问题②要计算组合种类数，组合用字母C（Combination）表示。在计算时，组合种类可以看作排列种类的子集。

可以用形式化的语言描述排列问题，即n个人中计算m个人的排列。继续思考前面的例子，如果要从5个人中选3个人站成一排，那么应该如何选择并排列？首先，从5个人中选择1个人站在位置1上，有5种选法；然后从剩下的4个人中选择1个人站在位置2上，共有4种选法；最后从剩下的3个人中选出1个人站在位置3上，一共有3种选法；因此共有60（ $5 \times 4 \times 3$ ）种排列方法。这种计算方法就是阶乘。使用字母A表示排列，具体计算公式如下。

$$A_n^m = n(n-1)(n-2) \cdots (n-m+1) = \frac{n!}{(n-m)!}$$

由于组合不用考虑选出来的人的排列顺序，因此，只需用计算出来的排列方法总数除以选出人数的排列方法总数。所有排列方法叫作全排列，n个人的全排列总数等于n的阶乘。对于前面的例子，组合方法总数就是10（ $60 / 3!$ ）种。使用字母C表示组合，具体计算公式如下。

$$C_n^m = \frac{A_n^m}{m!} = \frac{n!}{m!(n-m)!} = C_n^{n-m}$$

在AI算法中，排列组合主要用于计算特征组合（交叉）后的特征总数。

2.3 高等数学

看到这节，你可能会问：搞AI算法还需要学习高等数学？大家知道，高等数学涵盖的知识点比较多，并且比较难以理解。别担心，本书只讲解高等数学中与AI算法最相关的两个知识点——**导数**和**梯度**。这两个知识点在AI算法的公式推导中至关重要，有助于帮助我们理解算法模型在实际应用中起作用的原因。

2.3.1 导数

简单地说，导数主要用于描述函数图像中某一点切线的倾斜程度，这个倾斜程度又称为斜率。所谓连续函数，就是函数图像随着x值的连续变化，y值的变化也是连续的。连续函数不一定处处可导，可导函数的图像一定是处处连续的。

三角函数处处连续，同时处处可导。以正弦函数 $y=\sin x$ 为例，函数图像上的每个点都是光滑的，可以计算函数图像在该点处的切线斜率，如图2-1所示。

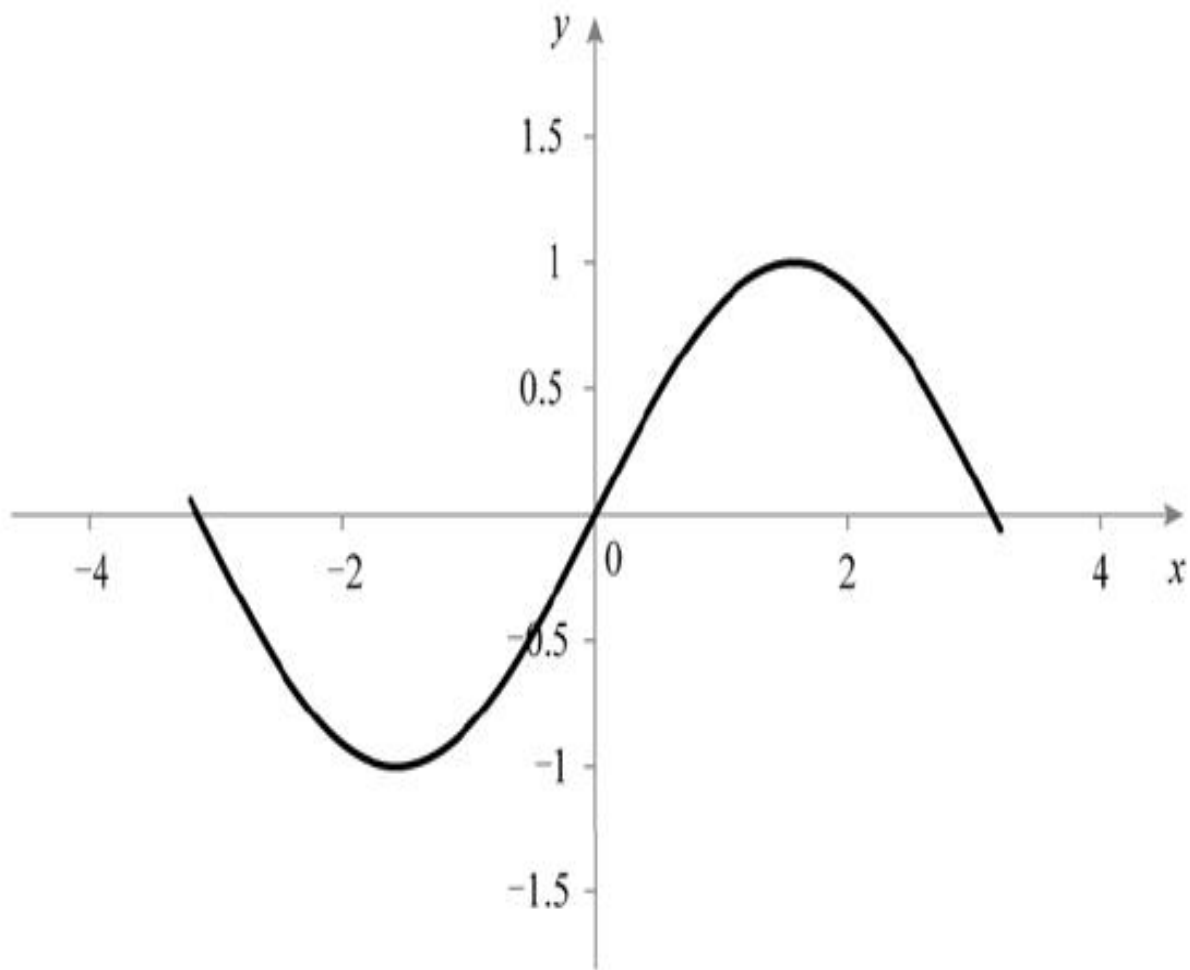


图2-1 正弦函数 $y = \sin x$ 的图像

如图2-2所示，用线性绝对值函数 $y = |x|$ 作为对比，可以看到该函数图像同样处处连续。但是，在坐标为 $(0, 0)$ 的点的位置虽然连续，却无法计算出切线的斜率，因此该函数是不可导的。

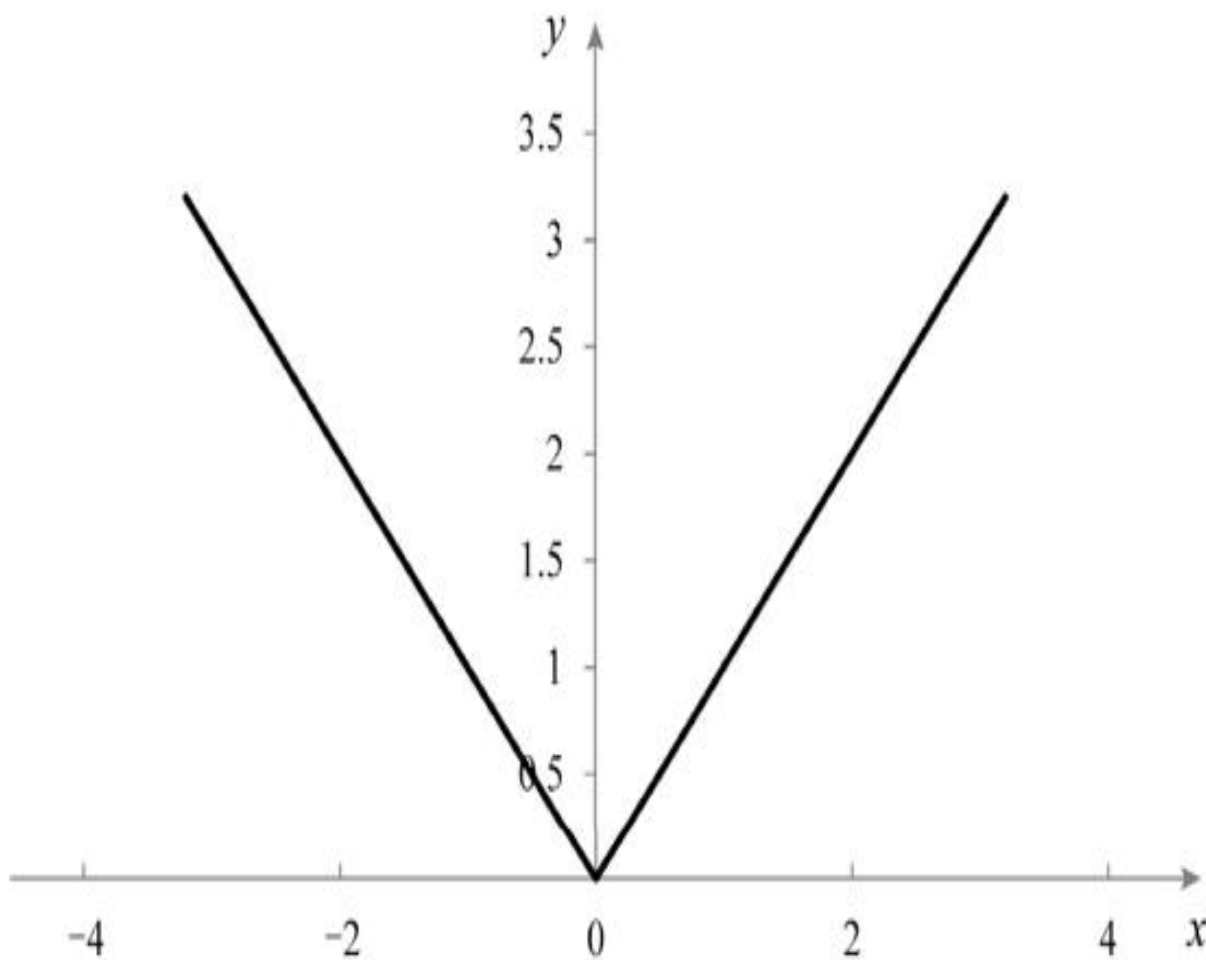


图2-2 线性绝对值函数 $y=|x|$ 的图像

在中学数学中，通常使用极限求解导数。在AI算法中不用那么复杂，只需掌握以下两点。第一，弄清楚导数的意义。导数是函数切线的斜率，因此可以用于表示函数值变化的快慢和趋势。例如，朝着变大的方向还是变小的方向，变大变小的速度如何，等等。第二，记住AI算法中常用的几个求导公式及导数的运算法则。下面介绍AI算法中常用的几个求导公式。

常数求导公式如下。

$$(C)'=0$$

幂函数的求导公式如下。

$$(x^n)' = nx^{n-1}$$

指数函数求导公式如下。

$$(a^x)' = a^x \ln a$$

对数函数求导公式如下。

$$(\log_a x)' = \frac{1}{x \ln a}$$

自然对数函数求导公式如下。

$$(\ln x)' = \frac{1}{x}$$

现有函数 u 和函数 v ，函数四则运算的求导法则如下。

函数加减法的求导法则如下。

$$(u \pm v)' = u' \pm v'$$

函数乘法的求导法则如下。

$$(uv)' = u'v + uv'$$

函数除法的求导法则如下。

$$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$$

此外，在使用神经网络的AI算法中，神经元的层数一般不止一层，相邻层的输出是下一层的输入，即一个函数的因变量是另一个函数的自变量，在求导过程中会用到一种称为**链式法则**的复合函数求导方法。复合函数 $f(x)=g[h(x)]$ 的链式法则如下。

$$f'(x)=g'[h(x)]h'(x)$$

此外，AI算法中的特征维数通常不止一维，即多维自变量。包含多维自变量的函数称为多维函数，多维函数的导数称为偏导数。偏导数的求法很简单，在对多维函数中的某一维变量求导时，将其他维变量视为常量即可。

2.3.2 梯度

函数的导数值又称为梯度，梯度是带有方向的，多维自变量所在的每个维度均代表一个方向。在AI算法中，特征数据通常是多维的，每个维度都有一个梯度，不同维度上的自变量对应的偏导数大小不一样，梯度的大小也就不一样。在算法训练过程中常说的某个特征学得快一点，就是因为这个特征在维度方向上的梯度较大。梯度传递是算法训练过程中的一个重点，其中的梯度消失和梯度爆炸问题会严重影响模型的最终效果，这部分内容将在第4章详细讲解。

2.4 概率与统计

2.4.1 名词解释

在学习新概念之前，掌握常见的名词解释，好比在下厨炒菜时能熟练说出调料名和菜名一样，有助于我们加深对每个环节的了解，让学习过程变得更加顺畅。此外，准确理解并说出相应的名词概念，也是一种专业表现。

1. 概率统计

研究自然界中随机现象统计规律的数学方法，称为概率统计，又称为数理统计。概率统计主要的研究对象为随机事件、随机变量及随机过程。

2. 随机事件

概率统计统计的是随机事件发生的概率。随机事件是指在大量的重复实验中发生的概率具有某种规律的事件。例如，抛掷同一枚硬币，“正面朝上”就是一个典型的随机事件。随机事件通常简称为事件。事件之间的基本关系如表2-3所示。

表2-3 事件之间的基本关系

符 号	数 学 描 述	意 义
Ω	事件的全集	必然发生的事件
$\emptyset$	空集	不会发生的事件
$A \in \Omega$	子集	表示一个事件 A
$A \in B$	集合 B 包含集合 A	A 发生一定导致 B 发生
$A = B$	集合相等	A 发生导致 B 发生, B 发生也导致 A 发生
$A \cup B$	A 和 B 的并集	A 和 B 至少一个发生
$A \cap B$	A 和 B 的交集	A 和 B 同时发生
$A \cap B = \emptyset$	A 和 B 的交集为空集	A 和 B 互斥, 不会同时发生, 即独立事件
$A - B$	A 和 B 的差集	A 发生, B 不发生

和集合运算一样，事件之间的运算也具备以下3个性质。

- 交换律： $A \cup B = B \cup A$ ， $A \cap B = B \cap A$ 。
- 结合律： $A \cup B \cup C = A \cup (B \cup C)$ ， $A \cap B \cap C = A \cap (B \cap C)$ 。
- 分配律： $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ ， $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ 。

3. 随机变量

事件会发生就会有结果，可以使用随机变量描述随机事件可能发生的结果，如抛十次硬币正面朝上的次数、某个地铁站每天的上车人

数、灯管的寿命、某个班级学生的身高和体重等。

4. 离散型随机变量和连续型随机变量

抛硬币正面朝上的次数和地铁站的上车人数都是可数的，最小单位是1，在数学中称为离散，这类随机变量称为离散型随机变量。灯管寿命、人的身高体重的取值是连续的，没有最小单位，在数学中称为连续，这类随机变量称为连续型随机变量。

5. 随机过程

简单而言，随机过程就是随机变量随某个参数变化的过程，这个参数通常是时间，如周一至周五某个地铁站随时间变化的候车人数。

6. 数学期望

抛掷一枚质地均匀的硬币，正反两面出现的概率各占一半，在进行足够数量的抛掷实验后，正反两面实际出现的次数会各自接近50%，这个50%就是数学期望。数学期望反映的是随机变量的平均值，其计算方法是用实验中事件的发生概率乘重复实验的总次数，一般用字母 μ 或E表示。

7. 方差

顾名思义，方差是某种差值，主要用于衡量随机变量的实际取值与其数学期望之间的偏离程度，计算公式如下。

$$E\{[X-E(X)]^2\}$$

8. 标准差

标准差是方差的算术平方根。

9. 协方差

协方差是指方差的多变量扩展，主要用于衡量两个随机变量X和Y之间的总体误差，计算公式如下。

$$E\{[X-E(X)][Y-E(Y)]\}$$

10. 概率分布

概率分布主要用于描述随机变量在随机事件中表现出来的一种状态，是概率关于随机变量的函数。

11. 概率密度

可以将概率密度直观地理解为事件发生的结果落在随机变量某段取值范围内的概率。从数学角度来说，就是概率分布函数在某段区间内的积分。有读者可能已经发现，只有连续型随机变量才有概率密度，离散型随机变量对应的概念为概率质量。

12. 条件概率

顾名思义，条件概率是指事件满足一定条件所发生的概率。在通常情况下，两个事件A和B， $P(A | B)$ 表示在事件B发生的前提下，事件A发生的概率。

2.4.2 概率分布

日常生活中到处都有概率分布的身影，本节按照从易到难思路，给大家讲解最常见的几种概率分布。

1. 伯努利分布

伯努利分布是最简单的概率分布，随机变量只能是离散型随机变量，只有两种取值（事件成功取1，失败取0）。生活中的伯努利分布有很多。例如，明天是否下雨，雷霆队下一场对勇士队的比赛是否能赢，等等。

伯努利分布的数学期望如下。

$$\mu=p$$

伯努利分布的概率质量函数如下。

$$f(x | p) = p^x (1 - p)^{1-x}, x = 0, 1$$

2. 均匀分布

均匀分布的随机变量可以是离散型随机变量，也可以是连续型随机变量，随机变量不同取值出现的概率相等，如摇骰子就是典型的均匀分布。

均匀分布的数学期望如下。

$$\mu = \frac{a + b}{2}$$

均匀分布的连续型随机变量的概率密度函数如下。

$$f(x) = \frac{1}{b - a}, a < x < b$$

均匀分布的连续型随机变量的概率密度函数图像如图2-3所示（假设 $b=2$ ， $a=1$ ）。

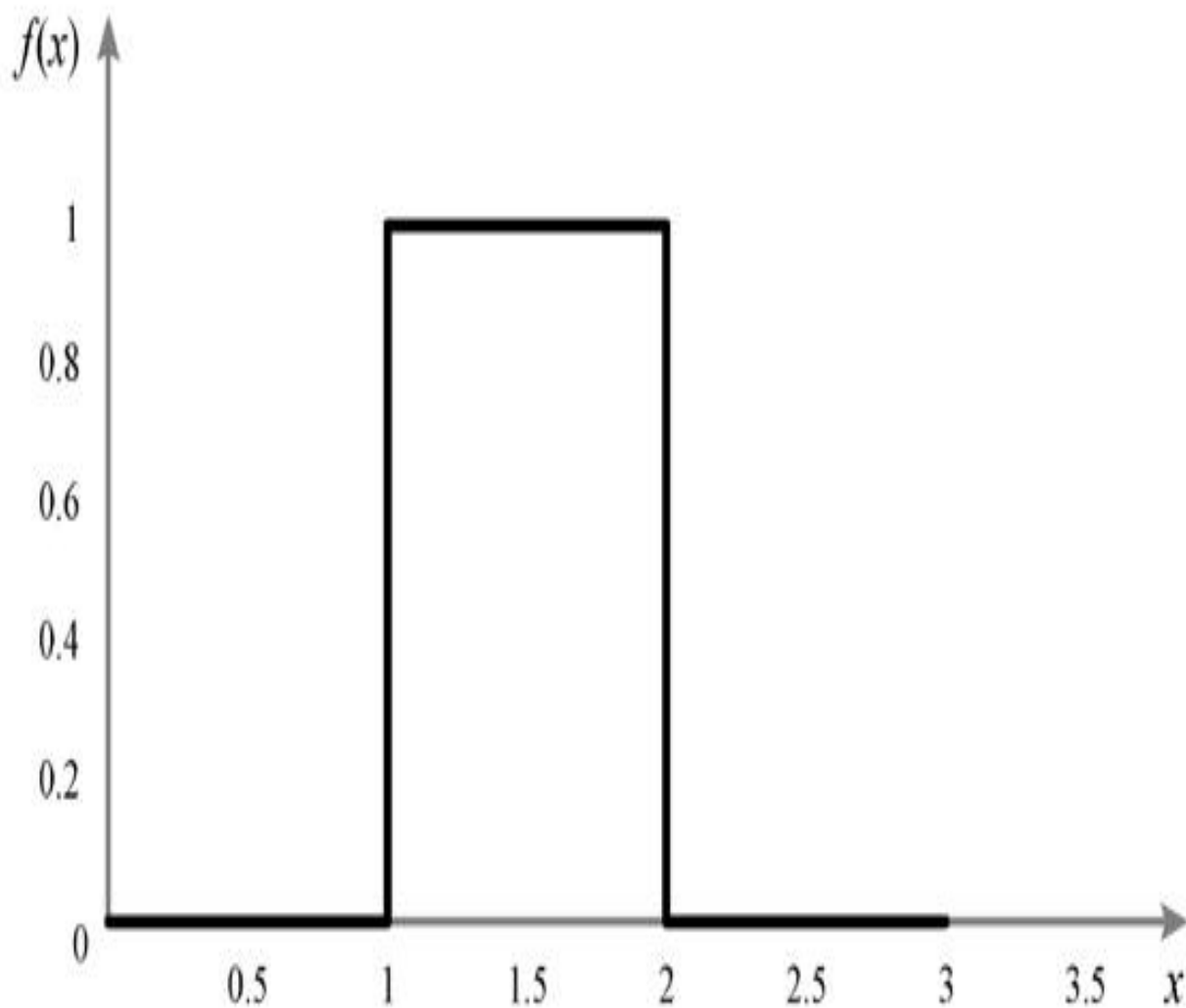


图2-3 均匀分布的连续型随机变量的概率密度函数图像

3. 二项分布

顾名思义，二项分布就是取值只有正、负两种结果的概率分布，因此其随机变量只能是离散型随机变量。用数学语言描述就是，关于 n 个独立的二值实验中有多少次为正的离散型概率分布。

二项分布最典型的例子是抛硬币实验，抛n次硬币，k次正面朝上的概率对应的数学期望如下。

$$\mu=np$$

根据排列组合，二项分布的概率质量函数如下。

$$f(x = k | p) = C_n^k p^k (1 - p)^{n-k}$$

当p=0.5、n=12时，二项分布的概率质量函数图像如图2-4所示。

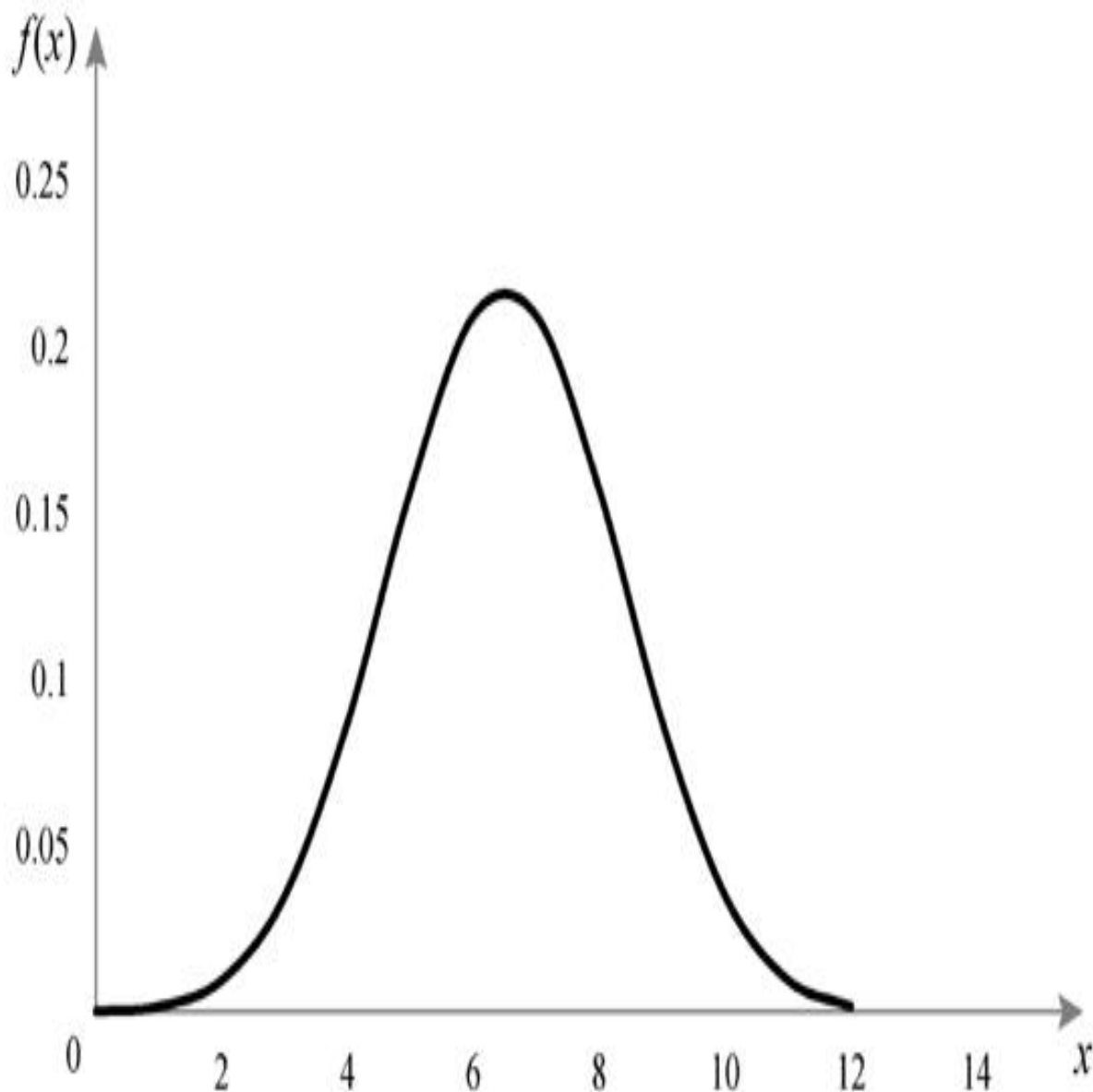


图2-4 二项分布的概率质量函数图像 ($p=0.5$, $n=12$)

4. 几何分布

和二项分布一样，几何分布的随机变量也只能是离散型随机变量。与二项分布不同，几何分布关心的是事件（或实验）发生 n 次，在第 x 次取得成功的概率。典型的例子是打靶，在 n 次打靶过程中第一次命中靶心的概率。几何分布的数学期望如下。

$$\mu = \frac{1}{p}$$

几何分布的概率质量函数如下。

$$f(x) = (1 - p)^{x-1} p$$

当 $p=0.5$ 时，几何分布的概率质量函数图像如图2-5所示。

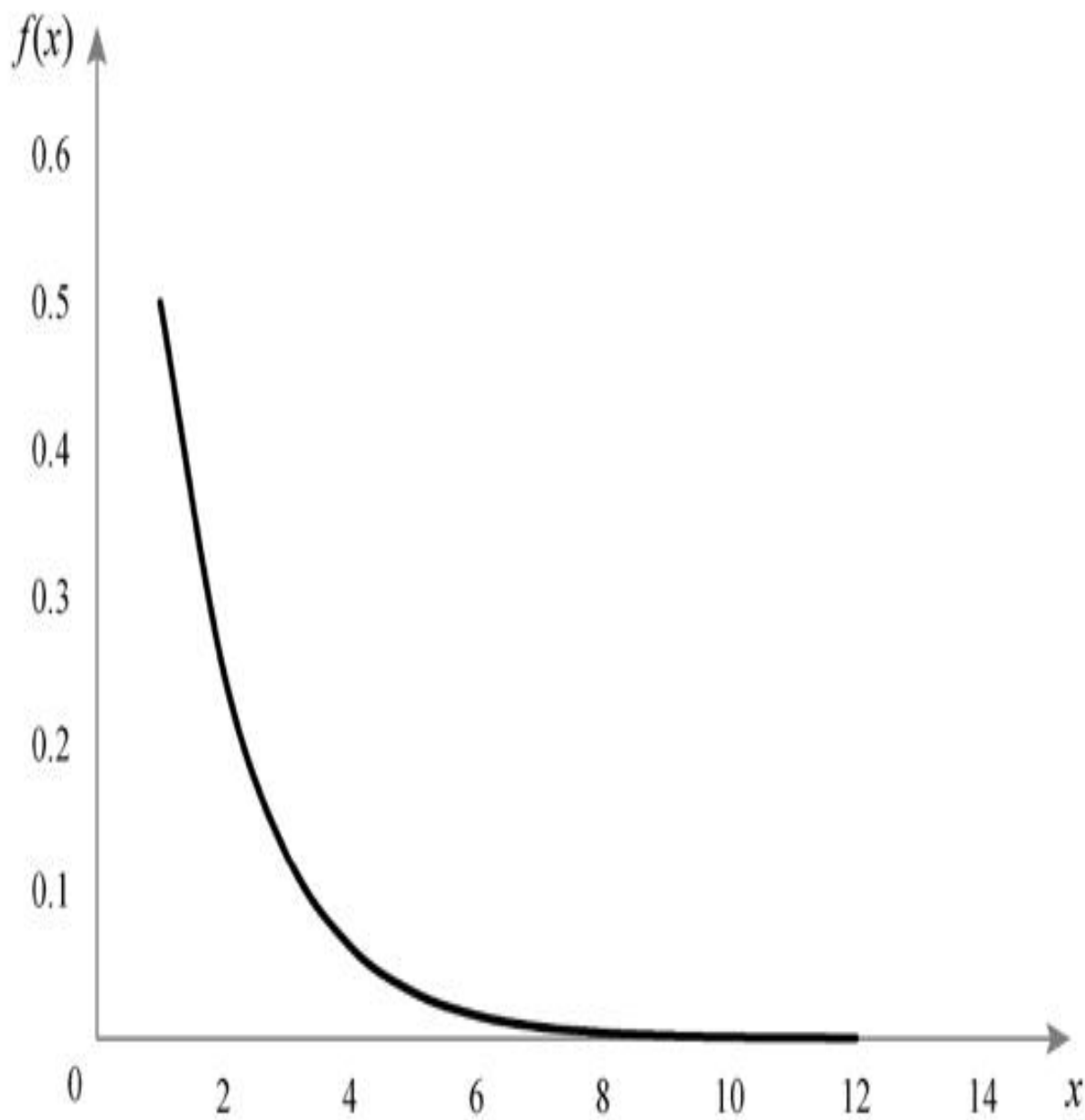


图2-5 几何分布的概率质量函数图像 ($p=0.5$)

5. 泊松分布

在计算二项分布的概率分布时，如果需要计算发生 k 次的概率，那么在二项分布中必须事先知道一个全局的 n 。然而，在实际问题中很难或无法预先知道对应的 n 是多少，如潜在乘坐公交车的乘客总数、潜在需要去银行办理业务的客户总数、潜在包子铺顾客总数等。

总数 n 未知，难道就没有办法求概率 P 了吗？聪明的你应该已经联想到了取 n 的极限来求解 P 。没错，取 n 的极限来求解 P 正是泊松分布的推导过程。

根据二项分布的数学期望

$$\mu=np$$

可以推导出

$$p = \frac{\mu}{n}$$

泊松分布的推导过程如下。

$$P = \lim_{n \rightarrow \infty} C_n^k \times p^k \times (1-p)^{n-k}$$

$$P = \lim_{n \rightarrow \infty} C_n^k \times \left(\frac{\mu}{n}\right)^k \times \left(1 - \frac{\mu}{n}\right)^{n-k}$$

$$P = \lim_{n \rightarrow \infty} \frac{n(n-1)(n-2)\cdots(n-k+1)}{k!} \times \frac{\mu^k}{n^k} \times \left(1 - \frac{\mu}{n}\right)^{n-k}$$

$$P = \lim_{n \rightarrow \infty} \frac{\mu^k}{k!} \times \frac{n(n-1)(n-2)\cdots(n-k+1)}{n^k} \times \left(1 - \frac{\mu}{n}\right)^{-k} \left(1 - \frac{\mu}{n}\right)^n$$

将上式各部分拆开计算，根据指数e的下述极限计算方法，代入极限求解，P最终变成了只与k、μ相关的式子。

$$\lim_{n \rightarrow \infty} \frac{n(n-1)(n-2)\cdots(n-k+1)}{n^k} \times \left(1 - \frac{\mu}{n}\right)^{-k} = 1$$

$$\lim_{n \rightarrow \infty} \left(1 - \frac{\mu}{n}\right)^n = e^{-\mu}$$

$$P = \frac{\mu^k}{k!} \times 1 \times e^{-\mu} = \frac{\mu^k}{k!} e^{-\mu}$$

根据泊松分布的推导过程可知，**泊松分布是二项分布的极限形式**，因此随机变量也只能是离散型随机变量。泊松分布的概率质量函数如下。

$$f(x = k | \mu) = \frac{\mu^k}{k!} e^{-\mu}$$

当 $\mu=0.9$ 时，泊松分布的概率质量函数图像如图2-6所示。

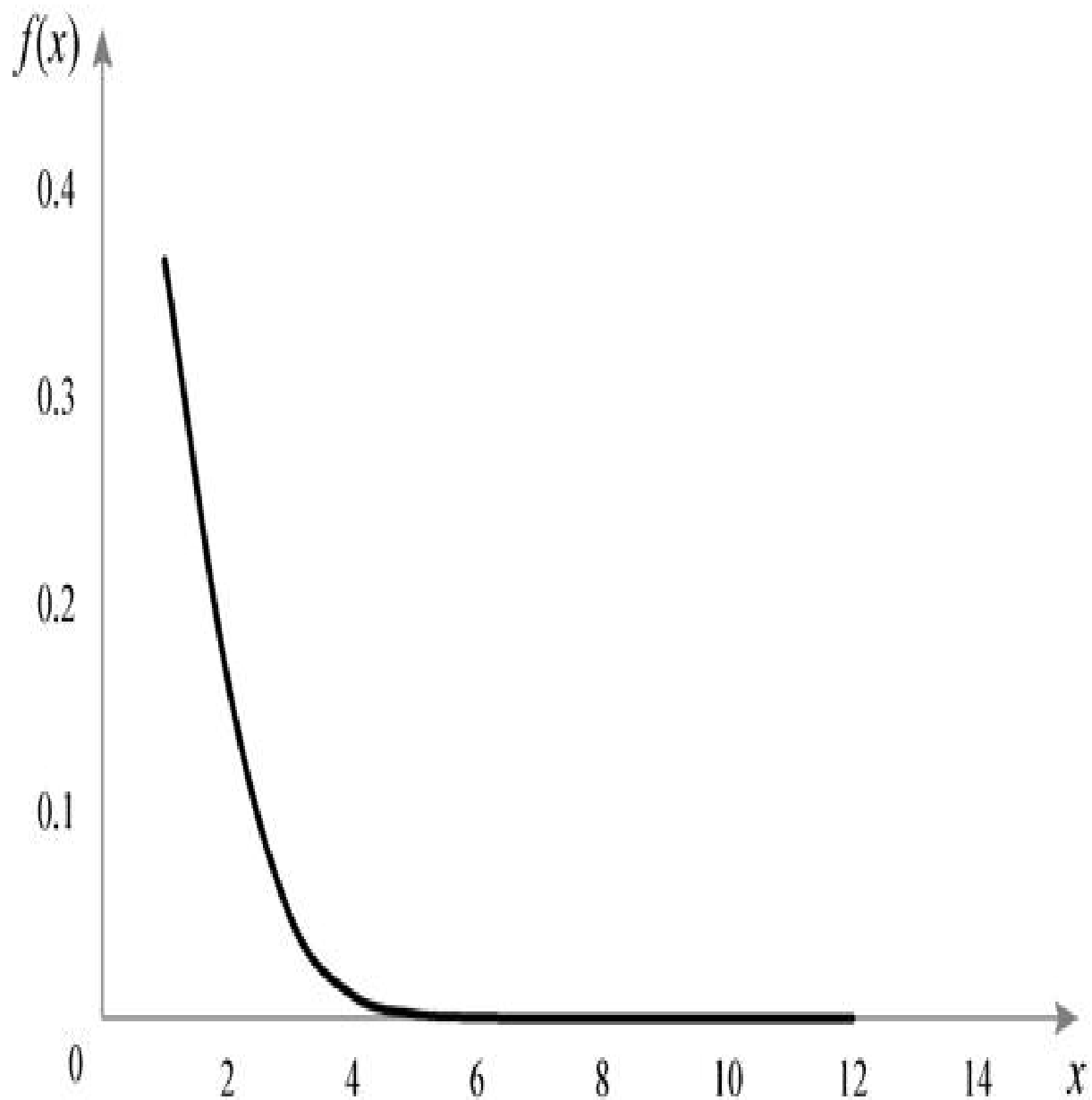


图2-6 泊松分布的概率质量函数图像 ($\mu=0.9$)

根据泊松分布的概率质量函数，已知均值 μ ，不需要知道总数 n ，就可以求得 k 值对应的概率。

6. 指数分布

对于泊松分布，如果将时间长度纳入考虑，就可以描述在某段时间内，事件的发生概率，而指数分布主要用于描述事件的时间间隔概率，如一家医院婴儿的出生时间间隔、一家网站的访问时间间隔等。因此，指数分布的随机变量是离散型随机变量，数学期望如下。

$$\mu = \frac{1}{\lambda}$$

指数分布的概率质量函数可以通过泊松分布推导而来，具体如下。

$$f(x | \lambda) = \lambda e^{-\lambda x}, x > 0$$

当 $\lambda=0.8$ 时，指数分布的概率质量函数图像如图2-7所示。

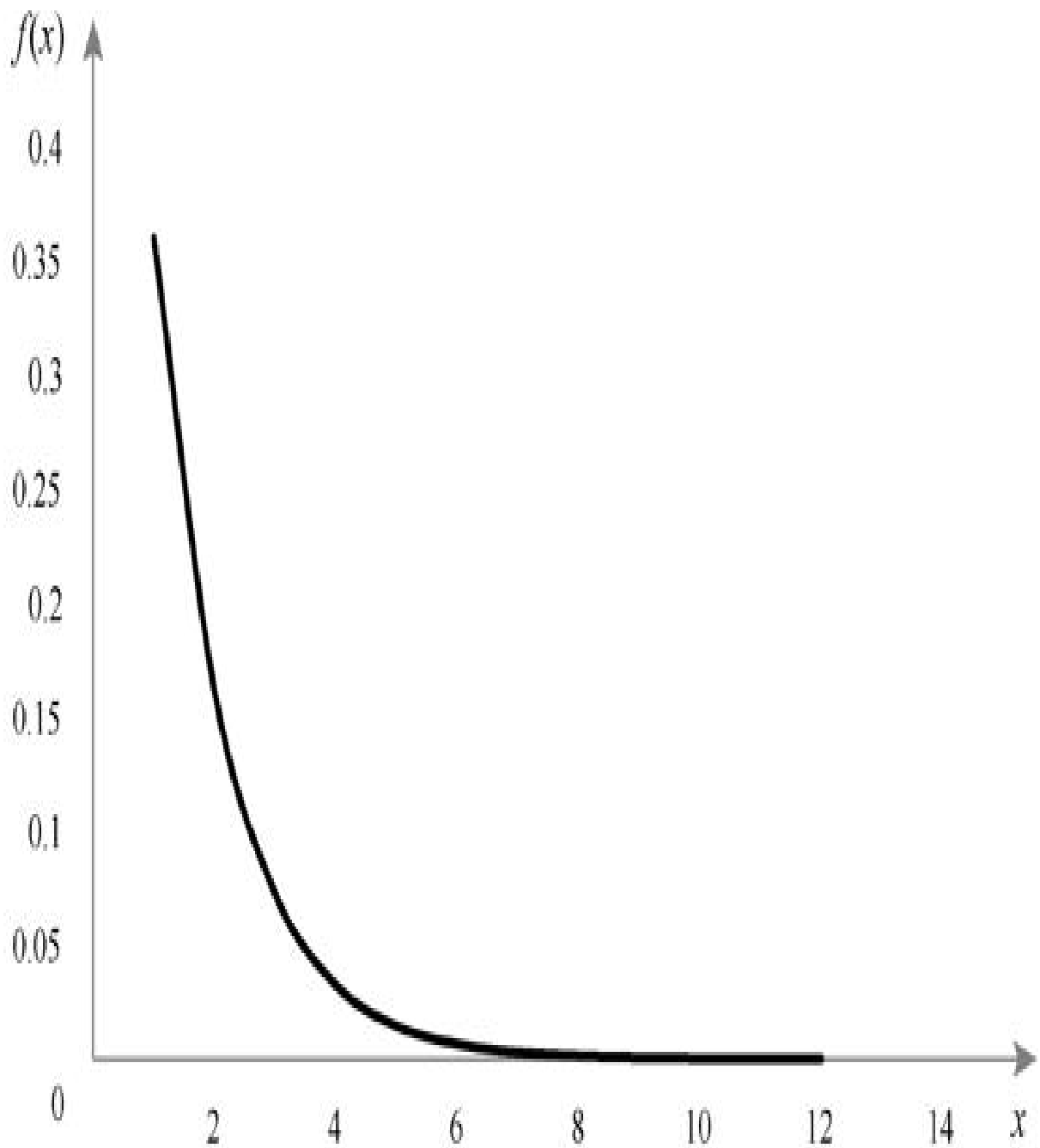


图2-7 指数分布的概率质量函数图像 ($\lambda=0.8$)

7. 高斯分布

高斯分布即正态分布，是生活中出现频率很高的一种概率分布，如一所学校所有学生的身高和体重、某一科考试的分数都服从高斯分

布。虽然称为高斯分布，但并不是数学家高斯提出的。其实早在高斯之前就有正态分布了，只是高斯率先将正态分布应用于天文学研究，他的研究对后世影响较大，因此称为高斯分布。高斯分布的概率密度函数如下。

$$f(x | \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

高斯分布的概率密度函数图像如图2-8所示。

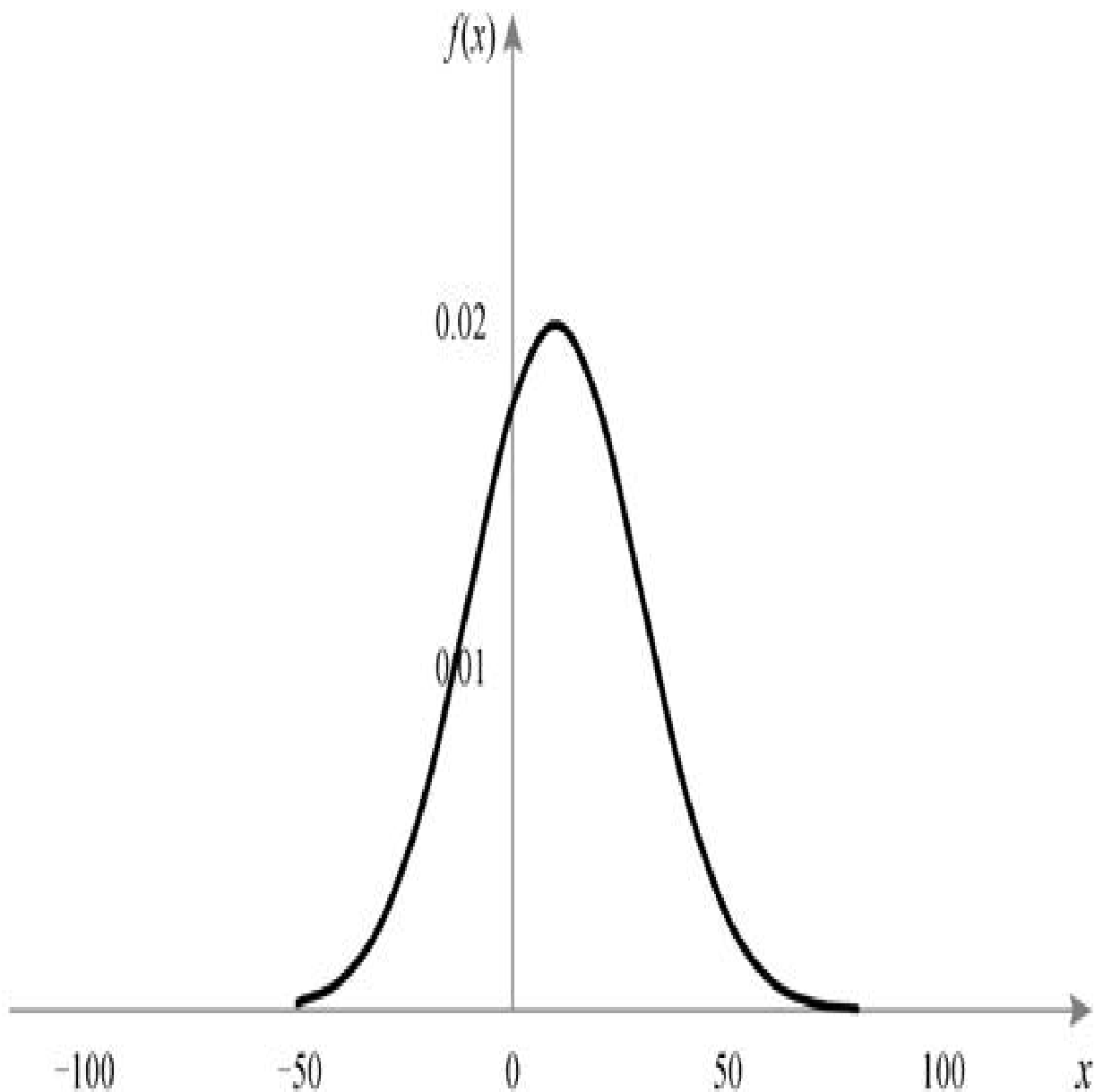


图2-8 高斯分布的概率密度函数图像

2.5 最优化原理

最优化原理是应用数学的一个分支，是关于最优设计、最优控制、最优管理问题的理论与方法。AI算法要解决的大部分问题都是在寻找一组适配模型的最优参数解，都可以归结为最优化问题。最优化

问题就是将问题先用最优化的方法建立数学模型，再以最优化的方式求解。

最优化问题分为有约束的最优化问题和无约束的最优化问题。有约束的最优化问题分为等式约束优化问题和不等式约束优化问题，这类问题通常使用拉格朗日乘数法和KKT条件解决；无约束的最优化问题分为全局最优化问题和局部最优化问题，这类问题可以使用牛顿法和最小二乘法解决。

最优化问题还有一个分支，叫作**凸优化**。凸优化的局部最优解就是全局最优解。而大部分AI算法问题，其实都是在解决凸优化问题。解决凸优化问题的方法是梯度下降法，大致步骤如下：首先初始化算法模型的参数权重，然后依据样本数据计算误差和梯度，最后根据梯度下降法优化各参数权重，直到误差下降到可接受范围内，训练结束。

最优化问题的学术理论要求较高，本书的侧重点并不在于此，感兴趣的读者可以自行深入学习。

2.6 动脑时刻

1. 石头平时比较喜欢自己在家里榨果汁，主要使用5种原材料：芒果、草莓、香蕉、牛奶和木瓜。石头在榨果汁时常用的4种配方如表2-4所示，5种原材料在不同超市的售卖价格如表2-5所示。以矩阵运算的形式计算这4种果汁配方的成本分别是多少。

表2-4 石头在榨果汁时常用的4种配方（单位：份）

种 类	配 方			
	配 方 A	配 方 B	配 方 C	配 方 D
芒果	1	0	1	1
草莓	0	2	1	1
香蕉	1	0	0	1
牛奶	1	1	0	1
木瓜	0	1	1	1

表2-5 5种原材料在不同超市的售卖价格（单位：元/份）

种 类	季 度		
	超 市 A	超 市 B	超 市 C
芒果	5	6	3
草莓	12	13	10
香蕉	2	3	2
牛奶	2	3	2
木瓜	25	22	20

2. 假设模型中有5个特征，通过特征之间的任意交叉组合，最多可以增加多少个特征？

3. 5个人围成一圈做游戏，一共有多少种不同的围法？

4. 在概率统计中，标准差是方差的算术平方根，既然有了方差，为什么还需要标准差？

5. 已知包子铺历史每天平均卖出 $\mu=100$ 个包子，为了保证未来每天不够卖的概率低于10%，每天最少需要准备多少个包子？（已知服从泊松分布）

6. 某医院平均每小时出生3个婴儿，在接下来的第15~30分钟，有婴儿出生的概率是多少？（已知服从指数分布）

2.7 本章小结

本章首先讲解了线性代数中的向量和矩阵，这两个概念对AI算法而言是最基本的数据承载体。然后讲解了与AI算法有关的排列组合及高等数学中的几个知识点。紧接着详细讲解了概率与统计。如果AI算法从业者对数据具备良好的概率与统计素养，那么在解决实际问题时，会将模型运用得更深刻、全面。最后简单讲解了什么是最优化原理。

本章内容都是与算法有关的数学基础知识，在语言描述上难免会有些晦涩。考虑到AI算法中数学知识的实际使用概率，以及本书希望面向的广大受众，本章只讲解了相关数学基础知识最核心的知识点，章节末尾的动脑时刻也尽量避免了生僻的数学公式。这样做的目的只有一个，就是让读者尽可能快地学习算法世界中的数学知识，以便在后面可以快速进入算法工程的实战中。

关键词回顾

标量

向量

行向量

列向量

矩阵

单位矩阵

对角矩阵

对称矩阵

排列组合

导数

梯度

随机事件

随机变量

离散型随机变量

连续型随机变量

随机过程

数学期望

方差

标准差

协方差

概率密度

条件概率

概率分布

伯努利分布

均匀分布

二项分布

几何分布

泊松分布

指数分布

高斯分布

凸优化

第3章 算法之招式

在《倚天屠龙记》中，九阳神功是上乘的内功心法。但如果张无忌只学会九阳神功，不会后面的乾坤大挪移等武功，充其量只能做到神功护体，而不能与人争锋。算法世界同样如此，即使数学学得再好，在算法世界中没有合适的运用场景，亦是徒劳。本章要讲解的就是算法世界中以数学为内力驱动的招式。反之，如果AI相关从业者的数学基础较差，就很难将这些招式运用自如，在学习后续算法的武功秘籍时也会比较吃力。

在解决实际问题的过程中，我们往往会遇到这样的问题：算法过程中的数据如何存储？用什么结构存储效率更高？数据有什么高效的查询和排序方法？某些特定问题是否有相应的算法可以将其巧妙地解决？如何衡量算法运行需要耗费的运算时间和存储空间？相信读者在学完本章的算法招式后，可以找到这些问题的答案。所谓算法招式，其实就是数据结构和基础算法（为了和机器学习算法区分开，本章讲解的算法统称为基础算法）。本章根据数据结构和基础算法之间相辅相成的关系，先讲解数据结构的相关知识，再讲解常用的基础算法。按照从易到难的顺序，尽量采用通俗易懂的语言，帮助读者修炼好算法世界的招式。

3.1 数据结构

如果数据结构是土壤，基础算法就是从这片土壤中生长出来的一株株植物。

3.1.1 数组与链表

数组与链表是最简单的两种数据结构。在不同的场景应用中，二者各有所长，在性能表现方面各有优缺点。

1. 数组

数组（Array）是一种顺序存储结构，其中的数据元素是采用相邻连续分配的方式存储的。由于是连续存储，因此只要知道某个元素在数组中的编号（下标）及数组的起始位置，相加计算偏移就能访问这个元素的值。数组按空间维度划分，可以分为一维数组、二维数组、多维数组，一维数组的存储图例如图3-1所示。

图3-1 一维数组的存储图例

上述描述或许比较抽象，下面我们以电影院选座位为例进行说明。小王、小李和小张一起去电影院看电影，在买票选座位时希望选择同一排的连号座位，这种座位分配方式就是一维数组的存储方式。假设小王、小李、小张按从左至右的顺序依次入座，编号分别为0、1、2。由于3个人挨着坐，因此只需知道最左边小王的座位号，通过对编号进行相加，就能知道另外两个人的座位号了。其实，一个影厅的座位可以理解成由一维数组（每一排）组成的二维数组，所谓的“几排几号”其实就是二维数组的索引方式。如果影厅有多层，就可以理解为三维数组，座位索引就是“几层几排几号”。

数组的这种分配方式存在一个很明显的问题，就是在插入新元素时，即使相邻存储空间足够，也需要从插入位置整体向后移动各元素；而在相邻存储空间不够时，需要在其他空余位置申请存储空间，

由于不在同一个连续的存储区域，因此新元素的添加和查询会比较慢。对于这个问题，一种最直观的解决方法是从一开始就分配足够的数组空间。这种方法存在如下两个缺点：

- 额外申请的存储空间可能永远用不上，在申请占用后无法给别的程序使用。

- 无法保证分配足够的数组空间，因为总会有不够用的时候。

针对这两个缺点，另一种数据结构形式——链表便出现了。

2. 链表

还是以电影院选座位为例。小王、小李和小张一起去看电影，小王是小李的朋友，有小李的联系方式，小李是小张的朋友，有小张的联系方式。到了电影院发现最近一场电影的放映时间已经没有连号的三个座位了，于是三人买了分开的座位。这种入座方式就是链表（List）的存储方式，不要求数据存储的连续性，有位置即可。在电影放映过程中，小王接到老板发来的紧急消息，需要他立刻返回公司。于是，小王将先行离开的消息通知了小李，小李又将该消息通知了小张，这种消息传递的方式就是链表的数据访问方式，即链式访问，小王只能联系小李，不能联系小张，只能由小李联系小张。

根据上述例子我们了解到，链表是一种链式存储结构，其中的数据元素在存储时不要求相邻、连续，当添加新的数据元素时，只需开辟任意一个新的存储空间，然后将链表末尾元素的指针指向这个新元素的位置。这样既可以避免出现元素频繁移动的问题，又可以非常快速地查询。链表和数组一样，支持多维结构（多维链表）。不仅如此，链表还支持双向查询（双向链表）。单链表的存储结构图例如图3-2所示。

图3-2 单链表的存储结构图例

任何事情都有得有失。链表虽然在元素插入方面占据优势，但不能随意读取任意位置的元素值。由于链表的前后节点相连，因此当需要读取某个编号的元素值时，必须从第一个元素开始访问，向后依次查询，直到找到该编号的元素值为止。而数组由于是连续的存储空间，通过下标即可快速获取当前编号的元素值。例如，进行随机取数，数组的效率比链表的效率要高很多。

概括数组和链表的优缺点：数组的查询效率高、扩展效率低，链表的查询效率低、扩展效率高。

3.1.2 队列和栈

队列和栈，与数组、链表一样都是顺序存储的数据结构。除数据存储的功能外，队列和栈分别提供了不同的数据检索能力，用于满足不同场景的需求。

1. 队列

队列（Queue）的数据插入特点是**先进先出**（First In First Out，FIFO）。FIFO在计算机的很多方面都有应用，如计算机内存页的替换策略、网络路由器的流量分配策略等。直观地理解，队列的例子就是生活中的各种排队。例如，在火车站出站后打出租车需要排队，按先来后到的顺序依次上车；在食堂窗口前排队，讲究的也是先来后到。

因此，我们可以将队列理解为提供FIFO功能的一维数据存储结构，既支持向尾部（队尾）加入数据元素，又支持移出头部（队首）的数据元素。

2. 栈

理解了队列，栈（Stack）就比较好理解了。栈的数据插入特点是**后进先出**（Last In First Out，LIFO）。栈的特性在计算机中有着广泛的应用，可以说程序员无时无刻不在和LIFO打交道。一个最典型的例子就是程序中的函数调用。除此之外，在满足递归定义的问题中，栈的运用也比较多。

因此，我们可以理解栈为提供LIFO功能的一维数据存储结构，既支持向头部（栈顶）加入数据元素，又支持移出头部（栈顶）的数据元素。

3.1.3 树

掌握了链表，**树**（Tree）这种结构就很好理解了。在讨论数据结构与算法时，经常可以听到二叉树、三叉树、多叉树，下面我们通过一个例子直观地了解一下，数据结构中的树是什么样子的。

一个深入敌后潜伏的情报组织，每级情报员都分配了若干名下级情报员，相邻两级情报员分别叫作彼此的上线和下线。为了防止下线在暴露之后对整个组织造成威胁，上线和下线之间是单向通信的，上线能联系下线，下线无法联系上线，并且信息通路不形成环路（如我下线的下线不会是我的上线）。如果每级情报员的下线固定最多有2名情报员，那么这种情报结构就是二叉树结构；如果固定最多有3名情报员，那么这种情报结构就是三叉树结构；以此类推。不仅结构相似，情报组织的信息传递方式和树的数据访问顺序类似，只能从顶部（根部）往底部传递。

不管几叉树，每个节点都以链表的形式指向若干个新节点，并且树中的节点不会形成环状连接。以最常见的二叉树为例，从根节点开始，每个节点都可以指向另外两个新节点，为了方便理解，这个节点称为父节点，指向的两个新节点称为左孩子节点和右孩子节点。二叉树的存储结构图例如图3-3所示。

图3-3 二叉树的存储结构图例

既然是树，就会有高度。从根节点开始向下累积的层数就是树的高度，如图3-3中的二叉树的高度为3。二叉树之所以被广泛应用，是因为它最简单，又最具有代表性。二叉树的种类有很多，如完全二叉树、满二叉树、平衡二叉树、霍夫曼树、B树等。不同种类的二叉树具有不同的性质、不同的应用场景，感兴趣的读者可以自行深入了解。二叉树不像链表的顺序遍历那么简单，因为每个节点最多有两个孩子节点，所以遍历方法有4种：前序遍历、中序遍历、后序遍历和层序遍历。前序遍历的口诀是“中左右”，即先遍历当前父节点，再遍历左孩子节点，最后遍历右孩子节点；相应地，中序遍历的口诀是“左中右”，后序遍历的口诀是“左右中”，遍历顺序以此类推。层序遍历稍微有所区别，它会逐层按从左至右的顺序遍历所有节点。下面来看一个例子，分别用4种遍历方法遍历图3-4所示的二叉树。

图3-4 二叉树的遍历

前序遍历：1 → 2 → 4 → 5 → 3 → 6 → 7。

中序遍历：4 → 2 → 5 → 1 → 6 → 3 → 7。

后序遍历：4 → 5 → 2 → 6 → 7 → 3 → 1。

层序遍历：1 → 2 → 3 → 4 → 5 → 6 → 7。

这里特别提一句，多棵树放在一起称为**森林**（Forest），森林的概念在后面的算法中也会用到。此外，前文特别提到树这种数据结构不能形成环状，如果形成环状，那么会是什么结构？没错，正是下一节要讲的图。

3.1.4 图

图（Graph）是一种包含节点和表达节点之间关系的边的一种数据结构。前面讲到的链表和树，其实都是图的特殊情况，链表只能向一个方向进行链接，树的节点之间不允许形成环。图结构没有这些限制，图中节点之间的连接称为边。图可以分为两种，分别为**有向图**和**无向图**。顾名思义，有向图就是边带方向的图，如图3-5（a）所示；无向图就是边不带方向的图，如图3-5（b）所示。可以将无向图看作每条边都有两个方向的有向图。

图3-5 有向图和无向图

图的边除了可以表示连接关系和连接方向，还可以附带权重等信息，从而进一步表示节点之间的某种关系。例如，以城市为节点，可以将城市之间的距离作为连接这些节点的边的权重。

图是一种非常实用的数据结构，在工作、生活的方方面面都能用到它。例如，如果将每个人作为节点，将人与人之间的关系作为边，可以构成一个大小为70多亿节点的社交网络图。著名的小世界现象，你和任何陌生人之间所间隔的人数不会超过6个，就是基于社交网络图得出的猜想。在计算机世界中，图经常用于描述任务的状态转换和调度关系。例如，节点代表待执行的任务，边代表任务相关性或依赖顺序。

图的存储方式有两种，一种是以链表形式存储，每个节点及其相邻节点都用一个链表表示，这种存储方式称为邻接链表；另一种是以数组形式存储，用一维数组记录所有节点，用一个二维数组记录所有边，这种存储方式称为邻接矩阵。邻接链表和邻接矩阵的优劣对比与链表和数组之间的对比类似，邻接链表使用的是链表，邻接矩阵使用的是二维数组。以图3-5中的有向图为例，邻接链表和邻接矩阵的具体存储形式如图3-6所示。

图3-6 邻接链表和邻接矩阵的具体存储形式

3.1.5 散列表

散列表 (Hash Table) 又称为哈希表。在讲解散列表之前，我们先从一个实际问题出发：如何在全国范围内较快速地根据姓名查询对应的身份证号码？如果使用数组或链表存储身份证信息，则需要从头到尾查询一遍才能保证找到所查姓名的身份证号码，时间复杂度都是 $O(N)$ ；如果使用二叉排序树存储身份证信息，时间复杂度也只能提高到 $O(\log_2 N)$ ，通常默认对数底为2，可简写为 $O(\log N)$ 。

这里补充说明一下什么是时间复杂度。 $O(N)$ 和 $O(\log N)$ 都是描述算法复杂度的符号，算法复杂度主要包含两个方面：空间复杂度和时间复杂度。其中，空间复杂度定性地描述了一个算法运行所需内存空间的数量级，时间复杂度定性地描述了一个算法运行所需时间的数量级。在通常情况下，我们优先考虑算法的时间复杂度。在时间复杂度中， $O(N)$ 表示当数据量为 N 时，计算机需要执行的基本运算次数的数量级为线性级别； $O(\log N)$ 表示当数据量为 N 时，计算机需要执行的基本运算次数数量级为对数级别。除了 $O(N)$ 和 $O(\log N)$ ，还有 $O(N\log N)$ 、 $O(N^2)$ 、 $O(N^3)$ 等，含义以此类推。 $O(\log N)$ 的时间复杂度比 $O(N)$ 的时间复杂度低，表示 $O(\log N)$ 的算法比 $O(N)$ 的算法更合适。例如，当 N 为1024时， $\log N$ 只有10。

$O(\log N)$ 的时间复杂度已经足够低了，但是散列表的时间复杂度能更低。散列表可以将查询身份证号码的时间复杂度降到 $O(1)$ ，即常数级别。散列表用到的关键技术是函数值映射，利用散列函数，将key值（姓名字符串）映射到一个有限的数值空间内。例如，一个长度为10亿的数组，如果使用恰当的散列函数，那么映射后的值会均匀分布到这个数组中。在查询时，计算被查询姓名的散列值，即可直接取出数组中对应下标的身份证号码。这里你可能会问，姓名有重复怎么办？没关系，这个问题在散列表中经常出现，这种现象称为键值冲突。键值冲突问题的解决方法比较简单，在遇到重复key值时，只需在对应的散列值后面增加一个链表，用于存储重复key值。

散列表的冲突存储方式与邻接链表的存储方式类似，重复的key值会被存储于同一个散列值生成的链表中。当然，散列函数直接影响到了键值冲突问题的大小，一个映射不均匀的散列函数有可能导致严重的链表倾斜，即大量数据被映射到同一个键值，导致对这些键值的访问退化成顺序遍历，降低检索效率。针对这个问题，一方面可以选取合适的散列函数；另一方面，在发生键值冲突后，可以采取二次散列法和设计公共溢出区等措施。

3.2 基础算法

算法是针对一类数学问题制订的完整而准确的解决方案，代表用系统的方法描述解决问题的策略机制。也就是说，对于符合一定要求的输入，算法必须在有限时间内获得相应的输出。不同的算法能用不同的时间效率、空间效率来完成相同的任务，它们的性能可以用时间复杂度与空间复杂度衡量。算法作为数据结构的产物，它的实现离不开数据结构的支持。

基于数组、链表、树和图的算法有很多，下面介绍几种常用且有趣的算法，帮助大家开启AI算法的思维之门。

3.2.1 排序

基础算法中最常用的算法，非排序莫属。现实生活中存在不少排序的例子。例如，人群按照身高站队需要排序，考试分数需要排序，整理卷子编号需要排序，等等。那么常用的排序算法有哪些，它们分别如何实现？

排序算法的种类很多，按照时间复杂度划分，从慢到快主要有以下9种排序算法。

$O(N^2)$ ：选择排序、插入排序、冒泡排序。

$O(N\log N)$ ：快速排序、堆排序、归并排序。

$O(N)$ ：计数排序、桶排序、基数排序。

1. 选择排序

选择排序（Selection Sort）的基本思路是每次选出剩余序列中的最值按先后顺序排列。如果要求按从小到大的顺序排序（升序排序），则每次选择最小值来组成新的序列；如果要求按从大到小的顺序排序（降序排序），则每次选择最大值来组成新的序列。

对于一个长度为 N 的待排序列，算法在执行第1轮排序时，会从 N 个元素中选出最值，需要执行 $N-1$ 次比较运算；算法在执行第2轮排序时，会从剩下的 $N-1$ 个元素中选出最值，需要执行 $N-2$ 次比较运算；以此类推，序列剩下的最后两个元素需要执行1次比较运算。这样总的比较次数是

算法复杂度的衡量是看最高数量级，此处为二次方项，在不影响当前数量级的情况下，将常数项系数省略，所以选择排序的时间复杂度是 $O(N^2)$ 。

以升序排序为例，选择排序的执行过程如图3-7所示。

图3-7 选择排序的执行过程

2. 插入排序

插入排序（Insertion Sort）的基本思路是依次取出序列中的每个元素并将其插入一个有序序列。

对于一个长度为 N 的待排序列，初始有序序列为空，算法在执行第1轮排序时，会取一个元素插入空的有序序列，需要执行0次比较运算；算法在执行第2轮排序时，会取一个元素插入长度为1的有序序列，需要执行1次比较运算；以此类推，算法在执行第 N 轮排序时，会取一个元素插入长度为 $N-1$ 的有序序列，需要执行 $N-1$ 次比较运算。这样总的比较次数是

因此插入排序的时间复杂度和选择排序的时间复杂度相同，也是 $O(N^2)$ 。

以升序排序为例，插入排序的执行过程如图3-8所示。

图3-8 插入排序的执行过程

3. 冒泡排序

顾名思义，冒泡排序（Bubble Sort）的基本思路是循环比较相邻的元素值，将最值像水里的气泡冒上水面一样选出来。与选择排序和插入排序相比，冒泡排序不需要额外的序列存储空间，在算法执行完成后，原序列即可变成有序序列。

对于一个长度为 N 的待排序列，以升序排序为例，每一轮排序都会从原序列的最后一个元素开始。算法在执行第1轮排序时，会从后向前依次比较相邻两个元素值，如果前面的元素值大于后面的元素值，就交换两个元素的位置，直到第一个元素，需要执行 $N-1$ 次比较运算；算法在执行第2轮排序时，会从后向前依次比较相邻两个元素值，直到第二个元素，需要执行 $N-2$ 次比较运算；以此类推，最后一轮比较为首的两个元素值，需要执行1次比较运算。这样，总的比较次数也是

因此冒泡排序的时间复杂度也是 $O(N^2)$ 。

以升序排序为例，冒泡排序的执行过程如图3-9所示。

图3-9 冒泡排序的执行过程

4. 快速排序

快速排序（Quick Sort）简称快排，是程序员面试时考官最常考查的排序算法之一，它的基本思路是**每次确定待排序列中一个元素的最终排序位置**。

对于一个长度为 N 的待排序列，以升序排序为例，算法在执行第1轮排序时，当前序列为原序列，从原序列中选出第一个元素，两个指针分别从序列两端向中间遍历，比较所选元素与指针遍历元素的大小，通过交换位置，确保所选元素的最终位置，并且将当前序列一分为二，分为左序列和右序列（左序列中的元素值都不大于所选元素值，右序列中的元素值都不小于所选元素值）；算法在执行第2轮排序时，当前序列为原序列的左序列，具体操作同前一步；算法在执行第3轮排序时，当前序列为原序列的右序列，具体操作同前一步；以此递推，直到找到所有元素的最终位置，快速排序结束。快速排序由于采用折半操作，因此时间复杂度为 $O(N\log N)$ 。

以升序排序为例，快速排序第1轮排序的执行过程如图3-10所示。

图3-10 快速排序第1轮排序的执行过程

需要特别注意的是，如果原序列本身就是有序序列，那么折半操作会退化成线性操作，时间复杂度也会变为 $O(N^2)$ 。快速排序之所以快，是因为它让每轮所选元素的最终位置尽量靠近序列中央，这样就起到了折半的作用。所以，快速排序的关键在于如何随机选择元素，让所选元素在数学期望下靠近中间位置。因此，在每一轮排序开始

时，只需加上一个随机函数，随机从当前序列中选择一个元素与待排元素事先交换，这样的快速排序又称为随机快速排序。

5. 堆排序

堆排序（Heap Sort）的基本思路是先将序列构建成一个最大二叉堆或最小二叉堆。二叉堆是二叉树的一种。最大二叉堆是指每个节点的元素值都比其左、右孩子节点的元素值要大的二叉堆，最小二叉堆是指每个节点的元素值都比其左、右孩子节点的元素值要小的二叉堆。

算法在执行时，每次从二叉堆中移出堆顶元素，并且将末尾元素移至堆顶，然后调整二叉堆结构，维持最大二叉堆或最小二叉堆的结构特点。如果构建的是最大二叉堆，那么得到的是降序序列；如果构建的是最小二叉堆，那么得到的是升序序列。由于二叉堆是二叉树的一种，维持最大二叉堆或最小二叉堆的结构特点是一个二分查找的过程，因此堆排序的时间复杂度和快速排序的时间复杂度一样，也是 $O(N\log N)$ 。

以升序排序为例，堆排序第1轮排序的执行过程如图3-11所示。二叉堆各节点的元素值可以使用数组存储，数组下标从1开始，假设父节点的下标为 i ，那么左、右孩子节点的下标分别为 $2i$ 、 $2i+1$ 。

图3-11 堆排序第1轮排序的执行过程

6. 归并排序

归并排序（Merge Sort）采用递归与分治的思想（在3.2.2节会详细详解），先保证局部有序，再保证整体有序，从局部到整体可以看作一个归纳合并的过程，这也是该排序算法名字的由来。

对于一个长度为 N 的待排序列，以升序排序为例，算法在执行第1轮排序时，原序列中的每个元素都可以看作一个局部有序序列，将相邻的两个局部有序序列合并成一个长度为2的局部有序序列，局部最大比较次数为1；算法在执行第2轮排序时，合并相邻的两个长度为2的局部有序序列，得到长度为4的局部有序序列，局部最大比较次数为3；以此类推，直到原序列有序。由此可见，归并排序总的执行轮次是 $\log N$ ，因此其时间复杂度为 $O(N\log N)$ 。

以升序排序为例，归并排序的执行过程如图3-12所示。

图3-12 归并排序的执行过程

7. 计数排序

计数排序（Count Sort）是一种用更多存储空间交换更高时间效率的排序算法，这种排序算法无须做任何比较操作，只需创建一个count数组，这个数组的长度大于待排序列中元素值的极差（最大元素值与最小元素值的差）。

对于一个长度为 N 的待排序列，值域为 $[0, K-1]$ ，初始化一个长度为 K 、各个位置上元素值均为0的数组，依次遍历待排序列，将count数组中以当前元素值为下标的位置加1，排序完成。如果要进行升序排序，那么按照下标从小到大的顺序遍历数组，遇到几就输出几个当前位置的下标值，直到遍历结束；如果要进行降序序列，那么按照下标从大到小的顺序遍历数组，遇到几就输出几个当前位置的下标值，直到遍历结束。计数排序只涉及线性操作，时间复杂度为 $O(N+K)$ 。很明显，在存储空间有限的情况下，计数排序对取值范围较小的待排序列有比较明显的速度优势。

以升序排序为例，计数排序的执行过程如图3-13所示。

图3-13 计数排序的执行过程

8. 桶排序

计数排序的最大问题是随着待排序列值域的扩大，空间复杂度也随之增加。桶排序（Bucket Sort）是计数排序的进化版本，它不是一对一的下标映射，而是通过一个映射函数，一对多地将大小接近的数存储于一个桶中，然后对每个桶中的序列进行排序。如果每个桶中排序的时间复杂度为 $O(N\log N)$ ，那么桶排序的时间复杂度为 $\sum O(N_i \log N_i)$ 。

9. 基数排序

计数排序和桶排序都较难解决混合数据的排序问题，如字符串之间的排序。基数排序（Radix Sort）是桶排序的升级版，其基本思路是从待排元素的低位到高位每轮都按位分桶，按位比较元素值的大小，摆脱了数据类型的限定。基数排序的时间复杂度为 $O(dN)$ ，其中 d 是最大位数。

在实际使用过程中，除了时间复杂度，有时还需要考虑排序算法的稳定性。排序算法的稳定性是指在排序过程中是否会改变任意两个相等元素的前后顺序。一个排序算法如果不改变任意两个相等元素的前后顺序，则是稳定的排序，否则是不稳定的排序。

为了方便读者理解和记忆，下面对以上9种排序算法在时间复杂度和稳定性上进行优劣对比，如表3-1所示。

表3-1 9种排序算法对比

3.2.2 递归与分治

1. 递归

简单地说，**程序调用自身的编程技巧称为递归**。递归（Recursion）的学术定义如下：一个函数、过程、概念或数学结构，如果在其定义或说明内部直接或间接地出现其本身的引用，则它们是递归的或递归定义的。

具体而言，递归可以分为递推和回归两个阶段。问题逐层往前递进推动，这个过程叫作递推；逐一解决子问题，最后汇总为原问题，这个过程叫作回归。二者交替配合，就形成了递归。我们通过一个直观的例子感受递归的整体流程。

某所中学的校团委为了筹集团员活动经费，计划向本校的每一位团员收取团费。该通知采取逐级传达的方式，校团委通知年级团委，然后年级团委通知班团委，最后班团委通知到班里的每一位团员。这种逐层递进传递任务的过程，就是递推过程。在任务发布后，只有在每一级的团费收集任务全部完成后，才能向上一级汇报。例如，在班里每一位团员的团费都收齐之后，班团委才能向年级团委汇报；在每个班的团费都收齐之后，年级团委才能向校团委汇报。这种在子任务全部完成之后，自下向上汇报的过程，就是回归过程。整个团费收集的过程，就是一个完整的递归过程。

当然，有递归就有非递归。所谓非递归，就是不用层级关系来表达任务，而是将任务拉平成线性关系表达任务。因此，支持递归实现的基础算法会比非递归实现理解起来较为简单直观。举个例子，求长度为 n 的斐波那契数列各项之和。递归写法和非递归写法的代码实现分别如下。

对比发现，递归写法的程序具备结构清晰、可读性强等优点，并且递归写法比非递归写法的代码更容易实现。当然，一些问题如果使用递归算法求解，会因为重复计算的问题让算法变得低效，递归的代码抽象，容易出错且很难调试。例如，斐波那契数列求和就不适合使用递归算法求解。所以，只有当问题本身是递归定义的，或者问题涉

及的数据结构是递归定义的时，递归算法才是最佳选择。经典的递归问题有八皇后问题、汉诺塔问题和全排列问题。

2. 分治

分治（Divide Conquer Algorithm）就是“分而治之”的意思，实质是**将原问题分成N个规模较小、结构与原问题相似的子问题，然后递归地求解这些子问题，最后得到原问题的解**。与递归算法相比，分治算法强调的是同层级问题分别治理的过程，而递归算法强调的是不同层级问题之间的逐层递进与回归的过程。

分治算法的思想在日常生活中随处可见。例如，某学院需要选拔一名乒乓球打得最好的学生，代表学院参加学校的乒乓球单打比赛。首先选拔出各班级的第一名，然后从各班级的第一名中选拔出各年级的第一名，最后从各年级的第一名中选拔出学院的第一名。将“选拔学院第一名”拆分成“选拔各年级的第一名”，再进一步拆分成“选拔各班级的第一名”，这个过程就是分治过程。

因此，分治算法有两个适用条件。其一，大问题能分解成若干个独立的子问题；其二，每个子问题的求解过程都和大问题相似。这两个条件决定了分治的递归过程是受限的，需要提前知道递归过程的终止条件及每一层的逻辑，因此分治算法的应用场景不如递归算法的应用场景多。经典的分治问题包括快速排序、归并排序及地毯填补。

3.2.3 贪婪算法和动态规划

所谓贪婪，就是只看眼前小利，不看长远收益的思想。贪婪算法是一种局部最优算法，动态规划则是一种全局最优算法。

1. 贪婪算法

贪婪算法（Greedy Algorithm）是指每一步都采取当前最好或最优（最有利）的选择，从而使结果是最好或最优的算法。一个问题是否适合使用贪婪算法解决，可以根据无后效性判断。无后效性是指**某个状态之后的收益，只与当前状态有关，不受之前状态的影响**。通俗地讲，就是眼前的最大收益可以带来最终的最大收益。现实生活中有不少问题满足无后效性，下面以经典的贪婪问题——超级书架为例进行讲解。

问题描述：有 N 个物品，第 i 个物品的高度为 H_i ，还有一个高度为 B 的书架，保证这些物品的高度之和大于或等于书架的高度 B 。最少从这 N 个物品中取出多少个物品，可以使取出来的物品堆起来的总高度不低于书架的高度 B 。

每次只需选择当前剩余物品中最高的物品，直到所选物品的高度之和超过书架为止，所选物品的数量就是答案。

2. 动态规划

动态规划（Dynamic Programming，DP）是解决多阶段最优化问题的一种算法。这个定义听起来有些令人费解，我们不妨对比贪婪算法来理解。贪婪算法认为选择当前步骤的最大收益可以带来最终的最大收益。试想，有没有这样一类问题，在当前步骤不选择最大收益，最终的收益却比贪婪算法的收益高？答案是肯定的。下面给大家介绍一个和超级书架问题类似的经典动态规划问题——**背包问题**。

问题描述：一个容量为 M 的背包，现在有 N 个物品，第 i 个物品的体积为 N_i ，如何选择物品装包才能使背包的利用率最大？

按照贪婪算法装包不一定得到最优解。例如，对于容量为10的背包，假设有5个体积依次为（6、5、3、3、2）的物品，使用贪婪算法会按从大到小的顺序装包，因此体积为6和3的物品被选出，背包容量利用率为90%。但是，最优的策略应该是将体积为5、3和2的物品装包，此时背包的利用率为100%。

那么，动态规划是如何得到上面这个解的呢？还是上面的背包问题。如果一开始只有4个容量分别为2、3、5、6的背包，那么最优的装包方式就是往4个背包中分别放入2、3、5和6。紧接着，加入3个容量分别为7、8、9的背包，最优的装包方式分别为，容量为7的背包在装入容量为5的背包的基础上再放入2；容量为8的背包在装入容量为6的背包的基础上再放入2，或者在装入容量为5的背包的基础之上再放入3；容量为9的背包在装入容量为6的背包的基础上再放入3。最后，一个容量为10的背包如何装包，显然最优方案是在装入容量为7的背包的基础上再装入3，或者在装入容量为8的背包的基础上再装入2，如此装包的容量利用率就达到了100%。这便是“多阶段最优决策”的一个完整展现。背包问题的动态规划过程如图3-14所示。

图3-14 背包问题的动态规划过程

依次循环遍历物品。第1轮，随着每个物品的放入，能在空背包状态下得到4种可放置容量，分别为2、3、5、6。第2轮，在第1轮可放置容量的基础上，不重复放入，新增7、8、9共3种可放置容量。第3轮，在第2轮可放置容量的基础上，不重复放入，即可装满容量为10的背包。

根据上述对“多阶段最优决策”过程的图文解释可以发现，动态规划的求解过程是一个从局部计算逐步推到整体计算的过程。归纳如下：首先，得到“最小规模”的局部最优解；然后，基于“最小规模”的局部最优解，得到“次小规模”的局部最优解；以此类推，从局部到整体，即可得到全局最优解。这个过程可以用递推公式来描述。例如，背包问题的递推公式是 $f(n)=f(n-c[i])$ ， $f(n)$ 表示容量为 n 的背包是否能装满， $c[i]$ 表示第 i 个物品的体积。这个递推公式又称为动态转移方程。背包问题的代码实现如下。

上述背包问题是最简单的一种背包问题，叫作01背包，即要么放要么不放，其经典变种问题有多重背包问题和完全背包问题，都可以

用动态规划求解。此外，典型的动态规划问题还有导弹拦截、字符串距离和最长公共子序列等。

3.2.4 搜索

搜索是更有组织纪律的枚举。枚举是指在一个有限范围内列举出所有可能的情况，以便找出所有解。因为枚举比较简单，所以不进行单独介绍。与枚举相比（如著名的暴力枚举法），搜索的针对性和目的性更强。搜索分为深度优先搜索和广度优先搜索，二者的侧重点不同。

深度优先搜索（Depth First Search，DFS）又称为深度优先遍历。深度优先，顾名思义，就是每轮搜索都会从当前节点沿一条路找下去，直到每条路都走到尽头，即遍历完所有节点或满足某个条件，算法结束。图3-15所示为深度优先搜索示例，起点为A，深度优先搜索的遍历序列为ABDECFG。

图3-15 深度优先搜索示例

宽度优先搜索（Breadth First Search，BFS）又称为“宽度优先遍历”或“广度优先遍历”。宽度优先搜索的每一轮会将距离当前节点最近的节点全部找出来，按这种方法依次找下去，直到遍历完所有节点或满足某一条件，算法结束。图3-16所示为宽度优先搜索示例，BFS的遍历序列为ABCDEFGF。

图3-16 宽度优先搜索示例

3.2.5 最短路径

最短路径（Shortest Path）的问题在生活中很常见，如交通运输规划、出行线路规划等。按照算法规模，最短路径算法可以分为全局最短路径算法和单源最短路径算法。全局最短路径算法有弗洛伊德算法（Floyd's Algorithm），单源最短路径算法有迪杰斯特拉算法（Dijkstra's Algorithm）、贝尔曼-福特算法（Bellman-Ford's Algorithm）。

1. 弗洛伊德算法

弗洛伊德算法是斯坦福大学计算机科学系教授罗伯特·弗洛伊德以当前公认的形式提出的。弗洛伊德算法运用了动态规划，可以求出一个图中任意两点之间的最短路径。基本思路是每轮选取两个节点，然后枚举其他节点，如果经过某个节点能让选取的两个节点之间的距离缩短，就更新这两个节点之间的距离。代码实现如下，其中 f 表示任意两点之间的距离，可知弗洛伊德算法的时间复杂度为 $O(V^3)$ 。

2. 迪杰斯特拉算法

迪杰斯特拉算法是由荷兰计算机科学家迪杰斯特拉于1959年提出的，主要用于解决有向图中的单源最短路径问题。其基本思路是以起始节点为中心，每次选一个距离起始节点最近的节点，然后扩展该节点相邻的节点并更新与它们之间的距离，直到扩展完所有节点。需要注意的是，迪杰斯特拉算法无法计算带负边的图。迪杰斯特拉算法示例流程如图3-17所示，其时间复杂度为 $O(V^2)$ 。

图3-17 迪杰斯特拉算法示例流程

图3-17中的迪杰斯特拉算法示例流程的解读如下。

（a）以 a 为起始节点，初始化起始节点的距离为0。

- (b) 更新与最优解节点a相邻的节点b和c的距离。
- (c) 更新与最优解节点c相邻的节点b、d、e的距离。
- (d) 更新与最优节点e相邻的节点d的距离。
- (e) 更新与最优节点b相邻的节点d的距离。
- (f) 与最优节点d相连的节点无须更新，算法结束。

3. 贝尔曼-福特算法

贝尔曼-福特算法是由美国数学家理查德·贝尔曼和莱斯特·福特提出的。如果迪杰斯特拉算法的思路是以点扩展边，那么贝尔曼-福特算法的思路是以边扩展点。贝尔曼-福特算法不仅能解决迪杰斯特拉算法无法计算的负值问题，而且能识别负值回路。贝尔曼-福特算法示例流程如图3-18所示，其时间复杂度为 $O(VE)$ 。

图3-18 贝尔曼-福特算法示例流程

图3-18中的贝尔曼-福特算法示例流程的解读如下。

- (a) 以a为起始节点，初始化起始节点的距离为0。
- (b) 使用与节点a相连的边更新节点b和c的距离。
- (c) 使用与节点b和c相连的边更新节点b、d、e的距离。
- (d) 使用与节点b相连的边更新节点d的距离。
- (e) 图中各边已无更新必要，算法结束。

3.2.6 最小生成树

最小生成树（Minimal Spanning Tree，MST）首先是一棵树，“最小”的含义是这棵树的边长之和最小。那你可能会问，为什么不叫“最小树”？不妨这样来理解，“生成”是一个动词，强调的是这棵“最小树”的产生过程，所以叫最小生成树。下面以一个实际问题为例讲解最小生成树的作用。

问题描述：假设有4座城市，分别为A、B、C、D，在这4座城市之间铺设光缆，城市之间的光缆铺设费用分别为 $AB=8$ 、 $AC=1$ 、 $AD=6$ 、 $BC=3$ 、 $BD=2$ 、 $CD=5$ ，要求任意两个城市之间可以通信，求铺设光缆的最小花费。

我们可以将城市作为节点，城市之间以边相连，将光缆铺设费用作为边长，构成无向图。求最小铺设费用等价于求这个无向图的最小生成树。最小生成树最常用的算法有普里姆算法（Prim's Algorithm）和克鲁斯卡尔算法（Kruskal's Algorithm）。

1. 普里姆算法

普里姆算法可以从任意节点开始建树。该算法的基本思路如下：从原点开始，每次添加与当前最小生成树相连且不成环的最短边，直到添加所有节点为止。对于上面的光缆铺设例子，以A为最小生成树的原点，首先添加与之相连且不成环的最短边AC，然后按算法法则依次添加BC边、BD边，直到所有节点均已添加，算法结束，最小花费为6（ $1+3+2$ ）。普里姆算法的时间复杂度为 $O(V^2)$ ，其推进步骤示意图如图3-19所示。

图3-19 普里姆算法的推进步骤示意图

2. 克鲁斯卡尔算法

类似于迪杰斯特拉算法和贝尔曼-福特算法之间的关系，如果普里姆算法的基本思路是以点扩展边，那么克鲁斯卡尔算法的基本思路是以边扩展点，具体如下：每次从待选边集中选出与当前森林不成环且最短的边加入森林，直到所有节点形成一棵树为止。该思路决定了克鲁斯卡尔算法只能从最短边开始构建最小生成树。对于光缆铺设例子，首先将AC边加入森林，然后按算法法则依次加入BD边、BC边，此时所有节点已构成一棵树，算法结束，最小花费同样为 $6(1+2+3)$ 。这里需要用到排序，如果使用快速排序算法，那么克鲁斯卡尔算法的时间复杂度为 $O(E\log E)$ ，其推进步骤示意图如图3-20所示。

图3-20 克鲁斯卡尔算法的推进步骤示意图

值得一提的是，稠密图（边数较多的图）适合使用普里姆算法，而稀疏图适合使用克鲁斯卡尔算法。

3.2.7 树状数组

从本节开始的5个基础算法，其难度有所提升，建议读者选择阅读。

树状数组（Binary Indexed Tree）是一种具有树结构的数组，其利用二进制的计算特性，可以在时间复杂度为 $O(\log N)$ 的情况下动态维护一个序列，并且计算其任意子序列中的所有元素之和，算法思路十分简洁、巧妙。下面从二进制的特性开始，逐步讲解树状数组的实现思路。

有4个二进制数，分别为0011、0101、0110、0111，每次通过将原数的非零最低位加1来更新原数，可以得到4个变化过程，分别为
0011 → 0100 → 1000 → 10000、0101 → 0110 → 1000 → 10000、
0110 → 1000 → 10000、0111 → 1000 → 10000。观察这4个变化过程有没有

发现什么规律？没错，这种运算总能将原数变为比它大的、最接近的2次幂。假设有8个位置，编号分别为1~8，将其组成一个树状数组，如图3-21所示，灰色代表原数组，黑色代表树状数组。

图3-21 树状数组结构示例

黑色单元从左至右的相连关系就是上面描述的“加1变换”（ $B_1 \rightarrow B_2 \rightarrow B_4 \rightarrow B_8$ ， $B_3 \rightarrow B_4 \rightarrow B_8$ ， $B_5 \rightarrow B_6 \rightarrow B_8$ ， $B_7 \rightarrow B_8$ ）。有“加1变换”自然就有“减1变换”。“减1变换”能以 $O(\log N)$ 的时间复杂度找到任意元素的核心子元素，这样维护和计算任意子序列中所有元素的和就很方便了。例如，对于图3-21中的 B_7 ，通过“减1变换”得到序列（ $B_7 \rightarrow B_6 \rightarrow B_4$ ），正好可以计算 $A_1 \sim A_7$ 之和。

举个直观例子。假设原数组为[3,5,1,9,2,4,8,7]，依次遍历原数组，并且通过“加1变换”将每一个元素值累加到树状数组的相应位置上，这一步的时间复杂度为 $O(N \log N)$ ，相应的树状数组如图3-22所示。

图3-22 树状数组求和

在树状数组构建完成后，尝试计算前7个数之和。我们通过“减1变换”在树状数组中依次遍历到0111、0110、0100这3个位置，对应下标的元素值为8、6、18。因此前7个数之和为 $8+6+18=32$ 。如果使用原数组，则需要进行6次加法运算；如果使用树状数组，则只需进行2次加法运算。随着数列长度的增加，树状数组的时间优势会更加明显。此外，当原数组中某个元素的值增加或减少 d 时，同样可以通过“加1变换”来更新树状数组中相应的元素值。

无论是“加1变换”，还是“减1变换”，核心问题都是找到二进制的最低位，如何得到二进制的最低位呢？这就不得不提计算机最擅长的位运算了，最低位计算公式为

$\text{lowbit}(x) = x \text{ and } (x \text{ xor } (x-1))$

利用补码特性，该计算公式可以简化成

$\text{lowbit}(x) = x \text{ and } -x$

简化后的最低位计算公式是常数级的计算。树状数组查询和更新的时间复杂度为 $O(\log N)$ 。由于树状数组的构建要遍历整个原数组，因此时间复杂度为 $O(N \log N)$ 。

3.2.8 线段树

线段树（Segment Tree）的概念和它的名字一样，是以线段（通常是指数组的一个子数组，点可以看作线段的特殊形式）为节点构成的树。上一节讲的树状数组能高效率地计算任意子序列中的所有元素之和。但如果需要计算任意子序列中的最大值和最小值，树状数组显然是束手无策的，这时需要使用线段树。线段树可以解决包括树状数组解决的问题在内的所有子序列求解问题。相应地，线段树比树状数组复杂很多。下面通过一个具体例子讲解什么是线段树。

问题描述：一个长度为 N 的序列，每次询问任意一段区间内的最大值。

可以使用暴力枚举法，即通过每次遍历这段区间内的序列来找出最大值，其时间复杂度为 $O(N)$ 。但是在序列长度很长的情况下，这种算法的查询速度实在太慢。由于线段树具有二叉结构，因此可以将该问题的时间复杂度降至 $O(\log N)$ 。例如，有一个长度为6的序列[3, 2, 6, 5, 1, 9]，将其构造成线段树，如图3-23所示。

图3-23 线段树结构示例

采用递归建树。灰色表示叶子节点，表示原序列，用数组存储；黑色表示非叶子节点，表示该区间内的最大值，用链表存储。在每次询问某段区间内的最大值时，从根节点开始匹配，线段如果分别处于

左右节点，则分裂继续匹配，直到线段匹配成功为止，然后回溯比较最大值即可。这样每次查询最大值的时间复杂度就降为了 $O(\log N)$ 。

上面只说了线段树的基本特性，而**延迟标记**是线段树的精华所在。没有延迟标记，线段树的更新会变得很慢。例如，当原序列中某一段区间内的值同时增加 d 时，使用暴力枚举法对其进行更新的逻辑会十分复杂，时间复杂度为 $O(N)$ ；而使用线段树对其进行更新的时间复杂度为 $O(\log N)$ ，方法是给每个节点均增加一个延迟标记，这个标记的初始值均为0。

例如，将 $[0, 2]$ 区间内的数均增加4，从根节点开始匹配，发现根节点区间 $[0, 5]$ 包含区间 $[0, 2]$ ，向子节点继续查询，发现左节点区间为 $[0, 2]$ ，将其延迟标记值增加4，并且将左节点的值加4，变为10。由于左节点区间为 $[0, 2]$ ，因此不再向下查询，开始回溯，将根节点的值更新为新子节点之间的最大值10，如图3-24所示。

图3-24 线段树的更新

在图3-24中，三角形内的数字代表每个节点的延迟标记值。之后的每次最大值查询，如果当前节点的延迟标记值大于0，那么在向下查询时会带走延迟标记，重复上述延迟操作，直到完全匹配。这种延迟查询的方式大大提高了序列更新与查询的效率。接前文，可知当前延迟标记在 $[0, 2]$ 区间内的节点上，在查询 $[0, 1]$ 区间内的最大值时，带上延迟标记从 $[0, 2]$ 区间内的节点向左子树走，更新的是 $[0, 1]$ 区间内节点的值，即得到查询结果。更新后的线段树如图3-25所示。

图3-25 更新后的线段树

在更新后， $[0, 1]$ 区间内的最大值为7。这里需要注意一点，如果要带走延迟标记，那么为了保持左、右孩子节点的正确性，至少需要往每个孩子节点上带一次延迟标记，在图3-25中， $[0, 1]$ 区间虽然在左子树上，但延迟标记同样需要往右子树带过去。

3.2.9 平衡二叉树

平衡二叉树（Balanced Binary Tree）是由G.M.Adelson-Velsky和E.M.Landis于1962年提出的，又称为AVL树。平衡二叉树属于二叉搜索树（Binary Search Tree），所以在讲解平衡二叉树之前，不妨先讲讲什么是二叉搜索树。

二叉搜索树是能够表达一定元素顺序的二叉树。由于从树根往下能进行二分查找，因此二叉搜索树通常能在 $O(\log N)$ 的时间复杂度下查找某些节点的元素值。但是，不加条件的二叉搜索树往往容易退化成链表，使时间复杂度变成 $O(N)$ 。

平衡二叉树的全称为平衡二叉搜索树，主要用于解决左、右子树高度平衡问题。为了解决二叉搜索树退化成链的问题，平衡二叉树可以保证任意节点的左、右子树高度差不大于1。在表达同样序列大小关系的情况下，普通的二叉搜索树与平衡二叉树之间的区别如图3-26所示。

图3-26 二叉搜索树与平衡二叉树之间的区别

由此可见，平衡二叉树查找元素值的时间复杂度稳定在 $O(\log N)$ 。那么，平衡二叉树是如何保证任意节点的左、右子树高度差不大于1的呢？我们可以先想想左、右子树高度差大于1的情况一共有几种，只有将这些“大于1”的情况转变成“不大于1”的情况，二叉搜索树才能成为平衡二叉树。总体来说，左、右子树高度差大于1的情况有4种，如图3-27所示。

图3-27 左、右子树高度差大于1的4种情况

根据图3-27可知，通过镜面变换，图3-27（a）和图3-27（d）可以归为一类，图3-27（b）和图3-27（c）可以归为一类。为了将左、右

子树高度差大于1的二叉搜索树调整成平衡二叉树，可以通过以下两种旋转操作实现。

- 单旋。对于图3-27 (a)，将B提到树根，A变成B的右孩子节点，E变成A的左孩子节点，即可将该二叉搜索树转换为平衡二叉树，具体变换操作如图3-28所示。同理可单旋图3-27 (d)。图3-27 (a) 和图3-27 (d) 的处理区别只有左右之分，因此单旋又可以细分为左旋和右旋。

图3-28 单旋

- 双旋。双旋是指进行两次单旋。对于图3-27 (b)，第一次单旋是旋转A的左子树，使以B为根节点的左、右子树高度相等，并且将E提到A的左子树树根；第二次单旋，将E提到树根，A变成E的右孩子节点，即可将该二叉搜索树转换为平衡二叉树，具体变换操作如图3-29所示。同理可双旋图3-27 (c)。

图3-29 双旋

任意一棵二叉搜索树，从叶子节点开始，通过单旋或双旋一直变换到根节点，可以保证任意节点的左、右子树高度差不大于1。平衡二叉树的建树变换时间复杂度为 $O(N\log N)$ 。此外，平衡二叉树还有红黑树、SBT、Treap、伸展树等几个变种，感兴趣的读者可以深入学习。

3.2.10 并查集

并查集 (Disjoint Set) 并非常用的基础算法，却也是一种很有意思的算法。并查集能高效率地判断两个元素是否位于同一个图中。下面通过一个具体的例子讲解并查集可以解决什么问题。

问题描述：有N个人，约定存在血缘关系的为亲人，并且如果X和Y是亲人、Y和Z是亲人，则X和Z也是亲人。现给出M对人的血缘关系，保证这N个人都已涵盖在内。计算这N个人中一共有多少个家族，以及需要询问多少次任意两人是否属于同一个家族（询问次数用T表示）。

根据题目描述，不难想到暴力枚举法，即每次将有血缘关系的两个人所属的家族集合编号赋值为同一个值。可想而知，当N、M、T较大时，算法时间复杂度不可估量。此时，并查集横空出世。

并查集最常用的数据结构是森林，算法思路如下：首先初始化N个人作为N棵树，每棵树都作为一个家族；然后根据给出的M对血缘关系，将两个人所在的树合并为一棵新的树，组成一个新的家族。例如，有5组血缘关系，分别为(a, b) (b, c) (d, e) (d, f) (d, b)，并查集的构建步骤如图3-30所示。

在图3-30中，字母后面的数字代表家族编号。首先，(a, b)表示a和b有血缘关系，将b所在的树根指向a所在的树根，组成一棵新的家族树；然后依次按照此规则更新(b, c) (d, e) (d, f) (d, b)这4组血缘关系。值得注意的是，在处理血缘关系(d, b)时，由于b不是当前家族树的根节点，因此需要先找到根节点a，再将a指向d。此外，为了防止树退化成链表，通常将深度较小的树的指针指向深度较大的树的根节点，因此需要将图3-30 (f) 中的并查集森林结构转换为并查集的平衡结构，如图3-31所示。

图3-30 并查集的构建步骤

图3-31 并查集的平衡结构

根据图3-31可知，家族数量就是森林中最后形成的树的棵数。当查询任意两个人是否属于同一个家族时，只需分别比较这两个节点所

在树的根节点编号是否相同。为了方便理解，在并查集的构建过程中，直接将子树所有节点的家族编号都设置为根节点编号。

不难发现，即使按照上面所讲的平衡结构构建并查集，也会面临两个问题：其一，要合并的两棵树，如果当前节点距离各自根节点的距离并非最长距离，则会直接导致平衡结构构建失败；其二，即使构建了平衡结构，本身过长的分支也仍然存在。平衡结构只是并查集的一种简单优化，并查集最有效的优化方法为**路径压缩**。路径压缩的优化思想如下：在每次找树根时，将经过的所有节点都指向根节点，将较长的路径变成多子树。使用路径压缩方法优化并查集的时间复杂度为 $O(M\log N)$ ，而每次查询的时间复杂度为 $O(C)$ ，是常数级别。使用路径压缩方法进行优化的并查集构建步骤如图3-32所示，其中路径压缩在第(c)步体现。

图3-32 并查集的路径压缩构建步骤

3.2.11 匈牙利算法

匈牙利算法主要用于求解二分图的最大匹配问题。二分图又称为二部图，是一种特殊的图。假设 $G=(V, E)$ 是一个无向图，如果顶点集 V 可以分割为两个互不相交的子集 (A, B) ，并且图中的每条边 (i, j) 所关联的两个顶点 i 和 j 分别属于这两个不同的顶点集($i \in A, j \in B$)，则称图 G 为一个二分图。二分图最大匹配是指使二分图中左右两边相连但不相交的边数最多。

匈牙利算法的基本思路是查找增广路，增广路是指从一个未匹配节点出发，如果途经一个未匹配节点，则完成本轮增广路的查找，否则走交替路（出发点不算），直到交替路终止于对面未匹配节点，或者无边可走才结束。每次找出来的增广路都完成了一次两边节点的更新配对，有可能是新增配对，也有可能是修改之前的配对。

问题描述：现在有一个8人的小型化妆舞会，一共有4男4女，将男生分别标记为A、B、C、D，将女生分别标记为a、b、c、d。现在化妆舞会需要男女配对组成4队，在音乐声中一起完成华尔兹双人舞蹈。已知A与a、b、c相识，B与a、d相识，C与c、d相识，D只与b相识，熟人关系如图3-33所示。为了让大家都能与一个熟人作舞伴，要如何给这8个人配对？

图3-33 二分图中的熟人关系

针对上述例子，匈牙利算法的执行步骤如图3-34所示。

图3-34 匈牙利算法的执行步骤

按ABCD的顺序进行匹配，先匹配A和B。加粗虚线表示当前找到的增广路，加粗实线表示当前的配对。首先，图3-34 (a) 找到增广路 $A \rightarrow a$ ，图3-34 (b) 将增广路中的A和a配对；然后，图3-34 (c) 找到增广路 $B \rightarrow a \rightarrow A \rightarrow b$ ，图3-34 (d) 将增广路中的A和a解除配对，然后将B和a、A和b分别配对。这里可以看出，由于二分图的性质，增广路中每隔一条边匹配一条边，总能保证配对最大。接着配C和D。图3-34 (e) 找到增广路 $C \rightarrow c$ ，图3-34 (f) 将增广路中的C和c配对，图3-34 (g) 找到增广路 $D \rightarrow b \rightarrow A \rightarrow a \rightarrow B \rightarrow d$ ，图3-34 (h) 将增广路中的B和a、A和b解除配对，然后将D和b、A和a、B和d分别配对。至此，图中再无增广路，匈牙利算法结束。可知在每次查找增广路时，在最差情况下会遍历所有边。因此匈牙利算法的时间复杂度为 $O(NM)$ ，其中N表示二分图半边的点数，M表示二分图的总边数。

3.3 在线评测系统

在线评测系统 (Online Judge, OJ) 是一种在编程竞赛中评测参赛程序的在线系统，也可以用于平时练习。近年来也出现了一些针对求

职面试的在线评测系统。许多OJ网站会自发组织一些竞赛。此外，OJ网站通常会设立用户排名（以用户提交的答案通过数或某个题目的执行时间为排名依据）。

在OJ上提交评测程序后，系统后台会执行提交的程序，根据题目的要求输入数据，得到并评测该程序的输出结果。只有返回“Accepted”状态，才表示提交的程序正确。不同OJ的评测状态分类略有出入，大致可以分为3类，如表3-2所示。

表3-2 OJ的评测状态分类

比较有名的国外OJ有LeetCode、Topcoder，国内OJ有北大的POJ、浙大ZOJ，以及胡伟栋的Tsinsen清橙网络自动评测系统。

3.3.1 LeetCode

LeetCode是一种在线评测系统，不仅形成了题库、社区、竞赛三剑合璧的生态系统，而且打通了通往全世界各大互联网公司的求职通道。此外，LeetCode特别推出了LeetCode中国版的官方网站。

根据题目类型，可以将题目分为6类，分别为System Design、OO Design、Operating System、Algorithms、Database和Shell；根据题目难易程度，可以分为3类，分别为Easy、Medium和Hard。每题都有一个总的提交通过率，并且提供解题思路留言讨论区。题库还会提供若干个专题标签，用于给刷题者提供有针对性的索引。例如，算法类别专题标签有数组相关、图相关、动态规划相关；公司面试划分专题标签有谷歌公司面试最热100题、Facebook公司面试最热100题等。

3.3.2 POJ与ZOJ

POJ和ZOJ等以大学为代表的在线评测系统，主要用于竞赛培训，如IOI国际信息学奥林匹克竞赛、ACM国际大学生程序设计竞赛等。

POJ (Peking University Online Judge , 北京大学程序在线评测系统) 是一个提供编程题目的网站，它包含3000多道程序设计题，题目大部分来自ACM国际大学生程序设计竞赛和各种自行举办的竞赛，很多题目可以反映工作和生活中的实际问题。

ZOJ (Zhejiang University Online Judge , 浙江大学程序在线评测系统) 是一个提供信息学 (算法竞赛) 题库及进行在线程序评测的网站，它也包含3000多道程序设计题，题目大部分来自ACM国际大学生程序设计竞赛、全球各大赛区及著名大学的竞赛。

3.3.3 Tsinsen

很多人或许没有听说过Tsinsen清橙网络自动评测系统，但应该对其背后的开发者有所耳闻，那就是两届IOI金牌得主、清华大学计算机科学实验班毕业的胡伟栋。清橙网络自动评测系统和胡伟栋，对笔者而言并不陌生，因为笔者和胡伟栋曾就读于同一所高中，而且笔者也算是清橙的首批用户。

笔者记得在刚上高一时，参加了学校的信息学竞赛兴趣小组。那时我们不像现在有五花八门的OJ用于学习编程，尤其对于DIY题目，手动评测甚是麻烦。清橙网络自动评测系统就是那时由胡伟栋开发的一个自动评测系统，专门用于模拟竞赛评测。该系统后来升级，搬到了网上，也就是现在的清橙网络自动评测系统。比较遗憾的是，清橙网络自动评测系统于2019年9月1日不再对外提供服务。

3.4 动脑时刻

1. 为什么斐波那契数列不适合使用递归算法求解？

2. 如何使用递归算法解决汉诺塔问题？

3. 背包问题有很多变种与升级版本，本书讲解的是01背包问题，也是最基础的背包问题。如果同一种物品可以无限装包，这样的背包问题称为完全背包问题。那么，动态规划如何解决完全背包问题？

4. 导弹拦截问题，每次的导弹拦截高度都不能高于前一次，现在连续有 N 枚导弹袭击，高度分别为 H_i （ i 为 $1 \sim N$ 的整数），最多能拦截多少枚导弹？写出动态转移方程即可。

5. 为什么迪杰斯特拉算法无法计算带负边的图，而贝尔曼-福特算法可以？

6. 从算法原理上解释为什么普里姆算法适合稠密图，而克鲁斯卡尔算法适合稀疏图。

7. 如何计算树状数组中任意子序列中的所有元素之和？

8. 为什么说树状数组对计算任意子序列中的最大值和最小值束手无策？

3.5 本章小结

本章首先讲解了数据结构，包括数组、链表、队列、栈、树、图和散列表这几种数据结构的性质与用法。然后讲解了几种常用的基础算法，帮助大家拓展思维边界，提高思考能力。最后简单介绍了几个比较流行的在线评测系统，大家可以在这些在线评测系统上进行基础算法的实战训练。

数据结构和基础算法作为算法的基本招式，对之后更加顺利地学习算法的各种武功秘籍大有裨益。

关键词回顾

数据结构

数组

多维数组

链表

多维链表

队列

先进先出FIFO

栈

后进先出LIFO

树

二叉树

森林

图

有向图

无向图

邻接链表

邻接矩阵

散列表/哈希表

排序

选择排序

插入排序

冒泡排序

快速排序

堆排序

归并排序

计数排序

桶排序

基数排序

递归

分治

贪婪算法

动态规划

背包问题

动态转移方程

搜索

深度优先搜索

宽度优先搜索

最短路径

弗洛伊德算法

迪杰斯特拉算法

贝尔曼-福特算法

最小生成树

普里姆算法

克鲁斯卡尔算法

树状数组

线段树

二叉搜索树

平衡二叉树

单旋和双旋

并查集

路径压缩

匈牙利算法

在线评测系统

LeetCode

POJ与ZOJ

Tsinsen

第4章

算法之武功秘籍

归根结底，算法的目标是解决生活中的实际问题。在第2章和第3章中，我们讨论了算法内力（相关的数学基础知识）及算法招式（数据结构和常用的基础算法）。运用算法的内力和招式，可以解决生活中的大部分简单问题。但是当问题变得复杂时，如分类问题（如社交媒体的用户兴趣分类和热点关注话题分类）、推荐问题（如电商平台商品的推荐和排序问题）、预测问题（如预测股价走向、交通流量、商品销量）及与人工智能有关的问题（如怎样让计算机学会自动驾驶、下棋、对话）等，只用算法的内力和招式是远远不够的，此时需要更加精妙的算法武功秘籍。因为算法招式解决的只是某一类小问题，或者是大问题中的某一小块，就好比武当的梯云纵，解决的只是“腾空而上”这样的小问题。真正比武的时候，还要用太极拳、太极剑这类武功秘籍才可以克敌制胜。

本章会介绍机器学习算法的分类，并且详细介绍不同分类下的算法原理，帮助大家熟悉算法世界中的武功秘籍。

4.1 类别划分

机器学习涉及各式各样的算法，所解决的问题涵盖生活中的方方面面。本节分别从两大出发点归纳总结机器学习中的算法，帮助大家机器学习算法有一个整体把握。其中，第一个出发点为是否有监督信号，第二个出发点为学习目标。

4.1.1 按是否有监督信号划分

所有机器学习算法都是基于样本学习的。有标签的样本，相当于一个有经验的师父提前告诉你学习的目标是什么，以此监督你的学习过程，这个标签称为**监督信号**。带监督信号的机器学习称为**监督学习**（Supervised Learning），反之称为**无监督学习**（Unsupervised Learning）。当然，在监督学习和无监督学习之间，还存在**半监督学习**（Semi-Supervised Learning）。顾名思义，半监督学习就是所用样本中只有一部分带有监督信号的机器学习。

1. 监督学习

具体来说，监督学习是利用有标签的样本进行机器学习，并且调整模型的参数，使其达到所需性能要求的过程，这个过程又称为监督训练或教师学习。监督学习之所以又称为教师学习，是因为这个过程类似于教师授课、学生学习的过程。下面来看一个简单的例子。

问题描述：现有若干张画有不同种类动物（如猴子、老虎、狮子、大象、长颈鹿等）的卡片，每种动物卡片分别有各不相同的若干张，如何教会幼儿园的小朋友们认识这些卡片上的不同动物？

常规的教学流程是，如果拿出的是猴子卡片，就告诉小朋友们这是猴子，如果拿出的是狮子卡片，就告诉小朋友们这是狮子；以此类推，拿出画有哪种动物的卡片，就告诉小朋友们这是哪种动物。在反复教学几轮后，小朋友们大概就能区分哪些卡片是猴子，哪些卡片是狮子了。这便是所谓的监督学习过程，而事先告知的卡片与动物的对应关系，便是样本的标签，即监督信号，小朋友们对这些动物卡片的记忆方法就是监督学习算法。

常见的监督学习算法有**线性回归**（Linear Regression）、**逻辑回归**（Logistic Regression）、**K-近邻**（K-Nearest Neighbor，KNN算法）、**决策树**（Decision Tree）、朴素贝叶斯（Naive Bayes）、**神经网络**（Neural Network），还有用于降维的算法——**线性判别分析**（Linear

Discriminant Analysis, LDA)。无论采用哪种监督学习算法，其共同点都是教什么就学什么。即通过学习，使模型输出结果与指定样本标签值尽可能地接近。

2. 无监督学习

与监督学习相比，无监督学习没有监督信号，它利用未知标签的样本调整模型参数，学习某种特征、规律、趋势或既定模式，从而达到所需性能要求的过程。放眼回望，人类从原始社会一路走来，文明的进步何尝不是处处伴随着无监督学习？还以动物卡片为例，人类最开始并不知道猴子叫猴子、狮子叫狮子，只知道某类动物具有相同或类似的外貌特征。例如，某类动物经常用手挠头、活泼好动，喜欢在树上窜来窜去；某类动物体格健壮，颈部覆盖着厚厚的鬃毛，食肉。久而久之，同类型具有相同行为规律与外貌特征的动物，随着时间的推移逐渐被称为猴子和狮子。这便是无监督学习的过程，这种将动物归类的方法，其实就是无监督学习中的典型算法——聚类。图4-1所示为一个无监督学习示例。

图4-1 无监督学习示例

常见的无监督学习算法除了**聚类**（Clustering），还有降维算法——**主成分分析**（Principal Component Analysis, PCA）。聚类可以进一步分成4类，分别为**划分聚类**、**层次聚类**、**密度聚类**和**模型聚类**。

3. 半监督学习

半监督学习是监督学习与无监督学习相结合的机器学习，它使用大量未知标签的样本，同时使用部分已知标签的样本，用于学习某种特征、规律、趋势或既定模式。我们发现，在一些真实场景中，经常出现缺乏有标签样本的情况，而给样本打标签是一项十分费力、耗时的工作。显而易见，半监督学习的主要目的之一就是提高这方面的效

率。不过，并不是所有类型的样本都满足半监督学习的要求，半监督学习依赖以下3个前提假设。

- 平滑假设，处于某个维度空间的两个距离很近的样本，标签类似。
- 聚类假设，处于同一个聚类类别下的两个样本，标签类似。
- 流形假设，当两个样本从高维空间映射到低维空间的同一个局部邻域时，标签类似。

如果以上3个前提假设不成立，那么无标签样本不仅不能对模型的学习性能起改善作用，反而会损坏学习性能，导致半监督学习失效。但是，还有一些实验表明，在一些特殊情况下，即使前提假设成立，无标签样本也有可能损坏模型的学习性能。因此，要开展半监督学习，前期做好充分的数据调研尤为重要。

常见的半监督学习算法有**半监督聚类**（Semi-Supervised Clustering）、**生成对抗网络**（Generative Adversarial Nets，GAN）、**半监督分类**（Semi-supervised Classification）及一些基于图的算法等。

4.1.2 按学习目标划分

机器学习算法的类别除了可以根据是否有监督信号进行划分，还可以根据学习目标进行划分。顾名思义，学习目标就是算法模型最终要达到的目的。例如，识别动物卡片的算法模型，目的是区分不同的类别，用于回答“是”或“不是”类问题；销量预测、股票预测的算法模型，目的是预估某类数值的大小，用于回答“多少”类问题。这样，按照学习目标划分，可以将机器学习算法模型分成四大类。回答“是”或“不是”类问题的算法模型统称为**分类**（Classification），回答“多少”类问题的算法模型统称为**回归**（Regression）。除了分类和回归，另外两类算法分别是**聚类**（Clustering）和**降维**（Dimensionality Reduction）。

分类和回归的直观区分方法如下：模型的最终预测值如果是离散的二值（如0和1），就是分类；如果是一个连续的取值空间，就是回归。有读者或许已经发现，无论是分类还是回归，模型训练用的都是有标签的样本，因此它们都属于监督学习的范畴。

事实上，分类和回归已经涵盖了机器学习中的大部分热门模型，如最近几年特别流行的神经网络及其一系列变种（卷积神经网络、循环神经网络等）。常规的聚类属于无监督学习，当然也存在半监督聚类算法。降维主要有无监督学习的主成分分析（PCA）算法、监督学习的线性判别分析（LDA）算法等。

4.2 线性回归模型与逻辑回归模型

在监督学习模型中，线性回归模型和逻辑回归模型是最经典的两个模型，其中逻辑回归模型适用于大部分监督学习问题。不仅如此，学好线性回归模型和逻辑回归模型，对神经网络的学习也大有帮助。

4.2.1 线性回归模型

线性回归模型是最简单、直观的机器学习算法模型之一。线性是指特征值与输出值之间是一次函数关系。例如，对于 $f(\mathbf{x}) = \omega\mathbf{x} + \mathbf{b}$ ，其中 $\mathbf{x}$ 是特征值， $f(\mathbf{x})$ 表示输出值。回归是指通过调整一次函数的参数，使其尽可能准确地拟合数据的过程。

我们不妨从最简单的单特征线性回归模型讲起。所谓单特征，就是一个特征，假设预测目标只与一个因素相关。例如，要预测北京市房价，单特征是每套房子的建筑面积，我们的假设就是北京房价只与建筑面积有关。假设北京房价和建筑面积的关系如图4-2所示，坐标轴横坐标和纵坐标分别是建筑面积和每平方米价格。

图4-2 建筑面积与房价关系图

线性回归模型的目标是尽可能准确地拟合数据，即在图4-2中找到一条直线，使图中各点尽可能地接近这条直线。单特征线性回归模型公式为 $f(x)=ax+b$ ， x 为建筑面积， $f(x)$ 为每平方米的价格， a 和 b 是权重参数，其中 b 称为**偏置**（Bias），单特征下的 a 是一个标量，多特征下的 a 是一个向量。那么，图4-2中有这么多不同建筑面积的房价，如何找到一条合适的直线来表示它们的关系变化？这就说到机器学习中的模型训练方法了，目前最常用的模型训练方法为**梯度下降法**（Gradient Descent）。下面讲解梯度下降法具体是如何训练，从而求得建筑面积与房价之间的线性关系的。为了便于理解，假设线性回归模型拟合的直线经过原点，并且只考虑一个样本数据点(1, 4)，那么梯度下降法的训练过程如下。

（1）初始化模型权重参数 a ，假设初始值为2，由于经过原点， b 值固定为0，并且设置训练步长 $s=0.1$ ，设置训练终止阈值 $threshold=0.01$ 。

（2）将 $x=1$ 代入 $f(x)=ax$ ，计算实际输出值与真实值之差 $\Delta=f(x)-y$ ，得到的差值 Δ 即loss的值。如果loss的绝对值小于或等于 $threshold$ 的值，则结束训练。

（3）计算训练步长 s 与loss的积，即梯度，用权重参数 a 的值减去梯度的值来更新 a 的值，跳转至步骤（2），继续运行，直到训练结束。

按照上述流程，每轮迭代的权重参数 a 和梯度依次为(2,-0.2)，(2.2,-0.18)，(2.38,-0.162)，(2.542,-0.1458)，……直到loss的值小于 $threshold$ 的值，训练结束。此时的权重参数 a 和梯度十分接近(4, 0)，因此训练得到的回归函数为 $f(x)=4x$ ，训练迭代次数与loss之间的变化关系如图4-3所示。这个无限接近训练目标的过程，在机器学习中称为收敛（Convergence）。

图4-3 训练迭代次数与loss值之间的变化关系

当然，房价不可能只与建筑面积这一个因素有关系，机器学习算法模型的特征空间通常也不会只有一个维度。因此，一般使用向量 $\mathbf{x}$ 表示多特征值，使用向量 ω 表示多特征权重，使用标量 b 表示偏置，线性回归模型的公式就可以表示为 $f(\mathbf{x}) = \omega\mathbf{x} + b$ 。偏置的作用是无无论在几维空间，回归函数都可以表示不经过原点的图像。通常将偏置也放到 $\omega\mathbf{x}$ 中，具体做法是令特征向量 $\mathbf{x}$ 的每一维从下标1开始编号，将偏置作为权重 ω_0 与构造的特征值 $x_0=1$ 一起合入 $\omega\mathbf{x}$ 的乘积之中。这样，线性回归模型公式就简化成了 $f(\mathbf{x}) = \omega_0 \sim_n x_0 \sim_n \omega_n$ 。

4.2.2 逻辑回归模型

对于线性回归模型公式 $f(\mathbf{x}) = \omega\mathbf{x}$ ，其值域（取值范围）是未知的， $f(\mathbf{x})$ 相当于在计算 $\omega\mathbf{x}$ 之后，直接用线性函数 $y=x$ 将该结果“激活”输出。线性回归模型之名由此得来， $y=x$ 则被称为线性激活函数。既然有线性激活函数，自然就有非线性激活函数，非线性激活函数可以将未知值域映射到一个已知的值域范围内。其中最著名的非线性激活函数之一为Sigmoid函数。将Sigmoid函数作为激活函数的回归模型称为逻辑回归模型。Sigmoid函数的公式为

Sigmoid函数的图像如图4-4所示。

图4-4 Sigmoid函数的图像

根据Sigmoid函数的公式和图像可知，Sigmoid函数的值域为(0, 1)。将输出值映射到(0, 1)区间，目的是方便设定阈值完成二分类。无论特征维度的值是多少，逻辑回归模型都会利用阈值设定，在这个特征维度空间中将样本集合分为两部分。例如，二维空间可以用一条直线切分成两部分，三维空间可以用一个平面切分成两部分，以此类推。

除了Sigmoid函数，下面介绍另外3种常见的激活函数，它们分别为Sgn函数、Tanh函数和ReLU函数。其中ReLU函数在神经网络中应用广泛。

阶跃函数Sgn为二值输出函数，如输出-1和1，其函数图像如图4-5所示。由于阶跃函数具有不连续、不光滑的性质，而且输出值固定，因此不适合作为神经网络中的激活函数进行网络信息传递。

图4-5 Sgn函数的图像

双曲正切函数Tanh可以解决Sigmoid函数的输出值恒大于0的问题。其值域为 $(-1, 1)$ ，相当于将Sigmoid函数沿着纵轴拉伸了一倍，然后向下平移了一个单位。Tanh函数的公式为

Tanh函数的图像如图4-6所示。

图4-6 Tanh函数的图像

不过，在深度神经网络中使用Tanh函数和Sigmoid函数有可能造成梯度消失问题，这个将在后面详细讲解。

ReLU函数是一个非常简单实用的激活函数，其功能如下：如果输入小于0，则输出值均为0，否则输出值等于输入值。ReLU函数的值域为 $[0, +\infty)$ ，其函数图像如图4-7所示。

图4-7 ReLU函数的图像

在神经网络中使用ReLU函数能有效缓解梯度消失问题，尤其在深度神经网络中非常实用。ReLU函数的变种有LReLU函数、PReLU函数和RReLU函数等。

4.3 人工神经网络

在20世纪40年代，心理学家W.S.McCulloch和数理逻辑学家W.Pitts在分析、总结人类大脑神经元基本特性的基础上，提出了相应的数学模型。随着计算机硬件技术的飞速发展，神经网络从最初的单层感知结构发展成了如今流行的多层感知结构。那么，神经网络为何如此流行，具体又是如何工作的？

4.3.1 初识人工神经网络

根据生物学研究结论，人脑神经细胞数量的数量级大约为 10^{10} ，这些神经细胞之间通过突触（轴突和树突）相连。一种对于神经细胞运作的通俗理解为，神经细胞将所有树突传进来的信号相加，如果信号之和高于或低于某个阈值，那么细胞状态会相应地变为兴奋状态或抑制状态，由于大脑的神经细胞在不同时期的激活状态不同，因此在不同时期会表现出不同的大脑行为。

机器学习中神经网络的灵感来源于人脑神经细胞的这种信号传递现象。由于是人工模拟，因此这类机器学习算法的准确称呼为**人工神经网络**（Artificial Neural Network，ANN）。与人脑神经网络类似，人工神经网络由多个人工神经元相连组成，人工神经元的数学描述如图4-8所示。

图4-8 人工神经元的数学描述

首先，人工神经元用一个求和函数SUM（类似神经细胞的树突）来接收信息。在图4-8中， $x_1 \sim x_n$ 代表发送至该人工神经元的信号； $w_1 \sim w_n$ 代表这些信号作用在这个神经元上的不同程度，即这些信号的权重；偏置 b 从生物角度来看，可以理解为这个神经元本身所具备的信号弹性。然后，人工神经元用一个激活函数输出信号 y 。人工神经网络通常使用Sigmoid函数作为激活函数。与Sigmoid函数相比，ReLU函数具

有运算速度快、缓解梯度消失问题等优势，因此应用非常广泛，通常应用于深度神经网络的中间层（隐层）。

对于人工神经元构成的单层模型，激活函数不同，相应的名称也不同。事实上，线性回归模型和逻辑回归模型都属于人工神经网络的范畴。其中，使用线性激活函数 $f(x)=x$ 的单层神经网络模型，就是4.2节中的线性回归模型；使用非线性激活函数Sigmoid的单层神经网络模型，就是4.2节中的逻辑回归模型；使用阶跃函数作为激活函数的单层神经元模型，学术名称为**感知机**（Perceptron）。下面通过一个简单的例子讲解什么是单层神经元模型。假设有一男一女两个人，身高、体重分别是1.8m、70kg和1.6m、50kg，我们用这两个样本训练出一个能判断性别的单层神经元模型。

使用Sigmoid函数作为激活函数，单层神经元模型的表达式为

在上述表达式中， x 为特征输入（身高 x_2 ，体重 x_1 ，偏置1）， w 为要学习的特征权重。将两人的特征数据（1.8, 70, 1）和（1.7, 50, 1）重复输入模型中，用于学习参数向量 w 的值，学习方法采用前文讲到的梯度下降法，直到 $f(x=(1.8, 70, 1))$ 和 $f(x=(1.7, 50, 1))$ 的值分别趋向0和1，训练结束。

除此之外，人工神经网络衍生出了各式各样的结构变种，如**深度神经网络**、**卷积神经网络**、**递归神经网络**和**图神经网络**等，并且开辟了形形色色的研究方向。

4.3.2 深度神经网络

顾名思义，**深度神经网络**（Deep Neural Network，DNN）的神经元层数比较多，模型的输入、输出会经过比较深的人工神经网络结构，中间经过的人工神经网络层被称为隐层。既然是深度神经网络，到底多少层才能算深？业界其实没有统一的标准，一般少则三五层，

多则上千层，都可以称为深度神经网络。深度神经网络其实早在20世纪60年代就已经诞生，由于计算性能跟不上等原因沉寂了几十年。随着计算机硬件技术的飞速发展，以及深度学习的兴起，直到21世纪初，深度神经网络才逐渐热门起来。一个常规的深度神经网络结构如图4-9所示。

图4-9 常规的深度神经网络结构

所有机器学习算法的学习目标，都是寻找输出与输入之间最合适的映射关系。在图4-9中，如果深度神经网络中的每层人工神经元都使用线性函数 $y=x$ 作为激活函数，那么无论模型有多少层，模型最后的输出都会是原始输入经过线性变换的结果，最终效果相当于单层人工神经元模型的效果。与线性变换相比，非线性变换能够表达的信息要丰富很多。深度神经网络拥有强大的表达能力的原因，除了有深度外，每层神经元在激活输出时的非线性变换也至关重要。当然，深度神经网络同样面临很多训练方面的问题，如梯度消失和梯度爆炸。

深度神经网络由于实现比较简单、直观，模型的抽象表征能力较强，因此应用范围非常广，后文讲到的各种神经网络模型其实都可以加入深度神经网络结构，从而实现更加抽象、丰富的特征学习。

4.3.3 卷积神经网络

人类从呱呱坠地睁开眼的那一刻起，认识这个世界的主要方式之一就是视觉。人类通过视觉不断地认识身边事物的形状、颜色，并且在大脑中逐步构建出一个信息丰富且带有标签的万物图像库，从而通过看到的不同外貌来识别不同的事物。神经网络在**计算机视觉**（Computer Vision，CV）领域的深度运用，以及**图形处理器**（Graphics Processing Unit，GPU）的出现，使计算机也能像人脑一样，感知并识别这个五彩缤纷的图像世界。例如，从1000万张图片中识别出猫的Google Brain，以及击败人类围棋世界冠军的人工智能机器人

AlphaGo，背后都用到了—种图像识别技术，即**卷积神经网络**（Convolutional Neural Network，CNN）。

要理解卷积神经网络，先理解什么是卷积。我们不妨将“卷积”—词拆开来理解，“卷”意为翻卷，可以理解为数学变换中的翻转和平移的意思；“积”可以理解为积累、相加、翻倍的意思，因此“卷积”表示—种简单的数学操作。卷积操作通常被看作加权滑动平均法的一种推广。卷积神经网络就是运用了卷积操作的一种神经网络，运用卷积操作的神经网络层称为**卷积层**（Convolutional Layer）。卷积层由卷积核组成，卷积核的计算类似于人工神经元中 $y=\omega x+b$ 的计算，只是输入不限定是—维的 x_i ，可以是二维图像中的—个像素点 $x_{i,j}$ ，也可以是三维空间中—个物体在某—点的描述 $x_{i,j,k}$ 。每个卷积核通过学习不同的参数，可以感受局部视野，并且提取不同的局部特征。假设有—张 3×3 的图像，卷积核为 2×2 ，卷积核内初始权重均为1，那么卷积操作的计算方法如图4-10所示。

图4-10 卷积操作的计算方法示例

在图4-10中，卷积操作的具体计算过程如下。

左上： $3\times 1+2\times 1+4\times 1+1\times 1=10$ 。

左下： $4\times 1+1\times 1+7\times 1+9\times 1=21$ 。

右上： $2\times 1+6\times 1+1\times 1+5\times 1=14$ 。

右下： $1\times 1+5\times 1+9\times 1+3\times 1=18$ 。

卷积神经网络除了包含卷积层，还包含**池化层**（Pooling Layer）。池化分为最大池化和均值池化两种，最大池化是指每次取感受视野范围内的最大值作为输出，均值池化是指每次取感受视野范围内的平均值作为输出。池化层—般放在卷积层的后面计算，用于减少深度神经网络训练的参数数量，提高深度神经网络的训练速度和泛化能力。以

最大池化为例，池化大小为 2×2 ，图4-10中图像的池化操作计算方法如图4-11所示。

图4-11 池化操作的计算方法示例

在图4-11中，池化操作的具体计算过程如下。

左上： $\max\{3, 2, 4, 1\}=4$ 。

左下： $\max\{4, 1, 7, 9\}=9$ 。

右上： $\max\{2, 6, 1, 5\}=6$ 。

右下： $\max\{1, 5, 9, 3\}=9$ 。

当然，卷积神经网络的网络结构除了包括卷积层和池化层，还包括神经网络都有的组成部分，即输入层、全连接层和输出层。卷积神经网络在计算机视觉和图像识别领域发挥着至关重要的作用，如著名的AlexNet网络结构（详见第7章图7-1）。AlexNet是2012年ImageNet竞赛冠军获得者Hinton和他的学生Alex Krizhevsky设计的。在那年之后，很多更深的卷积神经网络相继被提出，如VGGNet和GoogLeNet。

4.3.4 递归神经网络

无论是深度神经网络还是卷积神经网络，相邻的两层都是从空间维度上进行展开与衔接的。我们知道在爱因斯坦的广义相对论中，空间和时间是两个并列的维度，共同组成了所谓的“四维时空”。那么，既然存在空间维度上展开的神经网络，自然也存在时间维度上展开的神经网络，这便是本节要讲的内容。

递归神经网络（Recursive Neural Network，RNN）又称为循环神经网络（Recurrent Neural Network），它会将一个网络结构单元的输出作为该网络结构单元在下一时刻的输入，所有神经网络层之间的衔接与

串联都会在时间维度上展开。一个典型的递归神经网络结构如图4-12所示。

图4-12 典型的递归神经网络结构

在图4-12中的递归神经网络结构中，隐层除了向输出层输出信息，还会将信息传递给下一时刻的自己，即下一时刻的隐层，这样做的目的是让神经网络在时间维度上产生“记忆”。

递归神经网络的典型应用领域为人机交互、自动翻译和文档补全等自然语言处理方面的场景。递归神经网络和深度神经网络一样，在模型训练过程中同样存在梯度消失问题，具体表现为容易“忘事”。由于递归神经网络是时间维度上展开的神经网络结构，因此递归神经网络需要升级自身模型结构，才能解决“忘事”这个难题。升级后的递归神经网络模型，叫作**长短时记忆**（Long-Short Term Memory，LSTM）**模型**。

作为一种升级版的递归神经网络模型，长短时记忆模型会在时间维度上增加若干个特定的神经网络结构，用于提升记忆关键信息的能力。长短时记忆模型的网络结构如图4-13所示。

图4-13 长短时记忆模型的网络结构

在图4-13中，大长方形框住的整体称为记忆单元，实线箭头表示当前时刻的信息传递，虚线箭头表示上一时刻的信息传递。与典型的递归神经网络模型相比，长短时记忆模型一共增加了3个神经网络层，这3个神经网络层在学术上分别称为**输入门**（Input Gate）、**遗忘门**（Forget Gate）和**输出门**（Output Gate）。进入记忆单元的箭头表示输入，从记忆单元出去的箭头表示输出。

总体而言，与普通的递归神经网络模型相比，长短时记忆模型的结构更复杂，可调整和变化的地方也更多。例如，增加peephole，将Cell的状态连接到每个门。长短时记忆模型可以简化为门控循环单元

(Gated Recurrent Unit , GRU) 模型。在长短时记忆模型中加入注意力 (Attention) 机制，可以产生多种变种模型。长短时记忆模型在语音识别、图像识别、疾病预测等领域都发挥着惊人的作用，是当下最流行的神经网络模型之一。

4.3.5 图神经网络

图神经网络 (Graph Neural Network , GNN) 是一种基于关系建模的神经网络。在讲解图神经网络之前，我们先回顾一下什么是图。根据第3章内容可知，图由顶点和边两部分组成。边可以分为有向边与无向边，包含有向边的图称为有向图，不含有向边的图称为无向图。图结构在现实生活中是广泛存在的，网站之间的超链接引用关系、社交网络中的朋友关系、学术界论文之间的引用关系都可以用图结构表示。基于图结构的研究课题有很多。例如，在社交网络的图结构中，将用户作为节点，将用户之间的朋友关系作为边，通过分析社交网络图结构中的信息，可以对图结构中的节点（用户）使用监督学习中的分类算法进行学习，进而对用户的兴趣、转发文章、评论等社交行为进行深入分析和预测。

与深度神经网络和递归神经网络不同，图神经网络由多层明确的神经网络层构成。图神经网络是一种直接运行在图结构上的神经网络。针对节点分类问题的图神经网络，其神经网络层聚合的信息不仅包括节点本身的特征，还包括与该节点相邻的边和节点的特征。考虑到相邻节点的位置、距离，在进行特征聚合时可以分为一度邻居、二度邻居和多度邻居。一个常规的图神经网络结构如图4-14所示。

图4-14 常规的图神经网络结构

图神经网络一般不直接用于进行分类或回归，通常将其作为图的一种表征学习融入其他模型的特征输入中。图的表征学习是指将图中的各种信息映射成一个N维特征向量，这种映射关系在机器学习中称为

特征嵌入（Feature Embedding）。图中各种信息的嵌入统称为**图嵌入**（Graph Embedding）。图嵌入最常见的用法有两种，分别为**点嵌入**（Node Embedding）和**边嵌入**（Edge Embedding）。如果输出信息是节点，则输出点嵌入的特征向量；如果输出信息是边，则输出边嵌入的特征向量。输出节点信息的应用比较常见。例如，从聊天网络和电商网络中识别出欺诈用户，从终端设备网络中识别出欺诈设备，等等。为了方便展示，图4-14中只用箭头标记了节点的输出信号，未标记边的输出信号。

经过各种嵌入后的特征向量可以和其他特征拼接使用，也可以直接输入神经网络的全连接层，继续加入模型进行学习。除了点嵌入和边嵌入，还有整图嵌入（Whole-Graph Embedding），这种用法在化学分子结构、蛋白质结构的分析探索中有较广泛的应用。

4.4 决策树

无论是回归模型，还是神经网络模型，都要通过纯数学变换和层层公式计算来获得期望的结果，解决问题的过程都不够人性化，让人难以直观理解。尤其是深度神经网络，即使最后的分类或回归效果可以做到极致，对于隐层在训练过程中发生了什么，模型到底是如何一步一步逼近最优效果的，到目前为止，业界也没有一个很好的答案，这就是模型可解释性问题。在深度神经网络发挥作用的场景中，深度神经网络模型的可解释性问题通常是用户最困惑、最担忧的问题之一。例如，随着时间的推移，模型的预测能力下降，从模型结构层面很难给出令人信服的理由。

本节介绍机器学习算法中可解释性最强的模型之一——**决策树**（Decision Tree）。近几年已经有学者研究如何利用决策树的性质解决深度神经网络模型的可解释性问题。

4.4.1 概念与方法

决策树是一种树形分类和回归模型，每个非叶子节点都相当于一个IF条件判断语句，模型通过逐个判定特征所属类别，对样本逐步进行分类。决策树模型不仅可解释性强，解决问题的角度也趋近于人类看待问题和理解问题的视角，比较简单、直观。下面通过一个例子，初步讲解决策树的工作原理。有10条关于西瓜的样本数据，如表4-1所示，我们要根据这些数据学习如何判断一个西瓜是否为好瓜。

表4-1 西瓜样本数据集

在表4-1中，西瓜有3种特征，分别为色泽、敲声、纹理。决策树会以某种顺序依次考查西瓜的每个特征，逐层缩小范围，从而判定这个西瓜是否为好瓜。由于树结构具有层次关系，并且决策树采用自上而下的建树模式，因此特征考查的顺序直接决定了最终分类的效果。根据顺序（色泽→敲声→纹理）构建决策树模型A，如图4-15所示。

图4-15 决策树模型A

然后根据顺序（敲声→色泽→纹理）构建决策树模型B，如图4-16所示。

图4-16 决策树模型B

对比决策树模型A和决策树模型B可知，不同的特征考查顺序的确会影响决策树的最终分类效果。那么，如何设置特征考查顺序，才能得到分类效果较好的决策树呢？这涉及信息论中的一个概念——**信息增益**（Information Divergence）。说到信息增益，不得不先介绍**熵**（Entropy）。熵在信息论中衡量的是数据的混乱程度，本质是香农信息量（ $p \cdot \log(1/p)$ ， $0 < p \leq 1$ ）的期望。信息量是可以衡量的，不然我们也不会觉得有时候说的话虽然多，却没什么信息量，而有的话却能一语中的。信息增益作为设置决策树特征考查顺序的依据，可以直观理解

为，在按照当前设置的特征考查顺序划分数据集后，如果可以最大程度减少数据的混乱程度，那么该特征考查顺序是好的特征考查顺序。

不妨举个例子，一个全是好瓜的数据集信息量为

$$-\log 1 = 0$$

此时该数据集的混乱程度达到最低，因为不用判断就知道里面全是好瓜；而对于一个一半是好瓜一半不是好瓜的数据集，信息量为

假设存在某个特征恰好能区分是否为好瓜，那么用这个特征划分这个数据集之后的信息增益计算方法为

ID3 算法每次选择最大信息增益的特征作为父节点，向下扩展决策树，直到不能继续扩展为止。

在计算信息增益时，我们发现，当属性取值较多时，其信息增益往往偏高。在极端情况下，如每个属性值只对应一个样本，信息增益必然等于最大值1。显然，这样的决策树泛化能力是最弱的。因此，**C4.5 算法**应运而生，该算法使用信息增益率的概念（信息增益率等于信息增益除以数据集中该特征的总信息量），大致思路如下：先选出信息增益高于平均水平的特征，再从中选择信息增益率高的特征，用于当前分裂。此外，还有一种CART算法，使用基尼指数选择特征，也能增强决策树的泛化能力。

4.4.2 剪枝

在决策树变复杂之后，由于判断分支过多，因此训练集中的某部分样本特征被当作所有样本的一般性质，就出现了**过拟合**（Over

fitting) 问题。决策树解决过拟合问题的主要方法是剪枝。剪枝分为预剪枝和后剪枝两种。

1. 预剪枝

节点依据信息增益和增益率扩展子节点，在这个过程中，验证集的整体准确率会随着节点的分裂增加发生变化。如果某个节点的扩展导致验证集的分类准确率降低，则舍弃该节点，这便是预剪枝。预剪枝不仅能降低过拟合风险，还能大大降低决策树的建树时间复杂度。但是，预剪枝由于使用贪婪算法决定是否对当前节点进行分裂，因此不能权衡决策树的全局利弊，并且会给最终模型带来欠拟合风险。

2. 后剪枝

后剪枝是一种全局优化方法。在决策树的建树过程全部完成后，后剪枝从下往上依次考查各个非叶子节点，考查方法和预剪枝一样，如果某个节点的扩展在验证集上并未提高准确率，则舍弃该节点。后剪枝既降低了过拟合风险，又降低了欠拟合风险。但是，由于后剪枝是在生成完整决策树之后进行的，因此模型训练的整体时间复杂度会比较高。

4.4.3 梯度提升决策树

由于决策树简单有效，可解释性强，因此可以将其包装成一些更高效的机器学习算法，如梯度提升决策树和随机森林。梯度提升决策树和随机森林之间的关系类似于串行和并行之间的关系，本节我们先讲解梯度提升决策树。

梯度提升决策树 (Gradient Boosting Decision Tree, GBDT) 是一个与梯度有关、对决策树进行提升的机器学习模型。梯度提升决策树

可以是分类树，也可以是回归树。我们不妨从后往前依次分解DT、B、G这几个定语，逐步理解GBDT的精髓。

DT（Decision Tree，决策树）。决策树分为分类树和回归树两种。回归树与分类树的原理机制相似，区别在于分类树只在叶子节点返回唯一分类，而回归树的每个节点都能返回一个回归值，通常为当前节点内所有样本的平均值。

B（Boosting，提升）。提升是指在原来模型的基础上做进一步的效果提升，提升决策树（BDT）的基本思路是采用多棵决策树串行建模。具体过程为，对于第一棵树之后的每一棵决策树，都基于前一棵决策树的预测结果进行二次建模，在这样串行建模的过程中，逐步修正预测结果，从而得到最优解。

G（Gradient，梯度）。梯度的大小反映了当前预测值与目标值之间的距离或差异。因此，在上述的串行建模过程中，除了第一棵决策树使用原始样本数据进行训练外，之后的每一棵决策树都使用前一棵决策树的预测值（平均值）与实际值之差（负梯度，可以理解为残差或增量）进行建树。这相当于对分错的样本有针对性地继续分类，使样本的总体残差趋近于0。由于除第一棵决策树外的其他决策树都会对目标的残差或增量进行建模预测，因此梯度提升决策树模型需要将串行建树过程中每一棵决策树的输出结果相加，才能得到最终的预测输出结果。下面通过一个经典例子“预测年龄”来讲解梯度提升决策树的具体工作过程。假设有4个样本，其特征描述分别如下。

- 样本A，消费较高、经常被学弟问问题，27岁。
- 样本B，消费较高、经常问学长问题，23岁。
- 样本C，消费较低、经常被学弟问问题，17岁。
- 样本D，消费较低、经常问学长问题，13岁。

梯度提升决策树串行建模的过程如图4-17所示。

图4-17 梯度提升决策树串行建模的过程

在图4-17中，梯度提升决策树模型对各个样本的预测结果，即每棵树预测结果的和。A的年龄为27（ $25+2$ ）岁，B的年龄为23（ $25-2$ ）岁，C的年龄17（ $15+2$ ）岁，D的年龄为13（ $15-2$ ）岁。

梯度提升决策树的核心思想概括如下：串行训练 n （ $n > 2$ ）棵决策树，其中第 i （ $1 < i \leq n$ ）棵决策树学习第 $i-1$ 棵决策树的负梯度（可理解为残差或增量），将 n 棵决策树的输出结果累加作为最终输出结果。

4.4.4 随机森林

随机森林（Random Forests，RF）是指由多棵决策树组成的决策树森林。组成随机森林的决策树既可以是分类树，又可以是回归树。与梯度提升决策树模型相比，随机森林模型的原理更直观、简洁，性能也更强悍。随机森林算法的核心思路有两个：**采样**和**完全分裂**。采样分为**行采样**和**列采样**，这里的行与列对应的是样本与特征两个维度。完全分裂是指决策树在每一次分裂扩展节点时，如果能分裂，就必须分裂，分裂依据可以是信息增益或信息增益率。

对于行采样，模型从 M 条数据中随机采样 m 条数据，分别作为每一棵决策树的训练集。在一般情况下， m 为 M 的算术平方根。行采样保证了每棵决策树使用的训练集各不相同，这在一定程度上抑制了过拟合问题。

对于列采样，每一棵决策树都从 M 个特征中随机挑选 m 个特征作为节点分裂特征来计算。在一般情况下， m 为 M 的算术平方根。列采样分为两种方式，一种是全局列采样，即同一棵树的建树过程均采用同一批采样的特征；另一种是局部列采样，即每一次节点分裂均单独随机挑选 m 个特征进行分裂。与行采样相比，列采样进一步保证了随机森林不会出现过拟合的问题。

随机森林采用并行建模方式，最终的模型预测结果由每一棵决策树的预测结果共同决定。如果是分类树，则通过投票产生最终分类；如果是回归树，则计算所有结果的平均值。

随机森林和梯度提升决策树都属于集成学习（Ensemble Learning）的范畴。集成学习是指包含多个子模型共同建模的机器学习算法。“三个臭皮匠，顶个诸葛亮”，这便是集成学习算法的精髓。

4.5 聚类

生活中有许多地方需要对事物进行分类，如超市货架、各大商场的品牌专柜、学校的社团等。将具有相同性质、相似功能的事物划分在一起的方法，在机器学习中称为**聚类**（Cluster）。

4.5.1 距离度量

我们知道，样本由特征组成，特征分为连续（有序）型特征和离散（无序）型特征两种。样本之间的距离度量是指样本特征之间的距离度量。

对于连续型特征的距离度量，最常用的是**闵可夫斯基距离**（Minkowski Distance），其计算公式如下。

上述公式首先计算两个样本之间每个特征值之差的 p 次方，然后求和，最后取 p 次方根。公式中的 i 表示特征序号， p 的值不同表示的距离含义不同。当 p 为1时，计算的是**曼哈顿距离**（Manhattan Distance）；当 p 为2时，计算的是**欧氏距离**（Euclidean Distance）；当 p 趋于无穷大时，计算的是**切比雪夫距离**（Chebyshev Distance）。

闵可夫斯基距离有两个明显的缺点：一是特征之间的量纲（单位）被等同对待，这通常是不合理的。二是未考虑特征值本身的分布规律。例如，如果特征值的分布规律类似于长尾分布，那么这种距离度量的效果通常会大打折扣。第一点可以采用特征值加权或归一化统一量纲解决，第二点可以采用特征值标准化或离散化解决。

除了闵可夫斯基距离，常见的连续型特征距离度量方法还有**马氏距离**（Mahalanobis Distance）、**余弦相似度**（Cosine Similarity）、**皮尔逊相关系数**（Pearson Correlation Coefficient）。

对于离散型特征的距离度量，最直接的度量方法是0/1匹配法，即对应位置的特征相同则为1，不同则为0，累加值的大小体现了样本之间的相似程度。基于0/1匹配法，还可以使用**杰卡德相似系数**（Jaccard Similarity Coefficient）进一步计算样本特征之间的距离。指定两个集合A和B，将杰卡德相似系数定义为A与B交集的元素数量与并集中的元素数量的比值，计算公式如下。杰卡德相似系数的值越大，说明相似度越高，当A和B都为空集时，杰卡德相似系数为1。

对于离散型特征的距离度量，还可以采用**值差分度量法**（Value Difference Metric，VDM）。值差分度量法的使用前提是样本必须是带有类别标签的。其算法原理如下：通过计算不同的离散型特征值在相同类别标签下所占比例的大小关系，得出离散型特征之间的相似度。

4.5.2 划分聚类

划分聚类是指根据确定的距离度量规则，将样本数据逐一归类，最终达到“类内样本数据足够近，类间样本数据足够远”的效果。

k均值（k-Means）**算法**是划分聚类的代表性算法，该算法的执行流程如下。

(1) 随机选择 k 个样本作为初始的 k 个类别中心。

(2) 将样本逐一划分到与这 k 个类别中心度量距离最小的类别。

(3) 将每个类别的中心点坐标（特征值）更新成当前类别下所有样本特征距离的平均值。

(4) 重复步骤(2)和(3)，直到类别中心位置不再发生变化或已达到最大迭代次数。

根据上述算法执行流程可知， k 均值算法受噪声和离群点的影响较深。因为类别的中心点坐标是根据样本特征距离的平均值更新的，所以一旦出现离群较远的样本数据，中心位置就会受到很深的影响。基于这个问题， **k 中心点（ k -Medoids）算法和 k 中值（ k -Medians）算法**应运而生。

k 中心点算法在每次更新类别中心点坐标时，都会通过枚举每个类别下的数据点，计算当前类别下其他数据点到该点的距离之和，将距离之和最小的点定为新的类别中心点。也就是说， k 中心点算法的类别中心点坐标一定是一条具体的样本数据，而不是计算出来的特征距离平均值。与 k 中心点算法不同， k 中值算法在每次更新类别中心点坐标时，都会将类别中心点更新成同类别下所有样本数据的中位数，直到每个类别的中位数不再发生改变。

k 中心点算法和 k 中值算法虽然都在一定程度上解决了噪声和离群点问题，但是由于在更新类别中心点坐标时不再是简单地取平均值，因此算法的时间复杂度会随之增加。此外， k 中值算法如果遇到离散型特征数据，就需要使用离散型特征的距离度量法进行距离度量。这时会有个新的算法—— k -Modes算法。 k -Modes算法的距离度量法可以是交集集中的元素数量、杰卡德相似系数等。

4.5.3 层次聚类

在划分聚类中，我们需要事先指定类别数量，然而人工指定的类别数量往往不是最优类别数量。本节要讲的层次聚类，无须事先指定聚类数量。该类聚类算法更关心同类别下样本之间的相似性，优先从相似性出发，而不是预先指定聚类数量。

所谓层次，就是类似于树结构的展开与回溯，逐层合并或拆分样本数据集。逐层合并是自底向上的聚类方法，称为**凝聚层次法**。如果使用凝聚层次法，那么开始的每个样本都是一个类，然后根据相似性度量法寻找同类样本，自下而上逐层地形成较大的类别。逐层拆解是自顶向下的聚类方法，称为**分裂层次法**。如果使用分裂层次法，那么开始的所有样本都属于同一个大类，然后根据相似性度量法逐层拆分样本，自上而下逐层形成较小的类别。

由于大部分层次聚类法采用的是凝聚层次法，因此本节重点讲解凝聚层次法的执行流程。

凝聚层次法的执行流程类似于哈夫曼树的构建过程。哈夫曼树每次都会选择权值最小的节点组成一棵新树，直到所有节点合并成一棵权值路径最短的树。同理，凝聚层次法会根据样本之间的最小距离来合并成类，具体步骤如下。

(1) 每个样本独自为一类，计算两个类别之间的最小距离。

(2) 合并距离最小的两个类为一个新类。

(3) 重新计算产生的新类与剩余所有类之间的距离。

(4) 重复步骤(2)和(3)，直到只剩下一个类或类间距离大于某个阈值。

类间距离的计算方法可以采用最小距离法、平均法等。其中最小距离法计算的是类别之间最近样本的距离；平均法是最常用的方法，先计算两个类别之间所有样本的距离，然后取平均值作为这两个类别之间的距离。

层次聚类不仅可以解决划分聚类需要事先指定类别数量的问题，在聚类过程中还可以发现类别之间的层次关系，并且能够聚出各种形状类别。不过，在计算类间距离时，层次聚类的算法复杂度很高，而且有可能形成链状类别。

4.5.4 密度聚类

层次聚类的最大问题是有可能形成链状类别，密度聚类则不存在这个问题。密度聚类是一种基于样本密度的聚类方法，类似于在一张地图上做人口密度分布图，密集区域通常被认为是城市所在地。因此，密度聚类是最直观的聚类方法。密度聚类的主要目标是寻找高密度区域，与距离聚类的不同之处在于，后者的结果通常表现为球状簇，而前者可以形成任意形状。只要参数设置合理，就不会形成链状类别，并且不会受噪声和离群数据点影响。

密度聚类算法都会涉及邻域和数量。**DBSCAN** (Density-Based Spatial Clustering of Application with Noise) 为比较有代表性的密度聚类算法，该算法用半径 R 表示邻域大小，用 N 表示邻域 R 范围内的样本数量。根据参数 R 和 N ，DBSCAN算法将样本分为以下3类。

- 核心点。在邻域 R 范围内数量不小于 N 的样本数据点。
- 边界点。在邻域 R 范围内数量小于 N 的样本数据点，并且该样本数据点落在其他核心点的邻域内。
- 噪声点。既不是核心点也不是边界点的样本数据点。

如果只采用核心点计算聚类，那么可以将DBSCAN算法看作 k 均值算法的变种。因为加入了边界点和噪声点的计算，所以DBSCAN算法比一般的聚类算法更强大。DBSCAN算法的执行流程如下。

- (1) 将所有样本数据点划分为核心点、边界点或噪声点。

- (2) 删除所有噪声点。
- (3) 将邻域 R 范围内的核心点连接成边，从而形成连通图 G 。
- (4) 计算连通图 G 中的连通分量 GCC_i ，形成初步聚类。
- (5) 将边界点划分到与之关联的核心点所在的类别。

在使用DBSCAN算法完成聚类后，我们不妨试想一下每个类内部有什么特点。首先，类内不同的核心点之间一定能画出至少一条经过全部核心点的线路。其次，不同的边界点之间，一定能通过某些直接或间接相连的核心点间接相连。这里涉及3个密度概念：**密度直达**、**密度可达**和**密度相连**。

如图4-18所示，如果从核心点 p 出发，可以直接与点 q 相连，则称 p 密度直达 q ；如果核心点 p 在经过若干次密度直达后，可以间接与 q 相连，则称 p 密度可达 q ；如果核心点 p 可以分别密度可达边界点 q 和 s ，则称 q 和 s 密度相连。密度可达属于密度相连，密度直达属于密度可达。根据密度概念我们发现，每个类中的任意两点之间都是密度相连的。

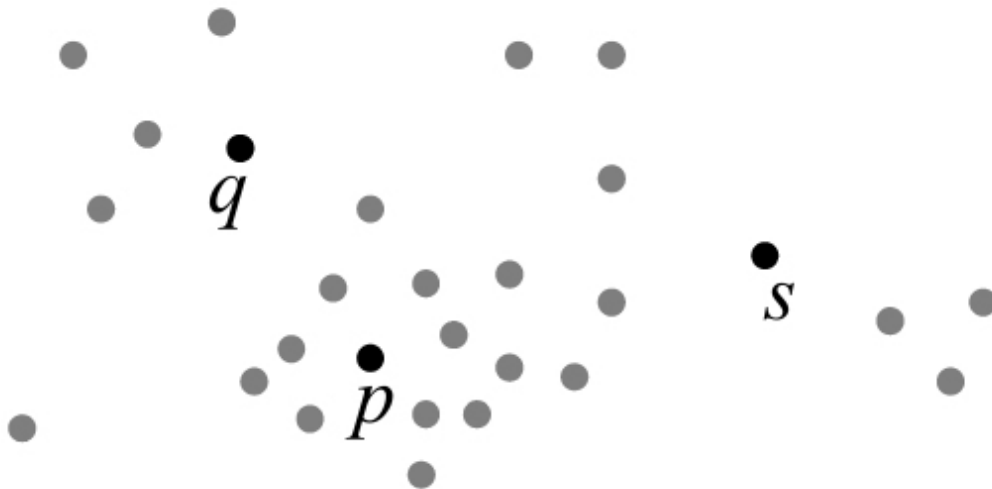


图4-18 密度概念区分

虽然密度聚类算法优点很多，但是其时间复杂度也非常高，一般通过构建数据索引来降低其时间复杂度。

4.5.5 模型聚类

模型聚类是指使用既定的算法模型计算不同样本之间的差别程度，进而将其归为不同的类别。模型聚类主要分为基于概率模型的聚类和基于神经网络模型的聚类，其中基于概率模型的聚类较为流行。在基于概率模型的聚类中，高斯混合模型聚类算法是最典型、最常用的模型聚类算法之一。

顾名思义，**高斯混合模型**（Gaussian Mixture Models，GMM）由高斯模型组成。高斯模型就是我们常说的正态分布，因此高斯混合模型可以理解为几个正态分布的叠加。高斯混合模型聚类过程与k均值算法的执行流程类似，具体步骤如下。

- （1）随机生成k个高斯分布作为初始的k个类别。
- （2）对于每个样本数据点，计算其在各个高斯分布中的概率。
- （3）对于每个高斯分布，将样本数据点在该高斯分布中的概率作为权重，加权计算并更新其平均值和方差。
- （4）重复步骤（2）和（3），直到每个高斯分布的平均值和方差不再发生变化或已满足迭代次数。

单特征的高斯混合模型聚类算法的执行流程图例如图4-19所示。

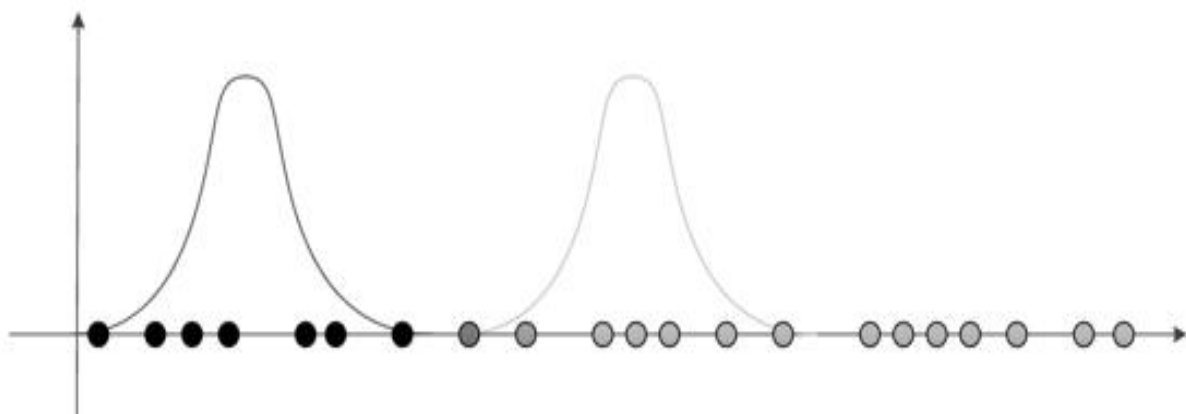
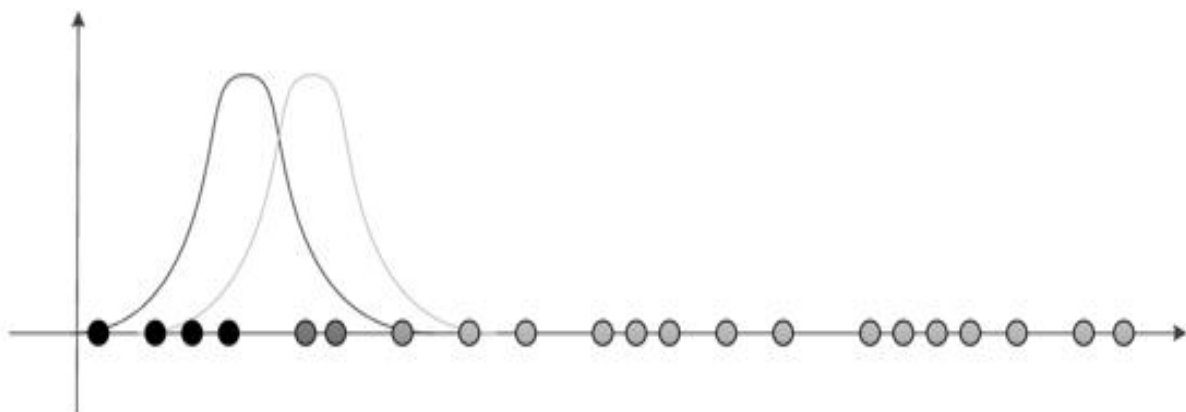
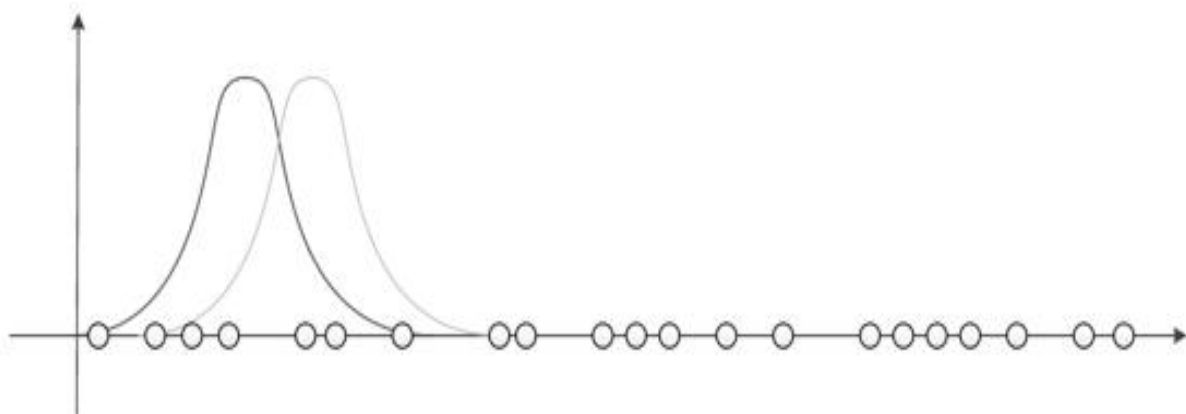


图4-19 单特征的高斯混合模型聚类算法的执行流程图例

如果样本数据集是多维情况，则需要计算协方差，将不同维度之间的关联性考虑进来。高斯混合模型聚类和k均值聚类一样，分类受类别个数的初始值影响较大。不过高斯混合模型聚类完的样本可以同时属于多个类别，这种聚类又称为**软聚类**。其他的模型聚类算法还有基于PageRank的软聚类算法、基于神经网络模型的自组织映射（Self-Organizing Maps，SOM）聚类算法等。

4.6 贝叶斯分类

托马斯·贝叶斯（Thomas Bayes，1702—1761）是18世纪英国的神学家、数学家、数理统计学家和哲学家，他是概率论理论的创始人，创立了贝叶斯统计，是“归纳地”运用数学概率，“从特殊推论一般、从样本推论全体”的第一人。**贝叶斯分类**（Bayesian Classification）算法是基于统计学的一种分类算法，其特点是使用概率表示所有形式的不确定性，学习或推理都要用概率规则来实现。

4.6.1 概率基础

概率（Probability）衡量的是事情发生的可能性大小。例如，投掷一个均匀的骰子，1~6中任意一个点数朝上的可能性都为1/6。在第2章，我们对概率相关的基本概念有了一个初步了解，本节进一步给大家讲解两种概率（先验概率、后验概率）和4个公式（加法公式、乘法公式/条件概率公式、贝叶斯公式、全概率公式），这些公式不仅能帮助大家更好地理解贝叶斯分类算法，在生活中的应用也十分广泛。

1. 先验概率

先验概率是指在检验与验证之前，根据经验预先推测出来的概率。例如，投掷一个均匀的骰子，我们事先就能推测每个面朝上的概

率是1/6。

2. 后验概率

后验概率是指在得到实验结果之后重新校验的概率，可以理解为对先验概率的修正。

3. 加法公式

加法公式如下：

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$A \cup B$ 表示事件A发生或事件B发生， $A \cap B$ 表示事件A和事件B同时发生。直观理解，加法公式的意思是事件A和事件B任意一个事件发生的概率，等于二者发生的概率之和减去二者同时发生的概率。加法公式对于任意两个事件A和B均成立，不要求A与B之间是条件独立的。加法公式示意图如图4-20所示。

图4-20 加法公式示意图

4. 乘法公式/条件概率公式

乘法公式/条件概率公式如下：

$$P(A \cap B) = P(A)P(B|A) = P(B)P(A|B)$$

$P(B|A)$ 表示在事件A发生的前提下，事件B发生的概率，即事件B的条件概率。直观理解，乘法公式的意思是事件A和B同时发生的概率，与事件A发生的概率和事件B的条件概率之积相等，并且与事件B发生的概率和事件A的条件概率之积相等。因此乘法公式又称为条件概率公式，其示意图如图4-21所示。

图4-21 乘法公式示意图

5. 贝叶斯公式

贝叶斯公式如下：

不难看出，贝叶斯公式是由乘法公式变换而来的。在机器学习中，贝叶斯公式通常写为

其中 D 是样本数据集， $P(h)$ 表示在没有进行样本训练之前预测目标 h 的初始概率，即前面提到的先验概率，先验概率反映了关于目标 h 的所有前提认知。而在机器学习中我们通常更关心后验概率 $P(h|D)$ ，即样本训练集 D 中目标 h 成立的条件概率。

6. 全概率公式

全概率公式如下：

在全概率公式中， B_i 表示两两不相同的时间段， $P(A|B_i)$ 表示在 B_i 时间段内事件 A 发生的概率，由于 B_i 两两不相容，因此事件 A 的全概率公式 $P(A)$ 可以看作加法公式。

4.6.2 朴素贝叶斯分类

在机器学习中，如何利用上面讲到的两个概率和4个公式进行分类呢？这便是朴素贝叶斯分类要做的事情。贝叶斯分类是指根据已分类样本的先验概率，估计待分类样本的后验概率。使用贝叶斯公式表示分类目标函数如下（其中 c 为类别， x 为样本）：

但是，在样本具备多个特征的情况下，很难根据训练样本得到上式中条件概率 $P(x|c)$ 的概率分布。针对这个问题，朴素贝叶斯分类应运而生。朴素贝叶斯分类避开了这个难题，对条件概率 $P(x|c)$ 的概率分布做了条件独立的假设，即不同特征之间两两不相关。既然满足条件独立，就可以采用乘法公式计算 $P(x|c)$ 的概率分布，公式如下（ d 为特征总数）：

有了条件概率的简化计算方式，即可得到朴素贝叶斯分类的目标函数，公式如下：

其中， $P(c)$ 可以直接统计训练样本中各个类别的比例。对于条件概率 $P(x_i|c)$ ，如果 x 是离散型特征，则可以通过计算在 c 类别中第 i 个属性取值为 x_i 的比例。如果 x 是连续型特征，则需要先假设特征符合某种分布规律，如常见的二项分布、高斯分布、泊松分布、伯努利分布等，再计算相应的条件概率。在得到条件概率后，计算出概率最大的类别，就是朴素贝叶斯分类模型的预测结果。

需要注意的是，在计算条件概率的过程中，有可能出现概率为0的情况，从而导致连乘之后的值为0。这里需要引入**拉普拉斯平滑系数**，简单理解就是分子、分母同时加上一个常数，用于避免概率为0的情况发生。可以证明，当训练集足够大时，加入拉普拉斯平滑系数的估计值会趋近真实的概率。

4.7 支持向量机

支持向量机（Support Vector Machine，SVM）是一种非常有潜力的分类模型。与逻辑回归模型、神经网络模型相比，支持向量机有更强的数学理论背景。因此与其他分类模型相比，支持向量机的学习门槛相对较高。

拆开来看，支持向量机分为支持向量和机两部分。前面提到，机在机器学习领域是算法的另一种称呼，如感知机、波兹曼机等。支持向量是指一些比较特别的样本，这些样本在真正意义上决定了支持向量机的模型参数取值。

在4.2节中，我们举了房价和建筑面积之间关系的例子。在这个例子中，我们给每个样本增加一个“是否成交”标签。在构建样本数据点后，使用线性回归模型得到的分类结果如图4-22所示，其中房价和建筑面积这两个特征分别为坐标系的横轴和纵轴，黑色样本点表示已成交，灰色样本点表示未成交。

图4-22 线性回归模型的分类结果

显而易见，线性回归能将“已成交”和“未成交”的样本数据划分开。如果将图4-22中划分的虚线稍微平移一点，或者旋转任意一个微小角度，其实也能达到同样的划分效果。那么，究竟哪一条虚线划分的效果是最好的呢？一种直观的判断方法为，虚线与两边样本之间平均距离越大越好。理论上如此划分的虚线，泛化能力是最好的，而支持向量机要做的就是找出与两边样本之间平均距离最大的虚线。

在形式上，支持向量机的模型公式和线性回归模型公式一样，也是 $f(\mathbf{x})=\mathbf{w}\mathbf{x}+\mathbf{b}$ 。支持向量机给出的最优划分结果如图4-23所示。

在图4-23中，两条灰色虚线与黑色虚线之间的垂直间隔距离相等。其中，处于两条灰色边界线上，支撑最优划分的样本数据点是支持向量，支持向量机的优化目标是最大化这个垂直间隔距离。

图4-23 支持向量机的分类结果

大家不妨先回忆一下点到直线的距离公式。直线方程为 $Ax+By+C=0$ ，点的坐标为 (x_0, y_0) ，点到直线的距离公式如下：

我们将正、负例样本标签分别设置为1、-1，将支持向量机的模型公式 $y=wx+b$ 代入公式（4-1），可以得到支持向量机的距离公式：

根据该公式可知，如果要最大化这个距离，则需要最小化权重向量 w 的L2范数。朴素的支持向量机和加了L2的线性回归模型几乎一模一样，所有我们可以像训练线性回归模型一样训练支持向量机。此外，使用**拉格朗日乘数法**（Lagrange Multiplier Method）和**KKT条件**（Karush-Kuhn-Tucker Conditions）有助于更优雅地解决支持向量机的相关问题，感兴趣的读者可以自行查阅相关资料。

4.8 动脑时刻

1. 逻辑回归和线性回归相比，非线性映射函数Sigmoid的优势体现在哪里？（提示：特征表达）
2. 阶跃函数是否适合作为神经网络中的激活函数，为什么？（提示：导数）
3. 图4-24所示为一个深度神经网络结构示例，尝试使用链式法则推导出反向传播的参数更新公式。

图4-24 深度神经网络结构示例

4. ReLU函数为什么可以缓解梯度消失问题？

5. 图4-25所示为卷积神经网络某一层的图例，尝试推导出卷积核的参数更新公式。

图4-25 卷积神经网络某一层的图例

6. 尝试推导出4.3.4节中递归神经网络的参数更新公式。

7. 尝试推导出4.3.4节中长短时记忆模型的参数更新公式。

8. 尝试写出图神经网络中可能的聚合算子种类。

9. 在密度聚类中，如何判断两点是否密度相连？

4.9 本章小结

本章首先从算法分类开始讲起，介绍了算法的分类方法。然后讲解了最常见的几种机器学习算法，包括线性回归、逻辑回归、人工神经网络、聚类、贝叶斯分类和决策树等，帮助大家机器学习算法有一个初步认识。当然，机器学习算法作为算法的武功秘籍，每种秘籍都有更深入的细节内容与前沿发展动向，本着普及算法的主要目的，本书旨在抛砖引玉，点到为止，感兴趣的读者可以继续深挖学习。

在掌握了各大主流机器学习算法的基础之后，笔者相信在遇到不同种类的机器学习问题场景时，读者应该能够大概判断出在不同场景中分别适合用哪一类算法模型求解了。

关键词回顾

监督信号

监督学习

无监督学习

半监督学习

线性回归

逻辑回归

K-近邻算法

决策树

朴素贝叶斯

神经网络

线性判别分析

主成分分析

聚类

划分聚类

层次聚类

密度聚类

模型聚类

半监督聚类

生成对抗网络

半监督分类

分类

回归

降维

线性回归

偏置

梯度下降法

收敛

逻辑回归

激活函数

Sigmoid

Sgn

Tanh

ReLU

人工神经网络

深度神经网络

递归神经网络

卷积神经网络

图神经网络

隐层

卷积层

池化层

长短时记忆模型

嵌入

图嵌入

决策树

信息增益

熵

ID3算法

C4.5算法

CART算法

剪枝

预剪枝

后剪枝

梯度提升决策树

随机森林

集成学习

距离度量

闵可夫斯基距离

曼哈顿距离

欧氏距离

切比雪夫距离

马氏距离

余弦相似度

皮尔逊相关系数

杰卡德相似系数

值差分度量法

划分聚类

k均值算法

k中心点算法

k中值算法

k-Modes算法

层次聚类

凝聚层次法

分裂层次法

密度聚类

DBSCAN

密度直达

密度可达

密度相连

模型聚类

高斯混合模型

软聚类

贝叶斯分类

概率

先验概率

后验概率

条件概率公式

贝叶斯公式

全概率公式

朴素贝叶斯分类

拉普拉斯平滑系数

支持向量机

拉格朗日乘数法

KKT条件

第5章

算法工程的组成部分

在武侠小说中，无论是内力、招式，还是武功秘籍，如果不组成一套完整的实战打法与人比武切磋，则终究比不出孰优孰劣、孰高孰低。同样地，对于前面讲解的算法之内力、算法之招式和算法之武功秘籍，如果不进行有效的串联与整合，而是生搬硬套，那么不仅在实际运用过程中会困难重重，而且取得的最终效果也会大打折扣。在算法世界中，将这三者融合在一起并运用的完整流程就是本章要讲的核心内容——算法工程。

本章会从原始数据的预处理开始，到算法模型验收为止，逐步讲解算法工程的完整流程。这个流程主要包括5部分，分别为数据分析、特征工程、建模与调参、效果评估和模型托管。

5.1 数据分析

在《三国演义》中，诸葛亮在调兵遣将时经常说：关羽听令，给你三千精兵，从左路包抄曹军；张飞听令，给你五千精兵，从右路包抄曹军。由此可知，诸葛亮在用兵打仗之前，必然已经掌握了己方有多少将领、精兵、残兵、粮草、战场地势等情报。再如成语“量体裁衣”，是说按照身材尺寸裁剪出合适的衣服，用于比喻做事要从实际出发，因地制宜。算法工程也是同样的道理。在算法工程中，能做到知己知彼、“量体裁衣”的关键步骤，非数据分析莫属。

无论是数据挖掘、机器学习相关的算法比赛，还是工业界的推荐、排序等场景中的算法应用，如果没有足够的数据分析结论作为支撑，那么能够取得的结果通常都不甚理想。为什么数据分析对机器学习算法结果的影响如此之深？有哪些实用的数据分析方法，这些方法在算法工程中又是如何起作用的呢？

5.1.1 宏观把握数据

拿到数据，切勿着急生搬硬套模型，先统计各类指标或特征的数量，把握数据大局。这个数量包括但不限于样本数据集的总数量、不同类别样本的数量、各类特征或指标的数量、缺失值的数量及特征值的分布等。宏观把握数据是数据分析的基础，也是容易被忽视的一点，它能帮助我们避免一些基本问题。例如，在后续工作开展之前，先搞清楚所定算法目标是否合理，可以帮助我们少走弯路、不走死路，从而提高算法工程的实施效率。

举个例子，在不考虑分布式训练的情况下，如果需要知道算法模型从训练任务启动到完成需要花费多长时间，则可以用训练样本的总数量除以batch size，再乘每批次训练花费的时间，就能估算出一个epoch花费的总时间，最后乘epoch的数量，即可得出算法模型的总体训练时间。如果在训练之前没有统计样本的总数量，那么模型训练何时结束，就只能对着屏幕默默等待了。

一份拥有512条数据、特征维度为1000的数据集，假设batch size为128，一个epoch包含4个batch，epoch和batch之间的关系如图5-1所示。

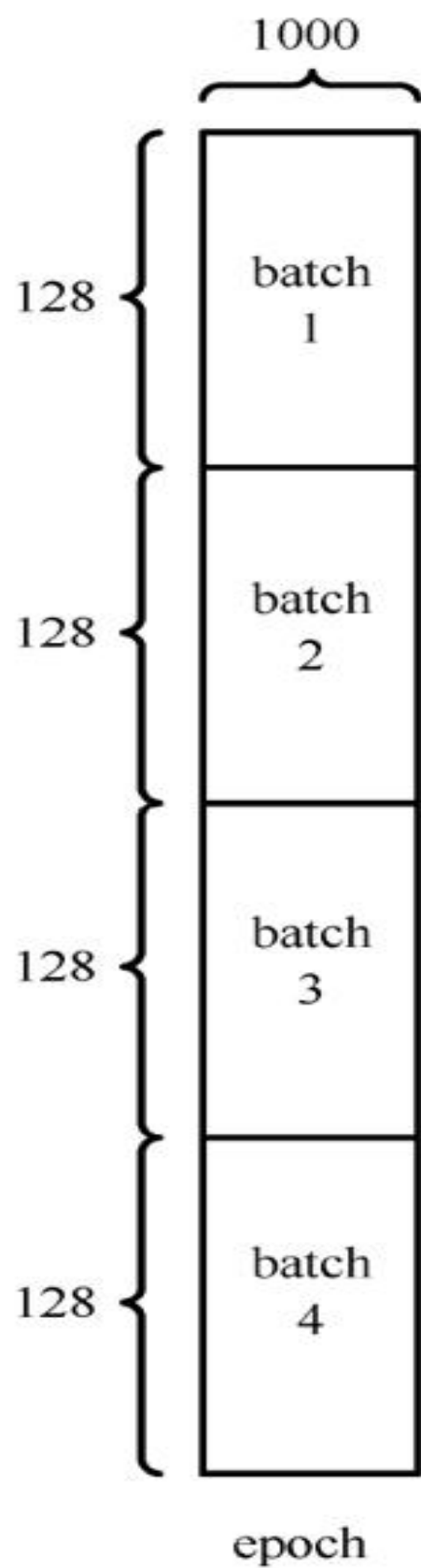


图5-1 epoch和batch之间的关系

除此之外，如果发现验证集或测试集中正、负例样本的一方过少，如只有1个正例样本的极端情况，则可以提前终止无意义的训练过程，因为针对极端比例下的类别样本数据集，无论是模型训练，还是模型评估，都会完全失效。事先统计各类别样本的数量，能提前发现样本不均衡的问题，尤其在类别样本比例差别较大时，一方面能帮助划分训练集、验证集和测试集的大小，另一方面可以对样本进行重采样或降低采样比例。样本不均衡问题会在5.2.3节的特征工程技巧中进一步讲解。

在一个动物分类数据集中，针对不同动物类别的数量统计结果如图5-2所示。

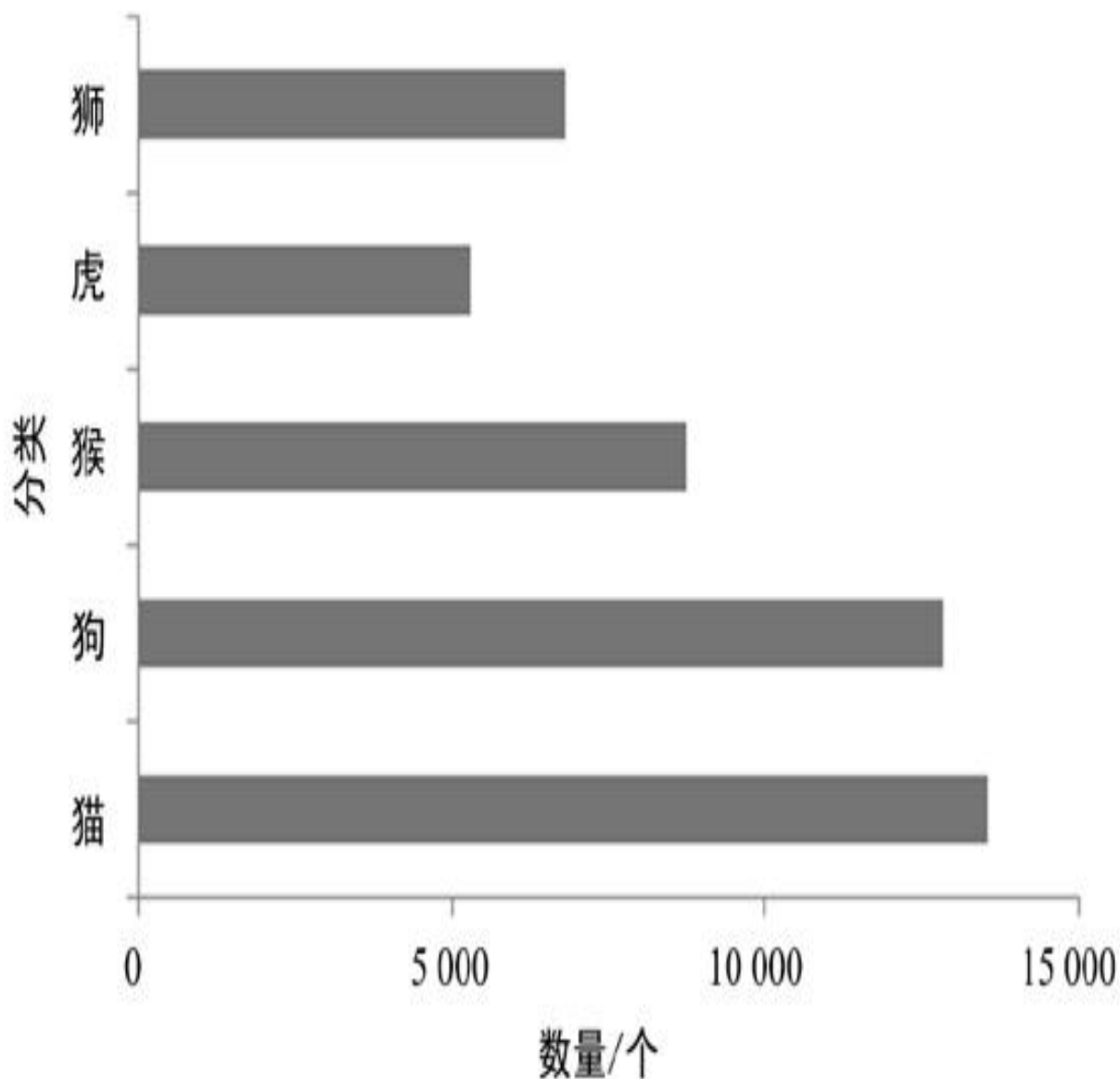


图5-2 动物分类数据集中不同动物类别的数量统计结果

对于数据字段的缺失问题（为空值或非法值），事先统计有助于掌握特征字段、标签字段等关键字段的缺失程度。如果缺失字段占比较低（通常1%以下可以忽略不计），那么可以直接丢弃；如果缺失字段占比较高，那么可能是源头数据出问题了，可以检查源头数据的收集过程是否存在问题，避免在训练过程中再出现问题，从而提高算法工程的整体效率。

总之，在动手开始搭建算法模型前，对样本数据集先有一个宏观把控，可以帮助我们对算法目标有一个更清晰、具体的认识，从而做到少走弯路、有的放矢。

5.1.2 微观感受数据

作为一门复杂学问，在机器学习算法工程中，无论采用什么模型，都不可能得到百分之百精确的结果。训练再好的回归模型，也不可能将预测值与真实值之间的预测误差降为0；即使分类效果再好的分类模型，也不可能保证每次的分类结果一定是正确的。在机器学习算法工程中，我们的工作尽可能地逼近目标和真相。除了继续调参优化模型，在数据层面，一方面，可以通过宏观把握数据提前发现问题；另一方面，可以微观感受数据，抽样观察单个样本数据的行为表现并思考其合理性，对算法工程的开展也大有帮助。例如，在包含用户行为序列的场景中，抽样观察具体的用户行为序列，设身处地地理解和感受用户行为特征，对用户行为序列的建模大有裨益。

以电商场景为例，要真正理解用户数据，就必须对用户数据进行充分的观察。总体而言，抽样观察用户数据的方式主要有以下两种。

第一，抽样观察用户个体维度的行为数据，换位思考，设身处地观察具体的用户行为序列，可以帮助我们更好地理解业务。行为数据是指用户在时间维度上的连续动作，如页面曝光、键盘输入、页面点击、鼠标滑动、触屏等行为。图5-3所示为一段完整的用户网上购物行为序列。笔者自从事电商行业相关算法工作以来，多次发现靠直觉得来的关于用户行为的观点，与用户的实际行为表现并不相符。例如，直觉上认为用户应该先单击商品A，然后单击商品B，由于运营规则、前端规则或用户偏见等原因，用户的实际行为表现可能会恰恰相反。

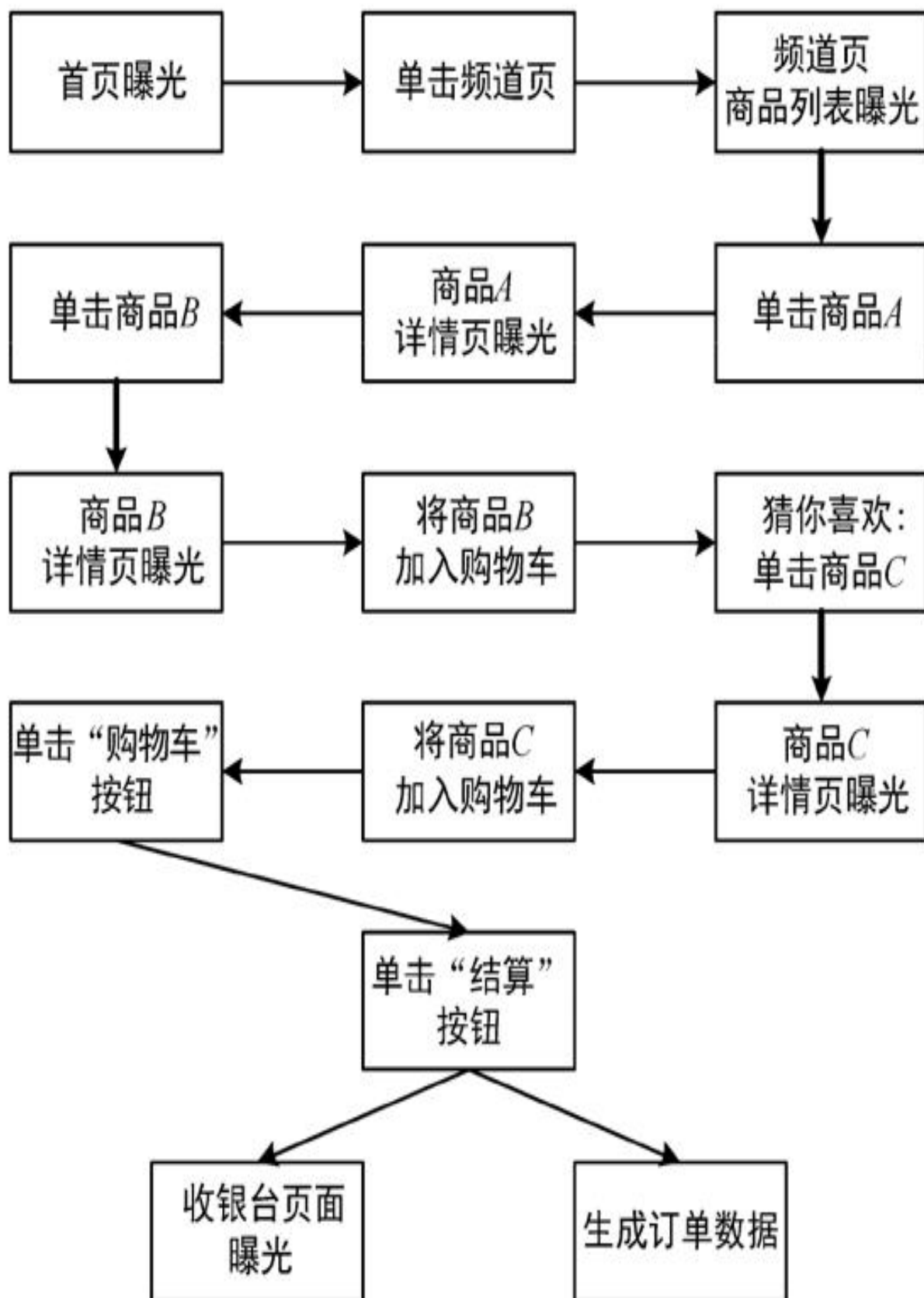


图5-3 用户网上购物行为序列示例

第二，抽样观察用户个体维度的统计指标，可以帮助我们更好地进行业务建模。并不是所有业务场景都适合使用复杂模型，有时基于规则的方法在某些业务场景中更容易出效果。笔者曾经跟进过一个保险返利项目，项目目标是挖掘参与保险购买活动的作弊用户，从而减少因不正当手段获取返利带来的损失，净化拉新指标。在项目前期，我们直接使用人工标记的作弊订单作为样本标签，基于逻辑回归训练了一个作弊分类器。但是，该作弊分类器在真实数据集上的分类效果很差。后来，我们根据对保险业务的重新理解，统计相关总体指标，提炼出几条满足非正常用户的规则，并且通过规则筛选出了一批作弊订单，然后在验证阶段通过电话逐一回访，发现大部分用户填写的手机号是无法拨通的空号，因此确认这些订单均为作弊订单。在这个反作弊场景中，基于规则的算法不仅简单易实现，作弊订单命中率非常高，而且具备极强的模型可解释性，只需根据当前作弊订单所命中的作弊规则，给出相应的规则描述。这便是微观感受数据带来的价值。

5.1.3 分析方法

无论是从宏观角度分析数据，还是从微观角度分析数据，都可以统一归为两个步骤：**计数**和**展示**。

1. 计数

所谓计数，就是围绕特定维度，通过特定方法计算得到统计量。统计量可以是绝对值类型，如人数、金额；也可以是相对值类型，如占比、点击率、转化率。

举例一，样本数量。维度可以是样本总体，也可以是不同的样本标签。基于这些维度可以统计得到样本总数量、不同样本标签下的总数量，如电商购物场景中每天的订单总量、销量排名前十的商品、不同种类下的订单总量、不同种类销量排名前十的商品等。

举例二，特征分布。维度是特征。如果是离散型特征，那么直接统计不同特征值出现的次数；如果是连续型特征，那么先进行等频或等宽的特征离散化处理，再统计每个特征区间出现的次数。例如，在电商购物场景中，将年龄特征作为统计维度，可以得到不同年龄段的顾客数量和订单数量；将历史消费额度特征作为统计维度，可以得到不同消费层次的顾客数量和订单数量。

举例三，用户行为。维度是标识用户的唯一编号。根据用户的唯一编号，可以关联该用户在某个时间段内的所有行为数据，然后按照时间的发展顺序排列，从而完整复现该用户在平台上产生的一连串动作序列。例如，在电商购物场景中，使用用户的账户编号追溯一次完整的用户购物过程，包括页面展现、页面点击、商品购买/加购、付款等行为。

举例四，商品访问量。维度是标识商品的唯一编号。根据商品的唯一编号，可以统计该商品在不同时间段内的访问量，对用户编号进行去重统计，还可以得到该商品在不同时间段内的访客数。

2. 展示

所谓展示，就是将统计结果直观地呈现出来。常见的展示方式有3种，分别为表格、图形和文本。

1) 表格。

当需要展示多维度数据时，可以采用表格这种既常见又实用的统计结果展示方式。表格不仅可以呈现丰富的数据，还可以在数据之间进一步对比，对于关键值和重要指标，可以通过加粗或改变颜色的方式突出显示。

2) 图形。

图形是最直观的统计结果展示方式。常用的图形有折线图、散点图、柱形图和饼图。其中**折线图主要用于看趋势，散点图主要用于看**

分布，柱形图主要用于看高低，饼图主要用于看占比。

3) 文本。

并不是所有的统计结果都适合使用表格或图形进行展示。例如，当统计结果只有一两个指标时，可以对其进行总结性的观点描述。虽然文本呈现的内容不如表格多，呈现效果不如图形直观，但它可以直接传达数据背后的意义。

5.2 特征工程

数据和特征决定了机器学习的性能上限，而机器学习算法只是在逼近这个上限。在具体问题中，数据样本空间通常是固定不变的，因此特征的质量尤为关键。提高特征质量的关键，就是**特征工程**

(Feature Engineering)。特征工程的本质是一种工程，所有关于特征数据的处理和加工，都属于特征工程的范畴。一方面，大部分机器学习算法对非结构化特征数据束手无策，因此必须通过数据预处理将某些原始数据格式转换成模型可接受的读取格式；另一方面，机器学习算法虽然具备一定的数据抽象学习能力，但是特征可能有千万种，如何从中选择特征、融合特征，甚至创造特征，从而提高算法模型的学习效率和质量，就显得尤为重要。特征工程主要解决的就是这两方面的问题，其作用是给模型算法铺路，将晦涩、粗糙的特征转换成对模型友好的特征。

5.2.1 数据预处理

在提取特征之前，需要对所用数据进行预处理，确保数据的可用性。数据预处理通常会检查缺失值和错误值。在缺失比例和错误比例可接受范围内，可以通过取平均值、中位数或众数的方式填补缺失值，也可以直接将包含错误值的数据样本行或列删除。如果缺失比例或错误比例过高，就需要考虑数据源本身是否存在问题了。

5.2.2 特征分类

总体而言，特征分为离散型（无序）特征和连续型（有序）特征两大类。离散型特征为文本类型的特征，如姓名、性别、邮编、城市、文档，离散型特征之间不存在顺序和大小关系。连续型特征通常为数值类型的特征，如年龄、时间戳、价格、销量，连续型特征之间存在顺序或大小关系。

5.2.3 工程技巧

1. 缺失值处理

是否丢弃包含缺失值的样本集的计算结果对比如表5-1所示。其中，左边为包含缺失值的样本集及丢弃包含缺失值的样本后的样本集的计算结果，右边为没有缺失值的样本集及相应的计算结果。

表5-1 是否丢弃包含缺失值的样本集的计算结果对比

姓名	▼	体重	▼	性别	▼	篮球	▼		姓名	▼	体重	▼	性别	▼	篮球	▼
小王		58		男		喜欢			小王		58		男		喜欢	
小李		61		男		喜欢			小李		61		男		喜欢	
小张		65		女		不喜欢			小张		65		女		不喜欢	
小刘		59				喜欢			小刘		59		女		喜欢	
小马		62		男		不喜欢			小马		62		男		不喜欢	
小汪		66		女		喜欢			小汪		66		女		喜欢	
小赵		60				喜欢			小赵		60		女		喜欢	
小孙		55		男		不喜欢			小孙		55		男		不喜欢	
性别	▼	人数	▼	打篮球	▼	比例	▼		性别		人数		打篮球		比例	
女		2		1		50%			女		4		3		75%	
男		4		2		50%			男		4		2		50%	
缺失		2		2		100%										

根据表5-1可知，在丢弃包含缺失值的样本后，女生中喜欢打篮球的比例发生了很大变化（从75%变为50%），因此不要轻易丢弃包含缺失值的样本。对于离散型特征缺失值，最简单的处理方法为填入空字符串；对于连续性特征缺失值，最简单的处理方法为补0。

关于缺失值的具体处理标准，对训练集而言，如果包含缺失值的训练样本太多，如超过一半，则说明收集的数据整体有问题，必须重新收集数据；如果包含缺失值的样本比例不高，则可以根据业务知识考虑是删除该样本，还是删除整个特征列。对测试集和验证集而言，如果是连续型特征，则可以使用训练集中该特征的平均值或中位数填补缺失值；如果是离散型特征，则可以使用训练集中该特征的众数（出现频率最高的特征值）填补缺失值。

2. 特征变换

常用的特征变换方法有三种：连续型特征离散化、特征编码和特征缩放。

1) 连续型特征离散化。

最常用的方法为**分箱**（Binning）（又称为分桶），分箱是处理连续型特征的一种方法。连续型特征的取值个数是无限的，而值与值之间轻微的变化对预测目标的影响较低，分箱可以基于这种轻微变化的规律，将线性特征转换成非线性特征，从而提升特征的表达能力。分箱的方式有三种。第一种为**等频分箱**，在特征的取值范围内，按照特征值从小到大的顺序排列样本，同时将样本按照数量均分成N份，每份作为一箱，同一个箱子中的特征值相同（通常用0和1表示）。例如，将样本按照某特征从小到大的顺序排列，然后按照每桶样本数量相等的方式将其分成4箱，如图5-4所示。第二种为**等宽分箱**，在特征的取值范围内，将特征值按照从小到大的顺序排列，并且将其均匀划分成N份，落在同一份中的样本属于同一箱，同一箱特征向量的结果相同，示例如图5-5所示。第三种为**聚类分箱**，和前两种按照频数或取值等分的方式不同，这种方式通过聚类算法将相似样本特征值聚在一起作为分桶特征。与人为指定分桶方式的等频分箱和等宽分箱相比，聚类分箱更灵活、合理。

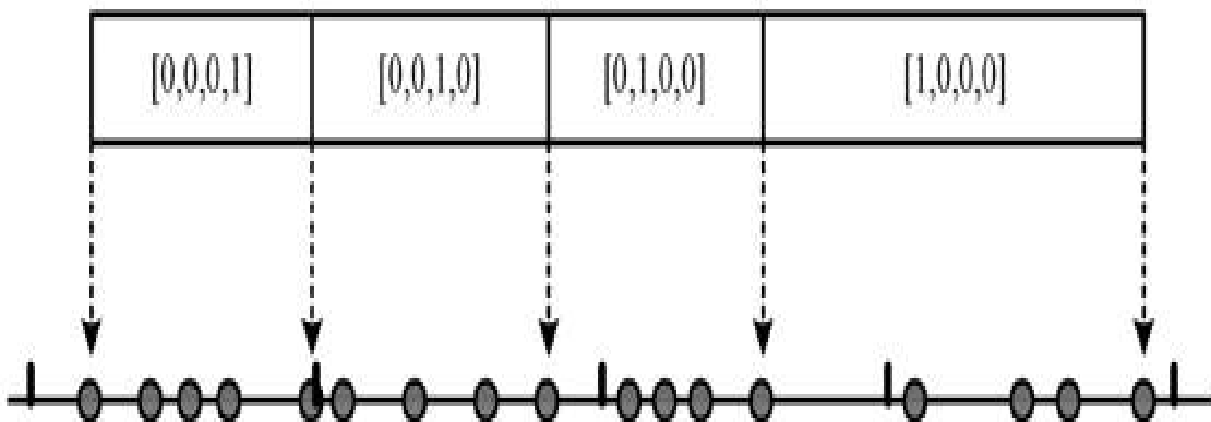


图5-4 等频分箱示例

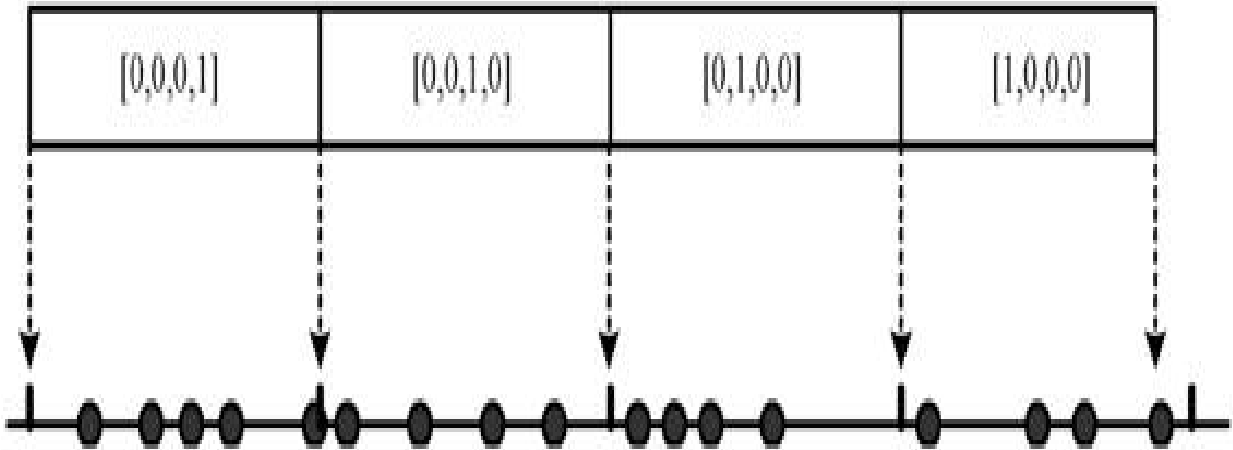


图5-5 等宽分箱示例

离散化之后的特征对于异常数据具有较强的健壮性，模型会更稳定。此外，离散化操作相当于引入了非线性特征，提升了特征的表达能力。不仅如此，离散化后的特征也比较方便和其他离散型特征进行后续的特征交叉等处理。

2) 特征编码 (Feature Encoding)。

当拿到的原始特征数据是文本类型的数据时，一般需要通过特征编码的方式将其转换成模型可识别的数值型数据。常用的特征编码方式有三种。第一种是**手动建立映射字典**，根据业务知识和目标任务，为离散型特征建立数字索引，用于进行映射。第二种是**label encoding**，这种方法仅适用于有序特征，对于一个有 m 个类别特征的样本数据集，在经过label encoding处理后，每个类别都会映射到 $0 \sim m-1$ 的整数上。第三种是**独热编码**，即One-hot编码，这种方法很常见，一个类别特征的取值一共有 m 种，在经过独热编码处理后，该类别特征会变为长度为 m 的0-1向量，并且只有特征值对应的位置为1。值得一提的是，前面讲到的分箱其实也是独热编码的一种实现方式。

独热编码的具体做法是将离散型特征所有可能的取值按照一定的顺序排列，同时生成一个维度相同且值全为0的向量；然后将对应样本离散型特征所处的位置设置为1。例如，编码图5-2中的动物类别离散型特征，编码结果如表5-2所示。

表5-2 独热编码示例

类 别	独 热 编 码
猫	[0, 0, 0, 0, 1]
狗	[0, 0, 0, 1, 0]
猴	[0, 0, 1, 0, 0]
虎	[0, 1, 0, 0, 0]
狮	[1, 0, 0, 0, 0]

3) 特征缩放 (Feature Scaling) 。

顾名思义，特征缩放就是将原先取值范围较大的特征，按照某种规则转换成取值范围较小的特征。在机器学习算法应用中，有时原始特征的取值范围很大，如果不预先采取特征缩放操作，压缩特征的取值范围，那么模型算法会无法正常运行，甚至会由于特征空间过大而无法加载到内存空间中。一方面，如果特征之间的量纲差别很大，那么特征之间的距离计算会失去意义；另一方面，特征的取值范围较大可能导致一些优化算法（如梯度下降）的收敛变得迟缓甚至失效。

特征缩放有三种常用方法，第一种叫作**对数变换**（Log Transformation）。对数变换将取值范围很大的特征转换到范围较小的区间内，对特征分布的形状有很大影响，可以降低特征值的震荡影响，弱化特征噪声，使特征分布更加匀称。如图5-6所示，在销量预测中，由于销量值波动很大，因此取自然对数可以弱化剧烈波动带来的影响。当然，由于是取对数，因此对零值或负值需要特殊处理，如统一设置为0。

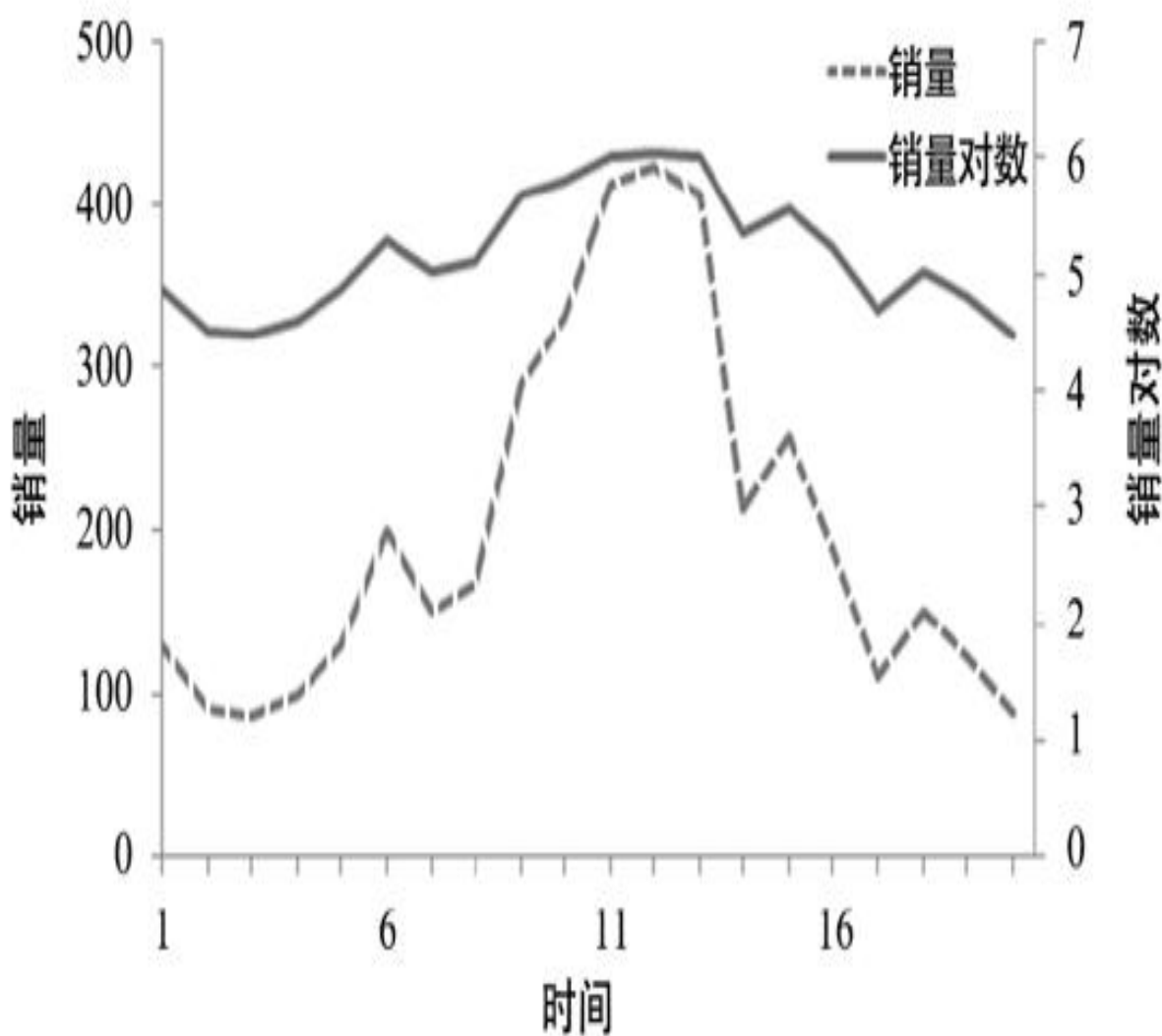


图5-6 对数变换的作用

特征缩放的第二种方法叫作**归一化**（Normalization），具体计算方法如下：首先计算得到当前特征的最大值和最小值，然后将原始特征值先减去最小值，再除以最大值与最小值之差，最后将特征值统一映射到[0, 1]区间。

特征缩放的第三种方法叫作**标准化**（Standardization），具体计算方法如下：首先计算得到当前特征的平均值和标准差，然后将原始特征值先减去平均值，再除以方差，最后将特征值统一映射到标准正态分布中。

特征缩放主要有两个作用，第一个是使梯度下降的过程不容易发生震荡，能更快收敛，如图5-7所示；第二个是在一些设计距离度量的模型算法中，起到统一量纲的作用。

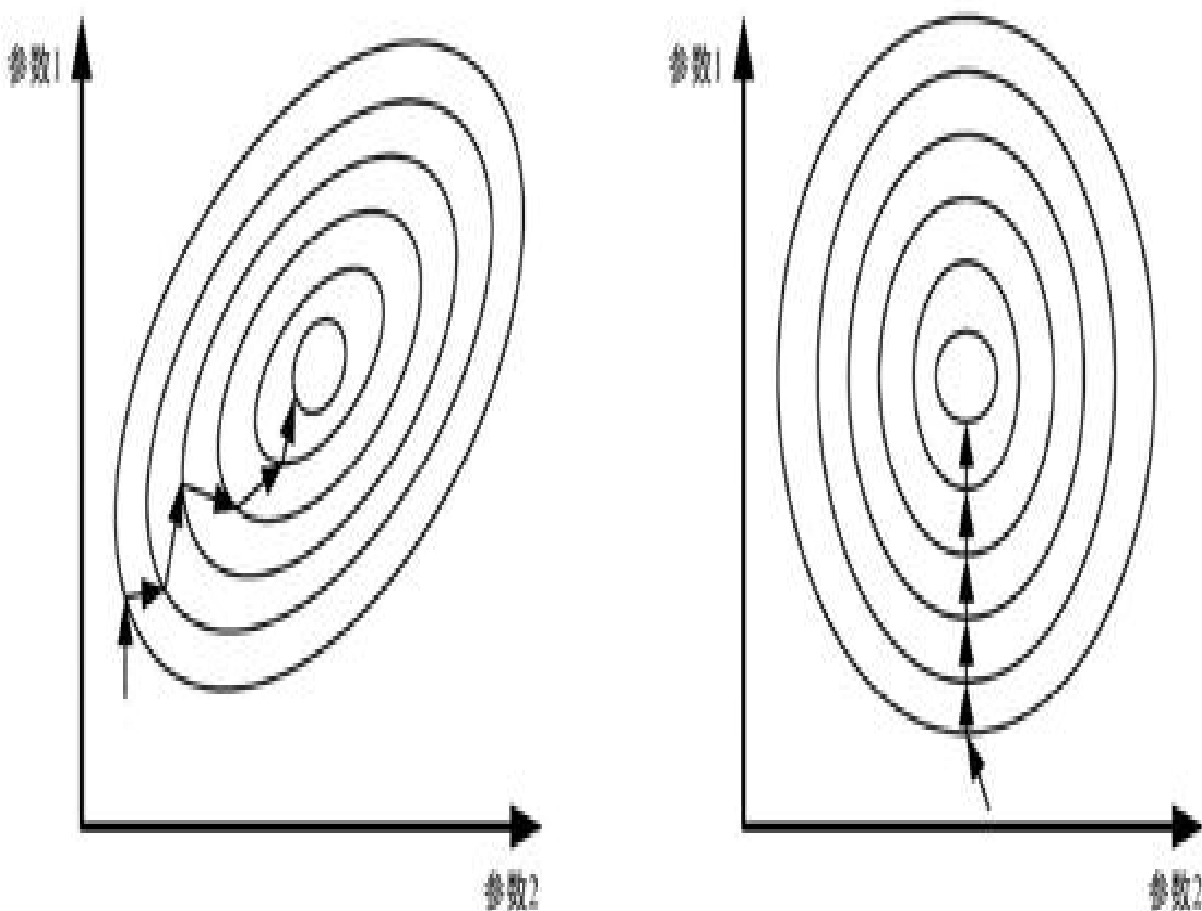


图5-7 不同量纲下梯度下降的对比

3. 特征嵌入

特征嵌入（Feature Embedding）是使用神经网络模型将输入的原始特征转换成指定维度特征向量的一种方法。严格意义而言，特征嵌入也是特征变换的一种，但与基于特征数据层面的简单转换相比，特征嵌入需要通过一个神经网络模型的训练过程来实现。此外，特征嵌入更加变化多端，基于不同的神经网络模型可以得到不同的特征嵌入向量。对于在距离度量下原本相近的样本，特征嵌入能使它们在新的特

征空间中保持原本相近的距离。例如，自然语言处理中的**词嵌入**（Word Embedding）就是将单字或单词映射成较高维度的特征向量，而原本具有语义相似性的单词（如“漂亮”和“美丽”、“英俊”和“帅气”、“中国人”和“炎黄子孙”等），在映射之后的嵌入向量之间仍然保持比较接近的距离。

4. 特征交叉

特征交叉（Feature Overlap）是一种融合多个特征的方法。可以使用独热编码对同时出现的离散型特征进行编码，也可以通过某个公式将多个连续型特征转换为新的抽象特征。例如，对于性别和兴趣这两个离散型特征，其中性别有男、女共两种，兴趣有篮球、游泳、摄影、旅游共四种，使用独热编码对二者同时出现的特征进行编码，结果如表5-3所示。

表5-3 特征交叉编码示例

性别+兴趣	独热编码
男+篮球	[0, 0, 0, 0, 0, 0, 0, 1]
男+游泳	[0, 0, 0, 0, 0, 0, 1, 0]
男+摄影	[0, 0, 0, 0, 0, 1, 0, 0]
男+旅游	[0, 0, 0, 0, 1, 0, 0, 0]
女+篮球	[0, 0, 0, 1, 0, 0, 0, 0]
女+游泳	[0, 0, 1, 0, 0, 0, 0, 0]
女+摄影	[0, 1, 0, 0, 0, 0, 0, 0]
女+旅游	[1, 0, 0, 0, 0, 0, 0, 0]

5. 特征选择

特征选择（Feature Selection）是在数据预处理完成后，筛选真正有价值的特征的过程。**过滤法**是一种常见的特征选择方法。过滤法分为两种，一种是**方差筛选法**，通过设置方差阈值，过滤掉方差较小的特征；另一种是利用目标相关性判断，即计算特征与标签之间的相关性，主要有以下四种特征相关性的计算方法。

- 皮尔逊相关系数法：输出范围为(-1, 1)，0表示无相关性，负值表示负相关，正值表示正相关。局限是所计算的特征和标签都要求是数值型的，并且只能检测出线性关系。

- 卡方检验法：本质是推测两组数据之间的差异，使用0.01或0.05作为显著性水平阈值，如果返回的p值小于显著性水平阈值，则相关；

如果返回的p值大于显著性水平阈值，则不相关。卡方检验法只能检验离散型特征。

- F检验法：是用于捕捉每个特征与标签之间线性关系的计算方法，既可以检验离散型特征，又可以检验连续型特征。返回的p值用法与卡方检验法中返回的p值用法一致。

- 互信息法：是用于捕捉每个特征与标签之间任意关系（包括线性关系和非线性关系）的计算方法。互信息法可以返回每个特征与目标之间互信息量的估算值，这个估算值的取值范围为 $[0,1]$ ，如果估算值为0，则表示两个变量独立；如果估算值为1，则表示两个变量完全相关。互信息法比F检验法更强大，F检验法只能检测出线性关系，而互信息法可以检测出任意关系。

除了过滤法，还可以使用模型法得到各个特征的重要性权重，常用的算法模型有**GBDT模型**和**XGBoost模型**。

6. 样本不均衡

样本不均衡（Sample Unbalanced）是指在带类别标签的分类样本中，不同类别中的样本数量存在差别。例如，二分类问题中正、负例样本比例过大或过小，多分类问题中存在的长尾类别，等等。样本不均衡的问题十分常见，一方面可能在样本数据的收集过程中就出现了问题，如少采集了某些类别的样本；另一方面，由于业务的天然特点，本身就采集不到比较均衡的样本数据，如欺诈检测、癌症检测等场景。样本不均衡问题有可能将模型的优化方向带偏，直接表现为使模型倾向于关注样本数量更多的类别，使效果适得其反。

并不是所有的样本不均衡问题都必须处理。

第一种情况：类别本身的样本绝对数量足够大，只是相对于其他类别较少。由于较少类别的样本量也比较充足，模型对该类别也能完成足够充分的学习，因此不需要将其视为样本不均衡的问题来处理。例如，在电商场景中，虽然商品的点击量远小于其曝光量，订单转化

量也远小于其点击量，但是商品的点击量和订单转化量的绝对值足够大，所以不需要处理样本不均衡问题。

第二种情况：类别样本的比例差别不大，如二分类正、负例样本比例不小于1:4。在保证样本量充足的情况下，在进行模型评估时，可以使用适合类别不均衡的评价指标（如AUC、F1值等）来评估。

具体来说，处理样本不均衡问题的常见方法有以下4种。

- 收集更多的样本。这种方法比较直观，通过整体扩充样本，使较小数量的类别也能拥有足够的样本数量。

- 数据增强。根据现有样本，通过数据变换扩充同类别的样本，这种方法在图像识别领域应用广泛，如将图片进行平移、裁剪、旋转、镜像等。

- 策略采样方法。对数量较少的样本进行过采样（又称为重采样）操作，即将其随机复制多份；对较多数量的样本进行欠采样操作。

- 基于模型的方法，如基于K-近邻算法的SMOTE（Synthetic Minority Oversampling Technique，合成少数类过采样）**算法**。

5.3 建模与调参

在完成数据分析和特征工程后，下面对算法进行建模与调参。如果将算法工程比作下厨，那么数据分析和特征工程相当于准备原材料（柴、米、油、盐、菜等），建模相当于盛原材料的容器（锅、碗、瓢、盆等），调参相当于对厨艺的锻炼。

5.3.1 建模

函数是数学中的一个基础概念，其核心是输入数据、输出计算值。建模是指构建一个以样本特征为输入，以针对该样本的特定预测结果为输出的函数。例如，分类模型输出待预测样本的预测类别，而回归模型输出待预测样本的预测值。

本书第4章已经讲解了多种常用的算法，可以直接用于对函数进行建模。需要强调的是，建模不一定要套用现成的算法模型，现成的算法模型也不一定能取得比较好的效果。可以根据充分的数据分析结果，因地制宜地进行建模。

此外，建模是一个不断迭代更新的过程。没有一成不变的数据，更没有一成不变的模型。一个合理的建模过程是由简到繁、增量迭代的。这种建模过程在实际业务中非常高效，一方面，可以快速取得前期的业务效果；另一方面，可以基于现有模型的效果，有针对性地调参或优化模型结构。

5.3.2 调参

调参的技术含量并不高，但在算法工程中却不可或缺。通俗地讲，调参就是人工调整机器学习算法模型的各项超参数，从而达到提升模型效果的目的。不同机器学习算法模型的超参数不同。例如，随机森林的超参数有学习率、子模型数、采样率、最大深度、分裂最大特征数、叶节点最小样本数等；神经网络的超参数有学习率、网络层数、神经元个数等。

在超参数较少的情况下，可以采用**网格搜索**（Grid Search）方法寻找最优超参数。所谓网格搜索，就是针对算法模型可调节的各项超参数，枚举超参数所有可能的取值组合，逐一尝试，然后选择模型效果最好的一组作为最终模型的超参数。当然，我们还可以使用自动调参工具（如Amazon开发的SageMaker Autopilot、Google开发的AutoML、AutoML的开源替代产品Auto-Keras等）帮助模型自动选择最优的超参数组合。

5.4 效果评估

前面的调参过程需要模型反复进行训练，从而得到不同的效果评估结果，用于指导模型超参数的优化方向。而效果评估符合预期的前提有两点，第一点是需要合理划分评估的数据集，第二点是选择合适的评估指标。

5.4.1 数据集划分

数据集划分的通常做法如下：首先将数据集随机打乱（shuffle），目的是使数据在切分时保持各部分同质。然后将数据集按照一定的比例分别划分为训练集、验证集和测试集。其中，训练集占据大部分数据，主要用于训练模型；验证集主要用于验证训练过程中每一轮结束后的模型效果，在训练过程中会被反复使用，可以设置不同的验证频率；测试集最后使用，主要用于测试模型的最终泛化能力，只使用一次，如果特征涉及时间维度，则建议单独划分最新一段时间的样本作为测试集，从而更准确地测试模型的泛化能力。训练集、验证集和测试集之间没有标准的划分比例，一般将划分比例设置为4:1:1即可。在一般情况下，验证集不宜过大，因为在每轮迭代结束后都会验证一次模型效果，如果验证集过大，则会延长训练所需的总体时间。同样地，验证集和训练集也不宜过小，尤其在正例样本占比较小时，样本量过小会导致验证和测试的结果波动过大而不可信。

除了上述常规的数据集划分方法，还有一种比较流行的方法——**K折交叉验证**。这种方法的基本思路如下：首先将随机打乱顺序后的数据集划分成大小两份，一份作为训练集，一份作为测试集；然后将训练集等分成K份数据集，每次训练只使用其中的K-1份数据集，将剩下的1份数据集作为验证集，用于进行效果评估。K折交叉验证在K-近邻算法中效果显著。图5-8所示为5折交叉验证的数据集划分形式。

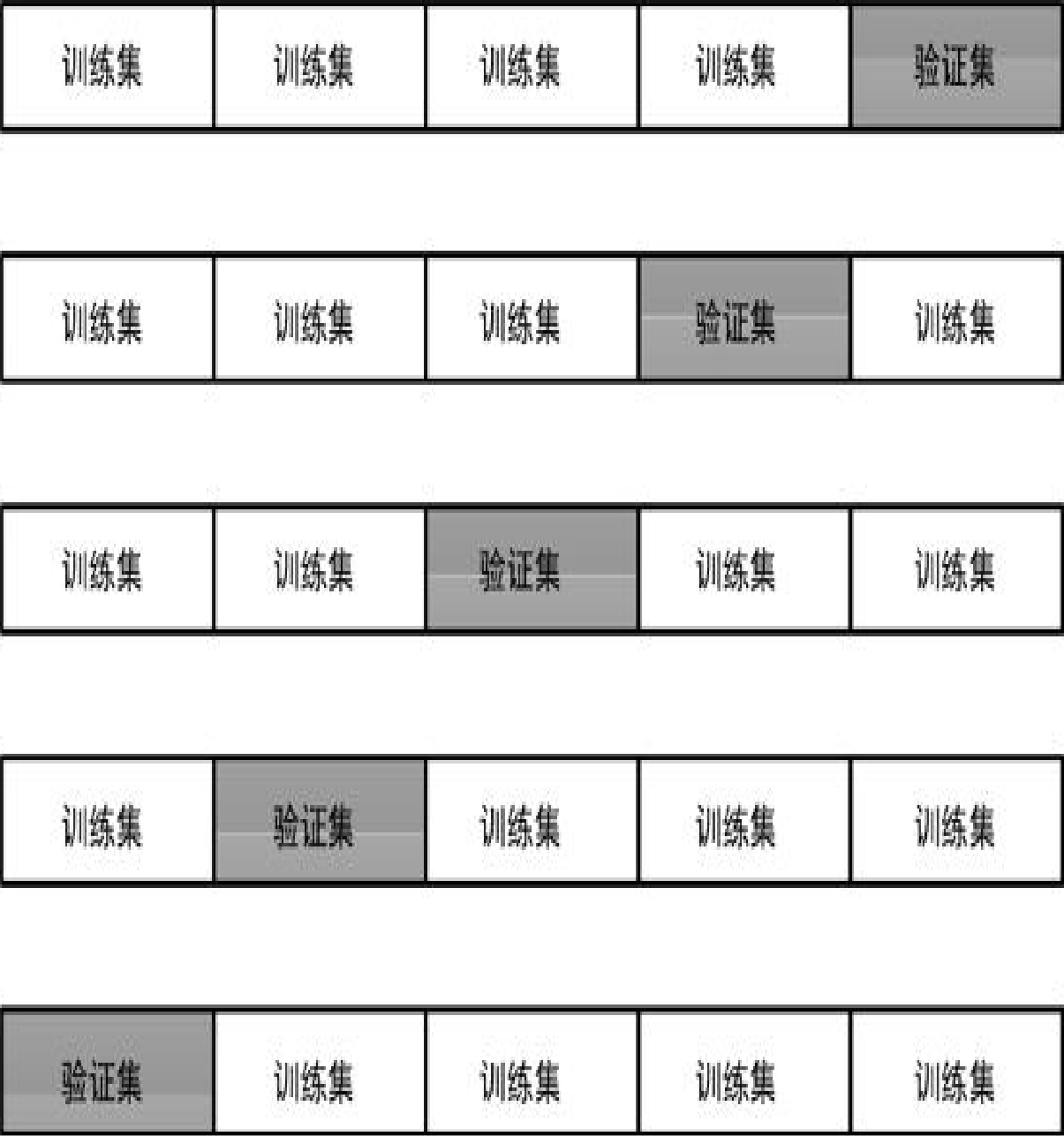


图5-8 5折交叉验证的数据集划分形式

5.4.2 评估指标

评估指标可以直观反映模型分类或回归效果的优劣。常用的分类模型评估指标有精确率、准确率、召回率、F1值、AUC和K-S。常用的

回归模型评估指标有MSE、RMSE、MAE和MAPE。

在介绍分类模型评估指标前，先介绍四个至关重要的概念。

- 真阳性（ True Positive ， TP ）：判定为正例样本，并且是真正例样本。
- 伪阳性（ False Positive ， FP ）：判定为正例样本，但不是真正例样本。
- 真阴性（ True Negative ， TN ）：判定为负例样本，并且是真负例样本。
- 伪阴性（ False Negative ， FN ）：判定为负例样本，但不是真负例样本。

四个概念的表格示例如表5-4所示。

表5-4 四个概念的表格示例

		真 实 值		总 数
		p	n	
预 测 值	p'	真阳性 TP	伪阳性 FP	P'
	n'	伪阴性 FN	真阴性 TN	N'
总 数		P	N	—

分类模型的评估指标就是围绕上述四个概念产生的。

- 精确率（Precision）：主要用于预测正例样本中有多少是真正的正例样本，是针对预测结果而言的，因此精确率又称为查准率，计算公式如下。

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- 准确率（Accuracy）：主要用于预测样本中预测正确的比例，计算公式如下。

$$\text{Accuracy} = \frac{\text{TP}}{\text{TP} + \text{TN}}$$

- 召回率（Recall）：主要用于预测样本中的正例样本有多少预测正确，计算公式如下。

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- F1值（F1 Score）：在准确率和召回率都要求较高的情况下使用，计算公式如下。

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

在用上述四个指标评估模型之前，需要先根据模型的预测值将预测样本划分为正例样本和负例样本。由于二值化过程模糊了模型的原始打分能力，因此这些评估指标无法衡量模型输出的排序性能，即样本在打分后的顺序。一个比较好的二分类效果，应该是两种类别的样本打分值尽可能地两边分开的。

AUC (Area Under the Curve) 计算的是ROC下方的面积，是判断二分类预测模型优劣的标准。AUC可以直接评估模型打分的排序效果，并且可以很好地量化描述分类模型整体性能的高低。因此，AUC在分类模型中应用广泛，但也是比较难理解的一种评估指标。

K-S (Kolmogorov-Smirnov) 可以通过衡量TP样本和FP样本累计分布之间的差值评估模型的风险区分能力。计算方法和AUC类似，但是AUC只评估模型的整体打分效果，并没有指出如何划分阈值，从而使类别预估的效果达到最好，而K-S可以指出分类的最佳划分阈值。

与分类模型的评估指标相比，回归模型的评估指标比较直观。由于回归模型本身输出的就是数值型的预测值，因此要衡量这个值的准确度，其实就是看预测值与真实值之间的距离。

• MSE (Mean Square Error , 均方误差) : 计算每个样本的预测值与真实值之差的平方，然后将其相加，最后取平均值，计算公式如下。

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

• RMSE (Root Mean Square Error , 均方根误差) : 在均方误差的基础上开方。对于预测值域较大的场景 , RMSE的评估值比MSE的评估值更稳定 , 计算公式如下。

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

• MAE (Mean Absolute Error , 平均绝对误差) : 计算所有预测样本的预测值与真实值之差的绝对值 , 然后将其相加 , 最后取平均值 , 计算公式如下。

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

• MAPE (Mean Absolute Percentage Error , 平均绝对误差百分比) : 计算每个样本的预测值与真实值的偏移大小百分比 , 然后将其相加 , 最后取平均值 , 计算公式如下。

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

5.4.3 直观理解AUC

AUC是衡量二分类模型优劣的一种评估指标，**表示正例样本排在负例样本前面的综合概率**。在分类模型中，预测结果一般以概率的形式呈现，如果要计算准确率，那么通常的做法是手动设置一个阈值，将对应的概率转换成类别，而这个阈值在很大程度上会影响模型准确率的计算。不妨举一个极端例子：在一个二分类问题中，一共有10个样本，其中9个样本为正例样本，1个样本为负例样本，在全部判正的情况下，准确率高达90%，显然这并不是希望得到的结果。尤其在这个负例样本得分最高的情况下，模型的性能实际上是极差的。AUC可以很好地描述模型整体性能的高低，如果用AUC评估负例样本得分最高的情况，模型的AUC得分会等于0（当然，通过取反可以解决小于50%情况的问题，不过这是另一回事了）。

为了方便解释ROC及ROC下方的面积，基于上一节讲解的TP、FP、TN、FN，下面介绍如下四个比率。

- 真阳性率（True Positive Rate，TPR）：在所有实际为正例样本的样本中，被正确地判定为正例样本的比率。

- 伪阳性率（False Positive Rate，FPR）：在所有实际为负例样本的样本中，被错误地判定为正例样本的比率。

- 真阴性率（True Negative Rate，TNR）：在所有实际为负例样本的样本中，被正确地判定为负例样本的比率。

- 伪阴性率（False Negative Rate，FNR）：在所有实际为正例样本的样本中，被错误地判定为负例样本的比率。

ROC是根据TPR和FPR绘制而成的，其中横轴是FPR，纵轴是TPR。我们知道，ROC越靠近右下角，曲线下方的面积越小，表示模型的预测能力越低；ROC越靠近左上角，曲线下方的面积越大，表示模型的预测能力越高。那么，如何直观理解这条ROC及ROC下方的面积（AUC的数学意义）呢？

我们可以先想想这条曲线是怎么画出来的。我们知道，线是由点组成的，结合横轴的FPR和纵轴的TPR，ROC上的任意一点表示一个FPR和TPR的数据对。起始点一定是坐标系的原点(0, 0)，即FPR和TPR都为0；而终点一定是坐标为(1, 1)的点，即FPR和TPR都为1。显然，(0, 0)表示所有测试数据都不判定，而(1, 1)表示所有测试数据都判定成正例样本。自然而然，两点之间的数据，可以通过不同的阈值计算得到。

我们不妨将问题再简化一下，不根据阈值划分正、负例样本，直接从样本级别进行划分。假设有一个训练好的二分类模型和一个由10个正、负例样本（正例样本5个，负例样本5个）构成的测试集，将测试集中的10个样本的模型打分按从高到低的顺序排序，得到的真实标签预测顺序为[1, 1, 1, 1, 1, 0, 0, 0, 0, 0]，即5个正例样本恰好都排在5个负例样本前面。然后按照纵轴最小步长为1/正例样本数、横轴的最小步长为1/负例样本数，绘制ROC。由于有10个样本，因此除了原点，还需要绘制10个ROC数据点，从而得到ROC，如图5-9所示。

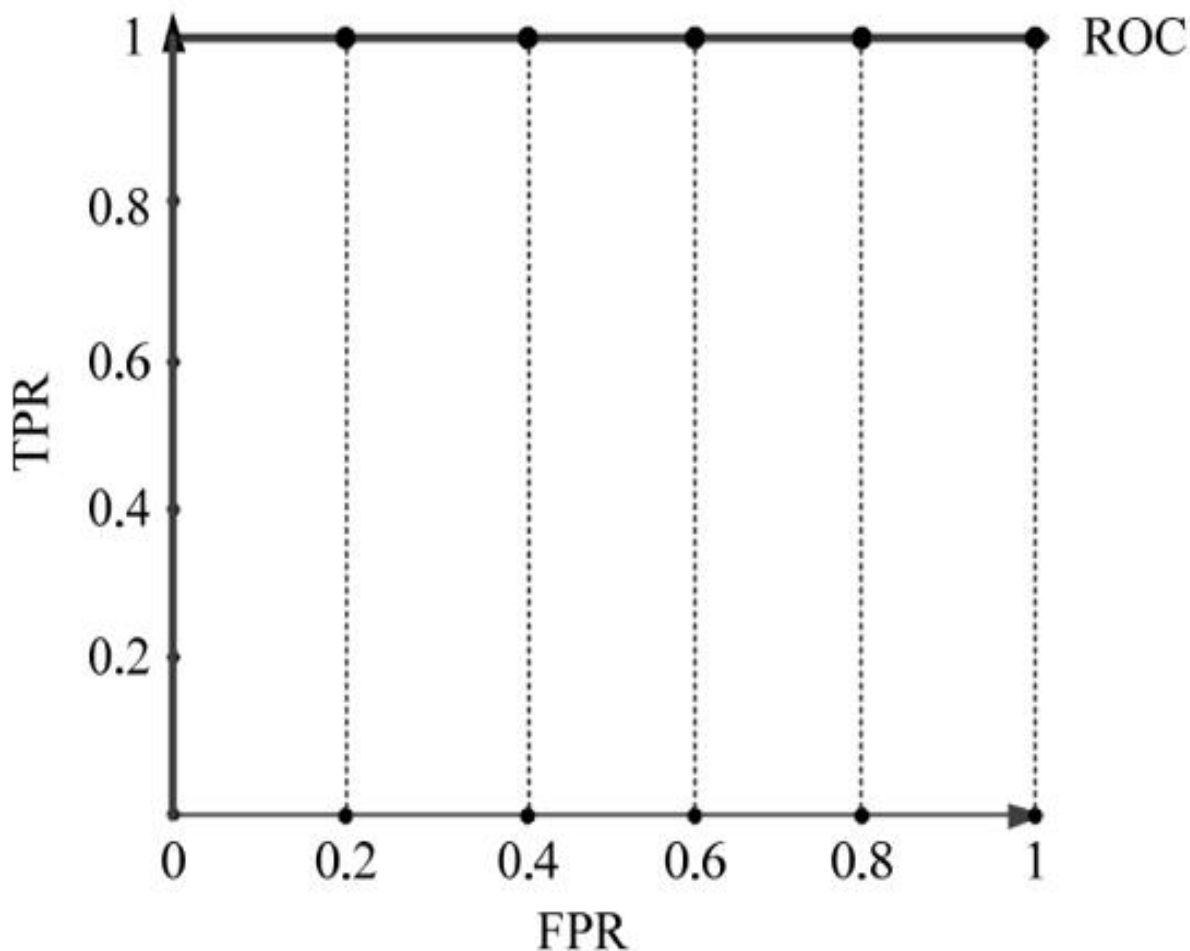


图5-9 ROC示例一

在图5-9中，根据样本预测结果的得分高低，从左至右遍历10个点。从原点开始，每次遇到1，都会向纵轴正方向移动纵轴最小步长，这里是 $1/5=0.2$ ；每次遇到0，都会向横轴正方向移动横轴最小步长，这里也是 $1/5=0.2$ 。不难看出，图5-9中的AUC等于1，表示正例样本排在负例样本前面的综合概率为100%。

我们不妨将真实标签的预测顺序修改为 $[1, 1, 1, 1, 0, 1, 0, 0, 0, 0]$ ，新的ROC如图5-10所示。

在图5-10中，ROC下方的面积为 $(1-0.2) \times 1 + 0.2 \times 0.8 = 0.96$ ，表示该排序结果的AUC的值为0.96。左上角阴影部分的面积为 $0.2 \times 0.2 = 0.04$ ，表示负例样本排在正例样本前面的综合概率为0.04。

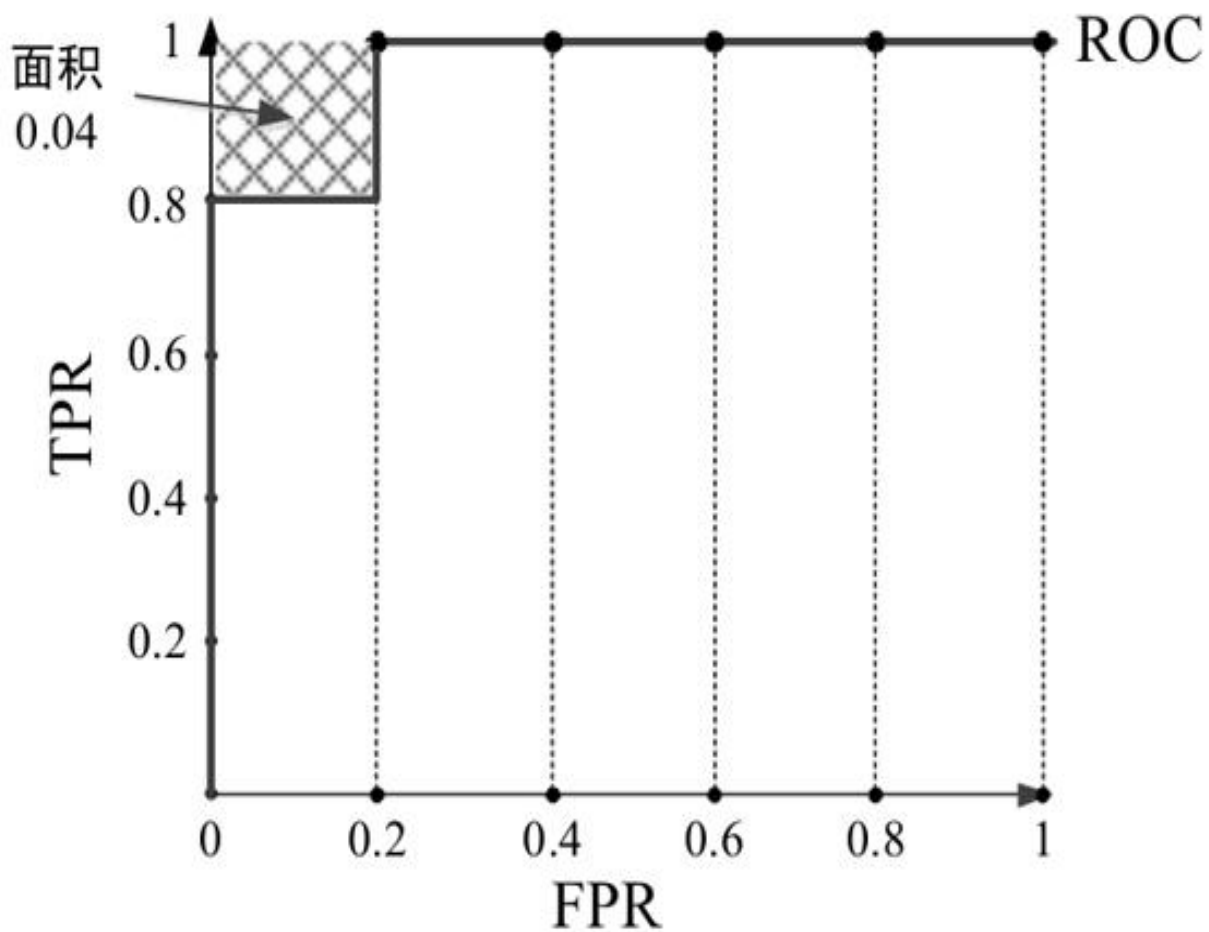


图5-10 ROC示例二

为了加深印象，我们再举一个例子，假设将真实标签的预测顺序修改为[1, 1, 1, 0, 1, 0, 1, 0, 0, 0]，新的ROC如图5-11所示。

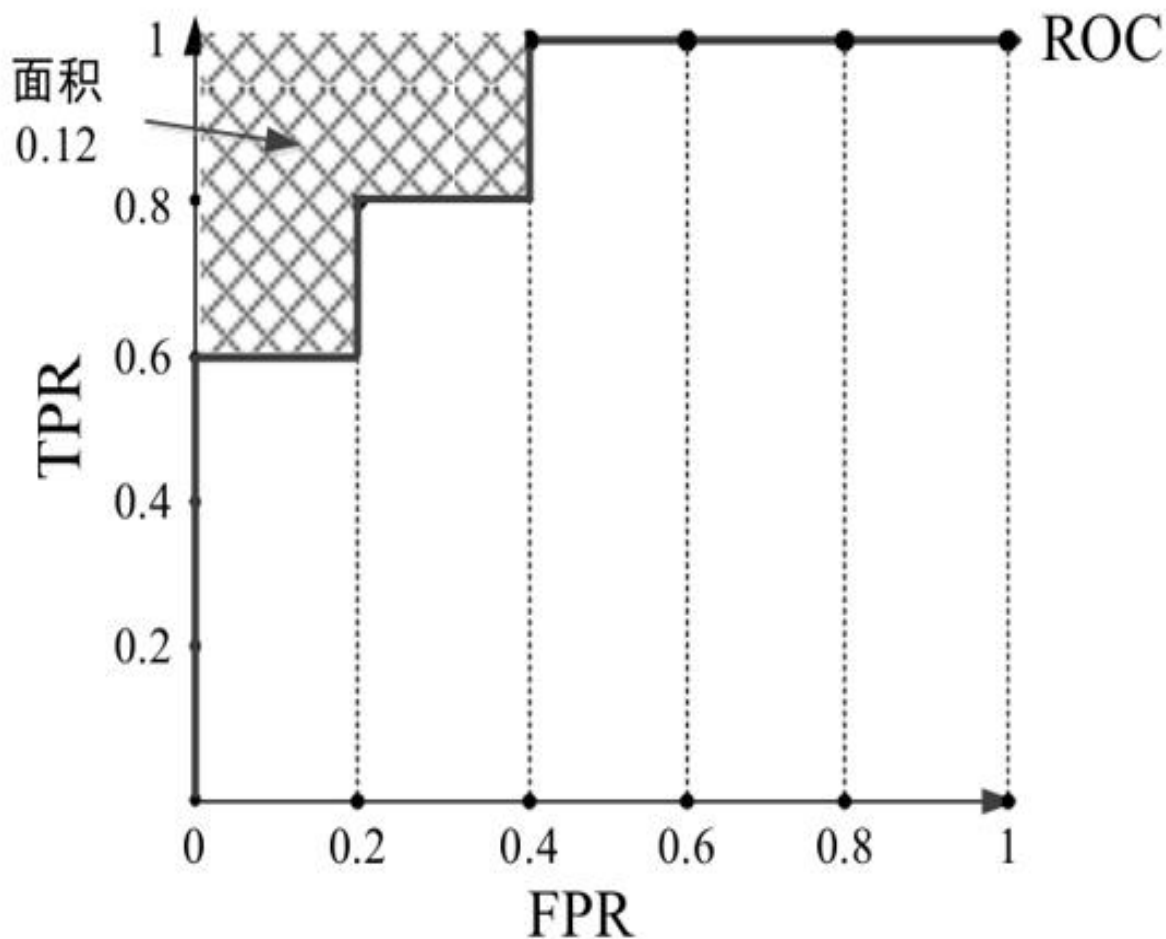


图5-11 ROC示例三

在图5-11中，ROC下方的面积为 $(1-0.4) \times 1 + (0.4-0.2) \times 0.8 + 0.2 \times 0.6 = 0.88$ ，表示该排序结果的AUC值为0.88。左上角阴影部分的面积为 $0.2 \times 0.2 \times 3 = 0.12$ ，表示负例样本排在正例样本前面的综合概率为0.12。

根据判定的预测顺序，随着阈值从小到大的变化，会有越来越多的测试样本被判定为正例样本。ROC从原点(0, 0)出发，每多判定一个真正例样本，ROC就沿纵轴正方向走一步；每多判定一个假正例样本，ROC就沿横轴正方向走一步。直到阈值将所有样本判定为正例样本，即走到终点(1, 1)。这样走下去的总步数，正好等于评估的样本总数。要想排序效果好，需要使ROC尽可能早且多地沿纵轴正方向走，

即尽可能早且多地判定真正例样本。由此可知，ROC下方的面积越大，模型的分类效果越好。

5.5 模型托管

在模型评估满足要求后，最后一步是将模型投入使用。机器学习算法的模型在生产、生活中的使用形式主要分为两种，一种是**离线**（offline）**批量预测**（预测又称为推理），另一种是**在线**（online）**实时预测**。对于离线批量预测，只需例行调度任务程序，每次载入保存好的模型，然后使用加载好的模型对待预测样本进行整体预测并记录结果。对于在线实时预测，如果每来一条实时数据，都加载一次模型文件，那么其性能开销会非常大。为了解决这个问题，**模型托管**应运而生。

顾名思义，模型托管就是模型的委托管理。对玩过网络游戏的读者来说，“挂机”一词一定不陌生，“挂机”这种自动在线完成某些游戏任务的行为，就是一种托管。类似地，模型托管主要用于在一台电脑主机上将保存好的模型加载到后台运行，提供访问接口，并且作为一个托管的Web服务以供调用。调用方可以通过HTTP之类的请求，将待预测的样本数据发送给模型托管服务端，从而得到实时的预测结果。

当然，并不是所有的算法工程都需要进行模型托管，只有满足实时预测需求的场景才需要进行模型托管，如电商推荐场景、广告推荐场景、在线交易场景等。对于广大算法兴趣爱好者的个人需求，在一般情况下，没必要通过模型托管来测试模型的预测情况。

5.6 动脑时刻

1. 在进行模型训练前，统计分析特征值的分布有哪些好处？

2. 现有用户编号、商品编号、用户访问日期、用户是否购买商品四个字段，尽可能多地列出可以统计的有价值信息。
3. 举例说明表格、图形和文本分别适合呈现什么样的数据？（注：可以结合使用）
4. 某分类模型针对不同类别的预测结果明细如表5-5所示，尝试根据表中数据分别计算精确率、准确率、召回率和F1值。

表5-5 某分类模型针对不同类别的预测结果明细

		样本真实值		总 数
		正 例	负 例	
预测输出	预测为正	30	20	50
	预测为负	10	40	50
总数		40	60	—

5.7 本章小结

本章给大家介绍了算法工程的组成部分，包括数据分析、特征工程、建模与调参、效果评估和模型托管，每个环节都至关重要。在掌握了算法工程的几个关键步骤后，相信在解决实际的算法问题时，能帮助大家少走一些弯路。

关键词回顾

算法工程

数据分析

batch size

epoch

宏观把握数据

微观感受数据

计数和展示

特征工程

缺失值处理

特征变换

离散化

分箱

等频分箱

等宽分箱

特征编码

独热编码

特征缩放

对数变换

归一化

标准化

特征嵌入

词嵌入

特征交叉

特征选择

过滤法

方差筛选法

皮尔逊相关系数法

卡方检验法

F检验法

互信息法

样本不均衡

网格搜索

K折交叉验证

精确率

准确率

召回率

F1值

AUC

K-S

MSE

RMSE

ROC

MAE

MAPE

模型托管

离线批量预测

在线实时预测

第6章

算法工程实战

在前面的章节中，我们从基础的数学理论到复杂的机器学习算法，从数据结构与基础算法到算法工程的各个组成部分，循序渐进，逐步给大家揭开AI算法世界的神秘面纱。看到这里，相信大家已经迫不及待要动手试一试机器学习算法的魔力了。本章会手把手教大家完成自己的第一个算法工程，包括设备配置、环境搭建、开源算法库、数据准备、数据分析、特征工程、模型训练、模型的保存与载入等。

一方面，与软件开发相比，算法工程对系统开发能力的要求相对较低，因此入门算法的读者不必担心自己不具备足够的编程语言基础和程序设计基础。另一方面，对于常见的机器学习算法模型，市面上已经有不少成熟的开源代码库，因此无须从零开始敲代码来实现算法。

6.1 环境准备

“巧妇难为无米之炊。”在构建算法工程之前，我们需要先完成若干项前期准备工作。例如，运行算法工程需要什么样的设备配置，选用哪种开发语言，需要搭建怎样的程序运行环境。对于这些问题，下面给大家一一解答。

6.1.1 设备配置

得益于电子产业的迅猛发展，如今一台普通的、能够正常工作的**个人计算机**（Personal Computer，PC）就可以满足运行算法工程的配置需求。不过，如果样本的数据量很大，如TB级甚至是PB级的数据量，则需要具备大规模**并行计算**（Parallel Computing）的能力才能支撑算法工程的运行，而普通PC并不具备大规模并行计算的能力。

并行计算是指同时使用计算资源的过程，是提升设备计算速度和处理能力的一种有效手段，具体可以分为时间并行和空间并行两种。时间并行类似于工厂流水线技术，在相同时间启动多个计算任务，从而有效提高计算性能。空间并行是指多台计算机通过网络连接起来，可以同时进行计算，从而解决单台计算机无法解决的大型问题。大规模并行计算需要分布式集群资源，一种解决方法是购置多台主机，然后手动搭建自己的小型集群；另一种解决方法是使用各大云服务厂商提供的云计算服务，如**亚马逊云服务**（Amazon Web Services，AWS）和**阿里云**（Alibaba Cloud）。无论采用哪种解决方法，对AI算法的初学者来说，操作要求都比较高。

回归本章主题，个人计算机所配备的**中央处理器**（Central Processing Unit，CPU）的计算能力，足够支持小型算法工程的运作。当面对使用深度神经网络的算法工程时，可以使用**图形处理器**（Graphics Processing Unit，GPU）、**张量处理器**（Tensor Processing Unit，TPU）和**神经网络处理器**（Neural Network Processing Unit，NPU）来加速模型计算。其中，TPU是谷歌公司专门为加速深层神经网络运算能力研制而成的一款芯片，NPU的典型代表是国内寒武纪公司专门针对神经元的计算与传递研发而成的芯片。不过，这些芯片价格不菲，感兴趣的读者可以自行斟酌选购或租赁使用。

6.1.2 环境搭建

要运行一个算法工程，除了硬件方面的需求，还有软件方面的需求。一台可以联网的个人计算机帮我们解决了硬件需求，还需要解决软件需求。软件方面的需求包括**操作系统、高级程序设计语言和开发环境**共3方面。

在操作系统方面，目前市面上主流的操作系统有微软公司的Windows操作系统、苹果公司的macOS操作系统及Linux操作系统，本章基于macOS操作系统进行算法工程的实战讲解。在相同的环境下运行算法工程，对不同操作系统之间的区别基本是无感知的，所以不必担心操作系统的选择。在之后的各个步骤中，只在必要时强调二者在操作上的差异，除此之外，默认二者的操作相同即可。

下面确定使用哪种高级程序设计语言。程序设计语言是指用于编写计算机程序的语言，高级程序设计语言更加抽象，是更加易于人理解和操作的程序设计语言。高级程序设计语言在不同平台上被编译成相应的机器语言，才能直接被机器执行，从而使高级程序设计语言不用依赖某种特定的机器和环境。目前，开发机器学习算法模型的几种主流高级程序设计语言有Java、C/C++和Python。与Java和C/C++相比，Python的学习门槛较低，而且语言风格简明、清晰，易于理解，适合算法初学者学习。因此，本书使用Python作为示例代码的高级程序设计语言。Python官方网站的首页如图6-1所示。

图6-1 Python官方网站的首页

在开发环境方面，下面介绍Python开发环境的安装步骤。Python有Python 2和Python 3两个主流版本。由于Python 3是Python 2的升级版，而且Python官方建议初学者直接学习Python 3，Python 2也只维护到了2020年，因此本书使用Python 3作为编程语言。Python 3可以安装在大部分已知的操作系统平台上，包括当下主流的3种操作系统Windows、macOS和Linux。

Python 3的官方网站下载页面如图6-2所示。根据图6-2可知，Python官方网站提供了3种主流操作系统所对应的安装包下载链接。值

得注意的是，无论是Python官方网站，还是接下来介绍的开源算法库，都是英文网站，因此具备一定的英语阅读能力是很有必要的。这里提供一个方法，可以利用一些翻译小工具帮助自己理解一些英文的含义，如有道词典和谷歌翻译。

图6-2 Python 3的官方网站下载页面

6.1.3 开发工具

“工欲善其事，必先利其器。”一款好的IDE（Integrated Development Environment，集成开发环境）能够使开发工作事半功倍。本书推荐使用PyCharm。PyCharm是一种Python IDE，有一整套可以帮助用户在使用Python语言开发时提高其效率的功能，如调试、语法高亮、Project管理、代码跳转、智能提示、自动完成、单元测试和版本控制等。

PyCharm官方网站的首页如图6-3所示，PyCharm官方网站下载页面如图6-4所示。PyCharm官方网站提供了3种主流操作系统所对应的安装包下载链接。如果偏好中文界面，则可以下载中文插件包，具体操作方法可自行查阅相关资料。

图6-3 PyCharm官方网站的首页

图6-4 PyCharm官方网站下载页面

6.1.4 基础调试

在安装好Python语言环境和PyCharm开发工具之后，我们先做一下简单的代码测试，用于保证Python程序能够顺利运行。

第一步，打开PyCharm程序，单击“Create New Project”按钮，打开“New Project”窗口，用于新建一个项目，在左侧选择“Pure Python”选项，如图6-5所示。将项目解释器（Project Interpreter）设置为已安装好的Python版本；如果解释器（Interpreter）下拉列表中没有对应的选项，则手动输入Python安装目录；项目可以被任意命名，这里我们将其命名为“hello”。

图6-5 PyCharm新建项目

注意：项目解释器设置的Python版本仅用于PyCharm中Python代码的编译和运行。例如，如果设置为Python 3版本，PyCharm就会按照Python 3的标准执行代码、校验、设置语法高亮等。如果不使用PyCharm运行代码，则需要确认所用Python的版本是否和代码匹配，因为Python 3和Python 2之间的代码存在差异。

第二步，右击左上角新建的项目名称，在弹出的快捷菜单中选择“New”→“Python file”命令，打开“New Python file”窗口，输入要新建的文件名称（这里输入“helloAI”），如图6-6所示。

图6-6 “New Python file”窗口

第三步，在新建的helloAI.py文件中输入如下代码，然后单击“保存”按钮，表示在控制台中输出“hello AI”。

第四步，右击空白处，在弹出的快捷菜单中选择“Run 'helloAI'”命令，运行代码，运行结果如图6-7所示，表示这行Python代码被成功执行。

图6-7 Python代码测试的运行结果

6.2 开源算法库

在算法工程的前期准备工作完成后，就可以开始实现第4章中的各种机器学习算法了。如果对自己的编码能力有信心，并且对机器学习算法的内部运作原理感兴趣，则可以尝试从零开始编码，自己上手实现神经网络、决策树和聚类机器学习算法，对这些算法的理解会比直接调用开源算法要深刻得多。当然，从零开始编码，从而实现机器学习算法并非本章的初衷。

关于机器学习算法的实现，本章会使用scikit-learn和TensorFlow两种比较流行的开源算法库，带大家进行算法工程实战。其中，scikit-learn适合初学者在单机环境下学习机器学习算法，如果需要分布式部署，则需要结合其他开源组件实现；TensorFlow是由谷歌人工智能团队谷歌大脑（Google Brain）开发和维护的开源算法库，比scikit-learn更庞大，并且支持分布式部署，在业界的工业级应用十分广泛。这两种开源算法库都有可读性非常强的Python接口可供使用者调用，下面分别介绍这两种开源算法库及安装方法。

6.2.1 scikit-learn

scikit-learn（简称sklearn）是基于Python语言开发的一款简单、高效的数据挖掘和数据分析工具，提供了包括回归、聚类、决策树、神经网络等常见的机器学习算法模型包。针对3种主流操作系统，sklearn官方网站分别提供了具体的安装方法。此外，sklearn有一个相当不错的中文网站，方便汉语母语初学者参考学习。

安装sklearn最简单的方法是使用pip命令。如果没有进行过alias处理，Python 3对应的命令应为pip3。sklearn的安装方法如图6-8所示，

选择合适的操作系统，使用pip命令安装sklearn。

图6-8 sklearn的安装方法

在安装成功后，在PyCharm界面底部单击“Terminal”按钮，执行以下命令。

或

如果安装成功，则会出现带有版本编号的sklearn信息，如图6-9所示。

图6-9 sklearn安装成功标志

6.2.2 TensorFlow

TensorFlow（简称TF）是谷歌公司内部演变出来的、高鲁棒性的、开源的工业级机器学习框架，其前身是DistBelief。DistBelief是由Geoffery Hinton带领团队开发的一个实现了广义反向传播算法的框架，被广泛应用于谷歌公司内部的科研项目及商业产品中。后来，谷歌公司决定将DistBelief升级成更加稳健、快速、通用的机器学习框架，调动了很多工程师和科学家投入框架改造项目，其中就包括闻名遐迩的Jeff Dean。

TF采用计算图的形式构建算法模型。如果将数据比作水，那么计算图就是一个首尾衔接的管道系统，模型运行过程就相当于管道系统通水的过程。计算图的设计方式对分布式训练天然友好，计算图可以

复制成多份，然后将其分发到不同的机器节点上进行训练。TF还支持多平台运行。例如，在PC上完成训练，在iOS或Android系统上进行预测。

TF支持使用pip命令进行安装。为了方便开发者查阅使用，TF官方网站支持多种语言界面，在官方网站界面右上角可以手动进行切换，这对非英文母语的初学者来说十分友好。TF同样支持3种主流的操作系统。在TF中文官方网站首页单击“安装”按钮，进入“安装”页面，如图6-10所示，在该页面按照右下方的命令安装即可。

图6-10 TF中文官方网站的“安装”页面

在安装成功后，在PyCharm界面底部单击“Terminal”按钮，执行以下命令。

如果安装成功，则会出现带有版本编号的TF信息，如图6-11所示。

图6-11 TF安装成功标志

6.3 算法实践

在安装好sklearn和TF后，就可以使用它们实现各种机器学习算法模型了。本节会手把手教大家实现两种常用的机器学习算法模型——线性回归模型和神经网络模型。

6.3.1 线性回归模型

1. sklearn版本

使用sklearn实现线性回归模型十分简单。首先从安装好的sklearn包中引用线性模型，代码如下。“from...import...”是Python中引用代码的惯用写法。

然后调用sklearn线性模型库中的线性回归类，得到一个线性回归模型的实例，代码如下。

这里我们只用了一行代码就完成了线性回归模型的构造，想必大家已经感受到了借助开源算法库的优势了。接下来，使用几条构造的样本数据测试这个线性回归模型。假设测试样本只有一个特征值 x 和一个回归标签值 y ，一共有3条样本数据，分别为 $(x=0, y=1)$ 、 $(x=1, y=2)$ 和 $(x=2, y=3)$ 。假设模型的学习目标是二维空间中的直线拟合，线性回归方程为 $y=wx+b$ 。使用fit方法学习这3条样本数据，然后输出学到的 w 和 b 的值，代码如下。

在模型训练完成后，就可以进行预测了。使用predict方法分别预测 $x=5$ 、 10 、 20 、 100 的样本，输出对应的预测结果。

使用sklearn实现线性回归模型的完整代码如下。

在PyCharm项目中新建一个Python文件，然后输入上述代码，单击“运行”按钮，预测结果如下。

2. TF版本

TF代码不像sklearn代码那样高度抽象，可以直接提供一个完整的线性回归模型。使用TF实现线性回归模型需要自己搭建模型的数据流图，进而完成相应的模型训练和预测。

首先引用TF代码包，还需要引用一个数据工具包NumPy来存储数据，代码如下。

然后使用TF构建线性回归模型的计算图，代码如下。其中x和y_true分别表示特征和标签，w和b分别表示特征权重参数和偏置，y_pred表示模型的输出。由于是图，并不包括实际的数据，因此x和y_true使用placeholder（占位符）声明，表示先占一个数据的位置。其中shape参数表示输入数据的规格，[None, 1]表示可以接受任意多行、每行只包含1列的数据，dtype参数主要用于规定输入数据的类型。

损失函数使用回归模型最常用的均方误差函数，优化函数使用随机梯度下降算法（Stochastic Gradient Descent，SGD）的升级版本AdamOptimizer，然后给优化函数设置一个合适的学习率，代码如下。

在计算图构建完成后，就可以构造数据样本了。为了方便对照，使用前面sklearn使用的样本，但用NumPy构造其特征值和标签值，代码如下。

有了管道系统，也有了水，下一步自然是将水注入管道系统，使其运作起来，代码如下。sess相当于TF的运行容器；run方法表示一次执行操作，包括参数初始化、训练和预测；feed_dict表示将具体的样本数据像水一样注入计算图，以x和y_true为管道入口；设置循环最多迭代学习200步，old_loss和curr_loss一起控制收敛，在达到指定预制值后提前退出训练，从而提高程序的执行效率。

在模型训练完毕后，即可得到训练好的模型参数w和b，将待预测样本数据直接输入线性回归模型公式，即可完成预测，代码如下。

使用TF实现线性回归模型的完整代码如下。

在PyCharm中输入上述代码，单击“运行”按钮，预测结果如下（输出行数过长，只展示尾部）。

可以发现，在保留一定精度后，TF的预测结果和sklearn的预测结果是一致的。

6.3.2 神经网络模型

1. sklearn版本

使用sklearn实现神经网络模型也非常简单，对应的神经网络算法包neural_network提供了基于多层感知机（Multilayer Perceptron，MLP）实现的回归模型和分类模型，分别为MLPRegressor和MLPClassifier。下面使用MLPClassifier完成一次简单的分类任务。假设测试样本只有1个特征值x和1个类别标签值y，一共有4条样本数据，分别为(x=6, y=1)、(x=8, y=1)、(x=12, y=0)和(x=15, y=0)。如果将上述4个数据点进行二分类，那么最合适的分割点为x=10。

使用sklearn实现神经网络模型的完整代码如下。其中hidden_layer_sizes参数表示隐层网络中分别包含多少个神经元（感知机）。

神经网络模型预测输出的结果如下。

2. TF版本

神经网络的隐层代码可以重复使用，下面编写一个隐层函数。其中用到的ReLU函数在4.2.2小节中讲解过。Dropout是指在深度神经网络训练过程中，按照一定概率暂时丢弃一部分神经元，这个操作有助于缓解神经网络的过拟合问题。

使用TF实现神经网络模型的完整代码如下。

神经网络模型预测输出的结果如下。虽然最终模型学到的参数值存在差异，预测输出的实际结果也不一样，但对二分类模型而言，由于以分值临界点划分类别，因此模型取得的效果大同小异。

6.4 工程实战

在熟悉开源算法库的使用方法之后，可以开始本章的重头戏，即在真实数据集上完成一次算法工程实战。数据在生活中无处不在，并且每时每刻都在不断地增加，如社交平台的用户聊天数据、电商平台的用户浏览数据和交易数据。当然，这些数据只有在相关公司从事相关算法工作的员工才有可能接触到，而且不能用于私人用途。算法爱好者虽然无法接触这些公司的数据，但可以使用仅供研究学习的开源数据集。本章基于一份开源的美国人口普查数据集，手把手带大家完成一次第5章介绍的算法工程。

6.4.1 数据准备

这份开源的美国人口普查数据集的官方网址在其附录中可以找到，单击首页的“Data Folder”可以看到下载地址。该数据集有14个特征，共48 842条样本数据。其中adult.data为训练集，共有32 561条样本数据；adult.test为测试集，共16 281条样本数据。该数据集提供了分类算法的目标，即预测一个人的年收入是否超过5万美元。

我们知道，特征类型主要分为离散型和连续型两种。在开始算法工程之前，我们先来了解这14个特征的名称、含义及类型，如表6-1所示，从而方便对后续特征数据的处理和转换。

表6-1 14个特征的名称、含义及类型

年龄、体重、受教育次数、资本收益、资本损失和每周工作小时数这6个特征为连续型特征，表示数值含义。其余8个特征为离散型特征，表示文本含义。代表年收入的类别标签在每条样本的末

尾，“>50k”表示年收入超过5万美元，“<=50k”表示年收入未超过5万美元。

在弄清楚数据字段后，我们将作为训练集的adult.data数据文件按列读取到Python程序中。

注意：在数据文件中，列与列之间用逗号分隔，并且需要删除逗号后面的空格。

我们使用如下代码读取数据，其中，strip()函数主要用于去除每条样本首尾多余的字符（回车符）及每个字段前后多余的空格符，然后将样本逐条存储于一个list变量中。

6.4.2 数据分析

在5.1节，我们总结了数据分析主要包括的两个步骤——**计数和展示**。

首先，我们对特征的**缺失值**进行计数，查看原始数据集中的特征值是否有缺失的情况。在6.4.1节，我们根据逗号切分存储后的特征数据，存储的数据类型均为字符串类型。因此要检验是否有缺失值，只需对空字符串计数。NumPy提供了非常方便的字段值检查方法argwhere，用于检查空字符串，具体实现代码如下。

该方法可以将所有满足值条件的特征值所在行的下标组成数组并返回，取数组长度即可得到缺失特征值的总数量。如果存在缺失特征值的样本，则可以使用平均值对其进行填充或将该条样本删除。当然，如果缺失特征值的样本占比过大（如已经超过了1%），则可能是源数据存在问题。

为了便于数据分析和作图展示，我们引入了另一个Python高效数据处理库Pandas。Pandas是基于NumPy的一种数据分析工具，与NumPy最大的区别是，Pandas可以提供数据索引，可以更加方便地进行数据的查询和筛选，从而执行一些类似于数据库表查询的操作。Pandas读取数据集的方式如下。

其中，字符串数组headers主要用于存储特征列名，read_csv方法主要用于从文本文件中以逗号和空格（'\s'）的组合符为分隔符读取每行数据，字符串数组na_values主要用于将数据字段中的字符串"null"、问号和空格符转换为缺失值（NaN），方便后续进行数据清洗操作。可以根据原始数据的具体情况，在字符串数组na_values中定义需要转换的无效字符。

使用shape方法可以查看读取数据集的形状，使用iloc方法可以按照序号查看具体的样本数据，如下面代码中的“0:3”表示查看第0条、第1条和第2条样本数据（下标从0开始计算）。

在读取完样本数据后，我们先清洗其中的异常数据值。在前面的数据读取配置中，使用na_values参数将自定义的异常数据值全部转化为了缺失值（NaN），因此可以使用Pandas提供的dropna方法删除包含缺失值的行。为了观察删除了多少行，在删除之前和之后分别输出一数据集的形状。注意dropna方法中的inplace参数，如果将该值设置为True，则表示直接在原数据集中删除包含缺失值的行，training_raw变量存储的数据集会发生变化；如果将该值设置为False，则表示将删除包含缺失值的行后的数据存储为副本，并且training_raw变量存储的数据集不会发生变化。inplace参数在很多方法中都可以应用。

在去掉包含缺失值的行后，即可正式开始进行数据分析。下面以单特征分析为例，讲解数据分析中的计数和展示。使用图形处理库matplotlib和seaborn完成作图。首先，查看训练集中的正、负例样本比例，即查看年收入是否超过5万美元的人数比例。计数和作图展示的代码如下。

其中，使用analysis变量存储用于进行数据分析的predclass标签；使用fig变量表示一张长20in、高3in的画布；使用countplot方法基于analysis变量统计不同predclass值的数量，并且在画布上画出条形图；使用savefig方法将当前画布以图像文件的形式存储于磁盘中。正、负例样本比例的统计结果如图6-12所示。

图6-12 正、负例样本比例的统计结果

根据图6-12可知，正、负例样本比例并未发生较明显失衡，暂时无须进行采样处理。

下面以age（年龄）特征为例，讲解如何开展单特征分析。age（年龄）特征是连续型特征，在统计数量时需要先对其进行分箱处理。我们知道，分箱分为等宽分箱和等频分箱，为了查看不同年龄段之间的数据差异，我们采用等宽分箱方法cut（等频分箱方法为qcut），将年龄按取值范围等距分成10份并绘制条形图。然后针对年收入是否超过5万美元人群的年龄分布，分别绘制条形图，并且观察差异。计数和作图展示的代码如下。

其中，使用age_bucket列存储分箱处理后的特征分段标签；使用subplot方法表示该画布由子图生成，3个参数分别表示行数、列数和当前子图编号，每个subplot方法后面紧跟一个作图方法，第一个子图是由countplot方法绘制的分箱后的年龄分布条形图，第二个子图是由

distplot方法分别绘制的正、负例样本标签下的年龄曲线拟合图，如图6-13所示；最后使用savefig方法将当前画布以图像文件的形式存储于磁盘中。

图6-13 分箱后的年龄分布条形图和正、负例样本标签下的年龄曲线拟合图

图6-13 分箱后的年龄分布条形图和正、负例样本标签下的年龄曲线拟合图（续）

根据图6-13可知，样本中30岁左右的人群占比较高，年收入超过5万美元的人群年龄整体高于年收入不超过5万美元的人群年龄。

上述针对年龄特征的简单分析，说明该特征对预测目标具有一定的区分度。其他特征分析以此类推。

6.4.3 特征工程

特征工程与数据分析是密不可分的，6.4.2节中的分箱操作就属于特征工程范畴。5.2.3节讲到的独热编码是特征工程中最常用的方法之一，下面以独热编码为例，将原始特征数据转换成可以直接用于模型训练的特征数据。

关于独热编码的实现，Pandas库提供了一个强大的方法get_dummies，它能自动将数据集中的非连续型特征逐个转换成独热编码。直接调用这个方法并输出新的特征列名，然后抽样观察具体的特征值，具体代码如下。

需要注意的是，类似于age的连续型特征在使用get_dummies方法处理后会原封不动地保留，如果需要将其转换成独热编码，则可以先

使用6.4.2节中使用的分箱技术新增一列该连续型特征的分箱特征，再调用`get_dummies`方法。

下面根据打印输出的特征名，从新生成的独热编码数据中分离出特征数据`x`和标签数据`y`，具体代码如下。

针对不同的数据分析结果，可以采用不同的特征工程处理方法。作为算法工程的重要部分，特征工程的根本目的是让特征更好地表达，从而使模型更容易学到点子上。

6.4.4 模型训练

有了可以直接“喂”给模型的特征数据和标签数据，下一步就是搭建模型和算法调参。使用开源算法库`sklearn`提供的逻辑回归模型搭建模型并完成模型训练，并且输出模型在测试集上的预测准确率和AUC指标，具体代码如下。

运行上述代码，可以发现，模型在测试集上的预测准确率和AUC指标都偏低，其中AUC指标只有0.6左右。不妨对逻辑回归模型进行简单的算法调参。在PyCharm中通过代码追溯功能，跳转到逻辑回归的初始化函数，发现最小误差参数`tol`的默认值为`1e-4`，对本案例中的模型训练来说偏高。因此，修改逻辑回归模型初始化函数的代码，具体如下。

运行修改后的代码，可以发现，模型在测试集上的AUC指标从原来的0.6提升到了0.76，预测准确率也从原来的0.78提升到了0.84，说明这次算法调参的效果是显著的。当然，模型调优还有很多方法，可以

是参数上的调优，也可以是模型结构上的调优，这里旨在抛砖引玉，不再赘述。

6.4.5 模型的保存与载入

在模型训练好后，可以通过Pickle模块将其存储于磁盘中。Pickle模块是Python提供的一个序列化与反序列化的标准模块，通俗地说，就是用于进行Python中数据对象和磁盘文件之间互相转换的模块。将训练好的逻辑回归模型存储于磁盘中，再读取使用，具体代码如下。

在重新加载存储模型后，其预测结果和之前的预测结果一致，证明模型的存储与恢复无误。除了Pickle模块，sklearn还提供了相应的模型持久化工具joblib。joblib适合用于存储和重新加载较大的模型，具体代码如下。

至此，算法工程的完整步骤讲解圆满结束。本章的完整源码参见本书配套的GitHub项目。

6.5 算法竞赛介绍

6.5.1 Kaggle

Kaggle是由联合创始人、首席执行官安东尼·高德布卢姆（Anthony Goldbloom）于2010年在墨尔本创立的，用于为数据科学家提供举办机器学习竞赛、托管数据库、编写和分享代码的平台。

Kaggle主要以Data Mining竞赛为主，与注重模型和算法的Machine Learning不同，Data Mining侧重于从数据和问题本身出发，希望针对问题的特性来设计最适合的方案，对数据的“嗅觉”要求更高。当然，Kaggle金牌的含金量是很高的。

6.5.2 KDD Cup

除了Kaggle，另一个含金量较高的竞赛莫过于有“数据挖掘世界杯”之名的KDD Cup了。KDD Cup是美国计算机协会（Association for Computing Machinery，ACM）的数据挖掘及知识发现专委会（SIGKDD）组织的数据挖掘与知识发现竞赛，每年举办一次，是数据挖掘人才的顶级盛会。

6.6 动脑时刻

1. 参考6.4.2节关于age特征的分析过程，另外挑选两个特征完成特征分析。
2. 基于6.4.3节的结果，实现一种有效的特征交叉。
3. 使用TensorFlow实现6.4.4节的模型训练过程。

6.7 本章小结

本章从零开始，手把手教大家完成了一个简易的算法工程。根据具体项目情况，算法工程中每个步骤的细节都可以更加丰富。经过对本章内容的学习，相信大家对如何开展算法工程有了一个比较清晰的认识，不会再感到无从下手了。

关键词回顾

个人计算机

并行计算

中央处理器

图形处理器

张量处理器

神经网络处理器

操作系统

高级程序设计语言

开发环境

Python 3

PyCharm

scikit-learn

TensorFlow

随机梯度下降算法

AdamOptimizer

Dropout

模型保存

模型载入

Kaggle

KDD Cup

第7章 进阶学习

相信前6章的内容帮助大家打好了一定的AI算法基础。随着硬件算力的提升，算法模型的创新与优化，以及对新需求和新场景的挖掘，出现了越来越多的新兴研究领域。本章会介绍机器学习算法世界中的三大进阶概念，即深度学习、强化学习和迁移学习。之所以选择这3个方向，是因为笔者认为，这3个方向恰好能完整刻画出人类智能的闭环。

深度学习，主要用于描述人类大脑数以百亿计神经细胞的活动状态（兴奋或抑制），神经元之间的深度连接形成了人类大脑思维与创造的能力。

强化学习，主要用于描述人类适应外界环境、学习外界环境的过程，趋利避害、最大回报、延迟享受、对外界环境的极强交互学习能力，使人类成了站在地球食物链顶端的生物。

迁移学习，主要用于描述人类知识迁移模仿和举一反三的能力，其中最有名的为仿生学，如模仿鲸鱼设计的潜水艇外形、根据蝙蝠的回声定位能力制造的雷达、受野猪启发发明的防毒面具等。

7.1 深度学习

7.1.1 起源

顾名思义，**深度学习**（Deep Learning）就是深层次的学习。那么如何完成深度学习？不妨联想一下我们在第4章讲到的神经网络中的隐层，所谓Deep，其实就是神经网络结构中的Deep，即多隐层的神经网络。因此，深度学习也可以简单理解为深度神经网络。

既然有深度学习，就有**浅层学习**（Shallow Learning）。如线性回归和逻辑回归等神经网络模型，都可以划为浅层学习范畴。浅层学习起源于20世纪80年代末期，随着**反向传播**（Back Propagation，BP）算法的发明，线性回归和逻辑回归等浅层神经网络模型在那个年代被相继提出，并且掀起了一股机器学习热潮。那个年代的人们就已经发现，利用BP算法使神经网络模型从大量的训练样本中学习的统计规律，可以用于预测未知事件的发生概率。这种基于统计规律的神经网络模型，又称为**感知机**（Perceptron）模型。在当时，与基于人工规则确定的系统相比，感知机模型在很多方面都展现出了优势。

图灵奖得主、加拿大多伦多大学教授、机器学习领域的泰山北斗Geoffrey Hinton和他的学生在2006年的国际顶级期刊Science上发表了一篇关于深度学习的文章，由此掀起了深度学习在学术界和工业界的研究浪潮。自2006年以来，深度学习在学术界持续升温，在图像识别、语音识别、自动驾驶等领域都取得了惊人的成果。其中，图像识别常用的模型是卷积神经网络模型，语音识别常用的模型是递归神经网络模型。

当然，深度学习离不开大数据的支撑。为了理解这一点，下面通过一个语音识别的例子进行说明。语音识别是大数据背景下的一个机器学习算法建模问题，所使用的训练样本数量通常高达百亿到千亿级别。Google在一次语音识别实验中发现，训练后的DNN在训练集和测试集上的误差基本相同，这是非常违反常识的，因为训练集上的测试误差通常小于测试集上的误差。其实只有一个解释，那就是大数据蕴含丰富的信息维度，即使是DNN这样的复杂模型，也只处于欠拟合状态，更不必说浅层神经网络了。

深度学习的本质是通过构建多隐层的神经网络模型，在海量的训练数据中学习更有用的特征，从而提升分类预测的准确性。所以，DNN只是一种手段，挖掘深度特征才是真正的目的。与浅层学习相比，深度学习有以下两点不同。

- 深度学习强调模型结构的深度，通常在5层以上，甚至可以包含多于10层的隐层节点。

- 强调特征学习的重要性，通过逐层非线性变换，转换样本的特征空间，从而使分类预测更加容易。

由此可知，深度学习既可以直接用于完成分类和回归问题，又可以作为特征向量加入其他模型的训练过程中，后者被称为特征嵌入。

7.1.2 难点与方法

看过《快乐大本营》综艺节目的读者应该知道有个多人游戏叫作“快乐传真”，我们不妨先回忆一下“快乐传真”的游戏描述。

人数要求：5~8人为宜，前后成一字形排列。

游戏规则：第一个人面向主持人或出题人，其他人向后转，背对第一个人。第一个人看主持人或出题人给的题目，通常是方便用肢体展示的成语或俗语。在看完后，主持人收起题目，第一个人拍第二个人肩膀示意其转身，然后通过肢体语言（不能说话和摆口形），将刚刚看到的词语的含义表现出来。第二个人在会意之后，根据自己的理解同样通过肢体语言传递给第三个人，以此类推。在传到最后一个人时，这个人说出自己理解的词语。

这个游戏的有趣之处在于中间的肢体传递过程，这个肢体传递过程类似于深度学习中层与层之间的信息传递过程。在肢体语言的传递过程中，如果前一个人的肢体表演不足，传递出来的信息量过少，就不会传递给后面的人足够的信息；如果前一个人的肢体表演过于夸

张，可能传递出过多的信息，就容易使后一个人理解到多余的信息。总之，无论是哪种情况，都会使最后一个人无法准确地说出所传递的词语。如果将游戏中的每一个传递人都看作一层神经网络，他们既能输入信息，又能输出信息，那么可以将第一个传递人到最后一个传递人作为一个整体，这个整体可以看作一个深度神经网络。这个游戏中的两种问题恰好对应深度学习中的**梯度消失**和**梯度爆炸**两大难题。

深度神经网络通过反向传播的梯度更新每一层的模型参数，这个梯度类似于《快乐传真》游戏中传递的词语，通过层层传递，如果信息量过少，就会出现梯度消失问题；如果信息量过多，就会出现梯度爆炸问题。那么，深度神经网络是如何应对这两大难题的呢？

1. 梯度裁剪

梯度裁剪（Gradient Clipping）又称为梯度剪切，顾名思义，就是减掉多余的部分，主要是针对梯度爆炸提出的方法。具体而言，就是设置一个梯度裁剪的最大阈值，在使用链式法则求导计算各层参数梯度时，如果梯度超过最大阈值，就将最大阈值作为当前梯度。

2. 参数权重正则化

应对梯度爆炸的另一种比较常见的方法是参数权重正则化。参数权重正则化主要有两种方法，一种叫作L1正则，另一种叫作L2正则，具体做法都是在损失函数中增加一项，作为损失函数的惩罚项。以均方差损失函数为例，加入L1正则的Loss函数。Loss函数表达式如下。

$$\text{Loss} = (y - \omega^T \mathbf{x})^2 + \alpha \|\omega\|_1$$

在上述表达式中， $\| \cdot \|$ 是范数符号，

$$\|\omega\|_1 = \sum_{i=1}^n |\omega_i|$$

。直观理解就是在损失函数中加入各个参数权重的绝对值之和，从而达到抑制参数权重值过大的目的。

加入L2正则的Loss函数。Loss函数表达式如下。

$$\text{Loss} = (y - \omega^T x)^2 + \alpha \|\omega\|_2^2$$

$$\|\omega\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

在上述表达式中， $\|\omega\|_2$ 是L2范数。L2正则的直观理解就是在损失函数中加入各个参数权重的平方之和，也能达到抑制参数权重值过大的目的。

进一步理解，L1正则能够使特征权重变稀疏，即训练后出现较多值为0的参数权重，主要用于对特征进行筛选。L2正则无法达到类似的效果。我们以两个参数为例，在二维平面坐标系中，x轴和y轴分别代表这两个参数权重。其中L1正则贡献的损失值等高图像表现为菱形，L2正则贡献的损失值等高图像表现为圆形或椭圆形。可想而知，原损失函数的等高线与菱形相切的稳定位置必定在坐标轴上，即必有参数权重为0；而与圆形相切的位置，有可能稳定在坐标系第一象限的任意

位置。所以L1正则能够提取稀疏特征，L2只能抑制梯度爆炸，防止模型过拟合。

3. 选择合适的激活函数

除了对梯度本身做文章，我们还可以在计算梯度之前，在神经网络每一层的激活函数上做文章。这个思想非常简单直观，如果激活函数的导数是一个常数，如1，就不存在梯度消失和梯度爆炸的问题了。同时，每层神经网络的参数都能得到相同的更新速度。满足这个条件的激活函数为ReLU函数及其一系列的变种函数，此类函数的具体描述参考4.2.2节。

4. 批规范化

梯度裁剪过于粗暴，而且很难确定最优的裁剪阈值，参数权重正则化只是间接缓解梯度问题，合适的激活函数也只能保证导数不会过大或过小。对于深度神经网络中的梯度问题，批规范化（Batch Normalization，BN）才是真正对症下药的方法。自深度学习发展以来，批规范化是深度神经网络模型训练最重要的成果之一，它不仅解决了反向传播过程中的梯度问题，还可以提高深度神经网络参数的收敛速度，从而提升训练稳定性。

批规范化的具体操作方法如下：对于每一个batch的训练数据，以特征（神经元）为维度，通过标准化计算，将每层网络激活前的输出值分布转换成均值为0、方差为1的标准正态分布，然后使用激活函数将其输出。这样，每一层、每一个维度的特征输出值都符合标准的正态分布输出值，从而保证深度神经网络模型在反向传播训练中的稳定性。

5. 加入残差网络

对深度卷积神经网络的发展而言，残差网络（ResNet）可谓神来之笔。残差网络由当时在微软亚洲研究院的何凯明及其小组成员在论文中正式提出，在2015年ImageNet大赛中首次亮相，便摘得大赛的桂冠。此后，几乎所有的深度神经网络都离不开残差网络的身影，残差网络堪称深度神经网络的必备良药。因为加入了残差网络，可以非常轻松地构建上百层，甚至上千层的深度，而不用担心梯度消失的问题。残差网络的原理比较直观，与深度神经网络的结构相比，加入残差网络包含很多跨层连接的捷径（shortcut）。这样做的灵感来源是，如果增加一层网络结构无法使结果变得更好，那么至少别让结果变差。当反向传播的梯度在当前层发生梯度消失时，当前层的输出应该原封不动地包含上一层传递来的梯度，这种恒等映射（Identity Mapping）的方式可以保证当前层反向传播的梯度不会消失。

7.1.3 经典模型：AlexNet

本节介绍深度学习的经典模型——AlexNet。选择AlexNet，不仅因为它在2012年的ImageNet大赛中以低于第二名10.8个百分点的误差摘冠，还因为其首次使用的ReLU函数和Dropout技术现在仍然是深度学习必备的实用方法。AlexNet由Geoffrey Hinton和他的学生Alex在发表于NIPS 2012的论文中提出，其网络结构如图7-1所示。

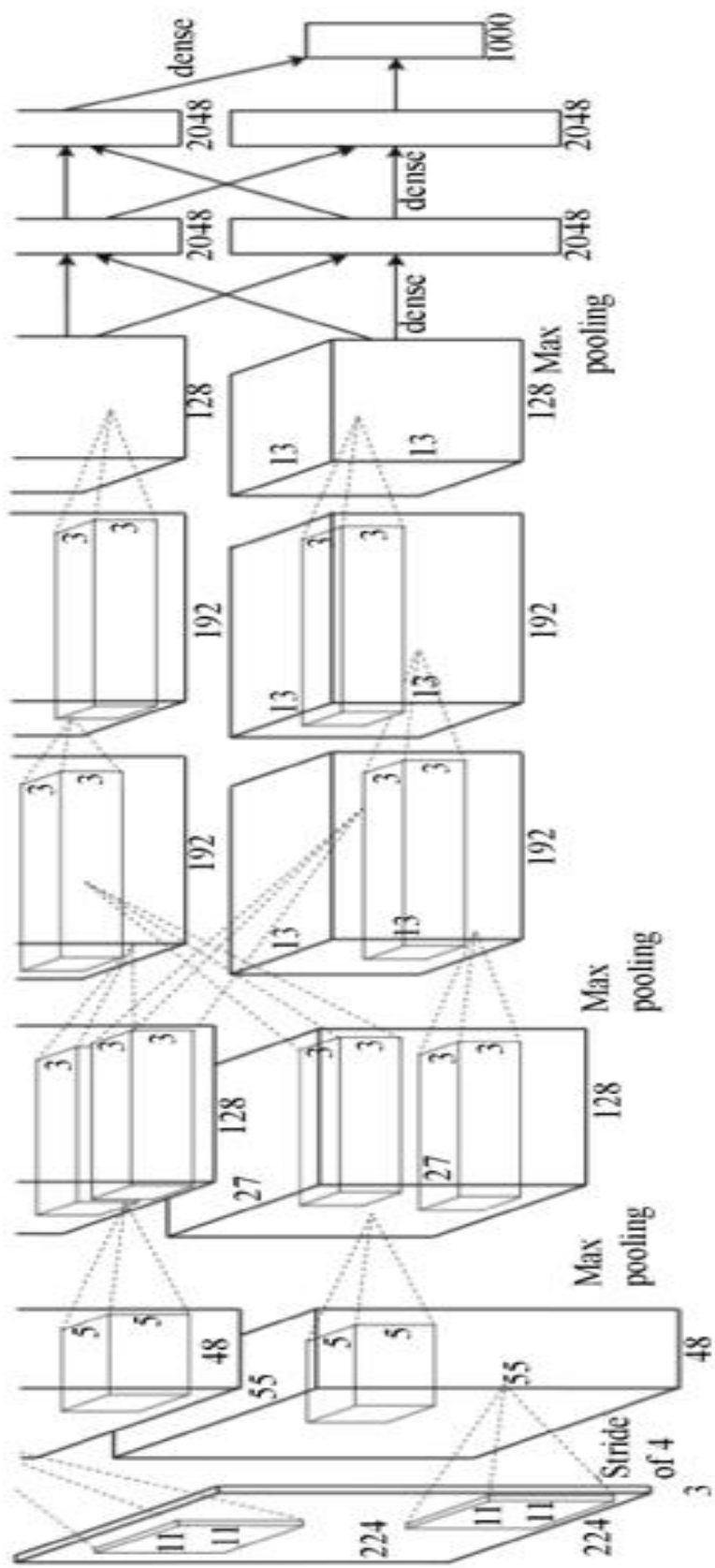


图7-1 AlexNet网络结构

第一层由卷积层、ReLU函数、LRN (Local Response Normalized)、最大池化层组成。在图7-1中，卷积层一共包含96个 11×11 的卷积核，分为两部分，每部分有48个卷积层，表示由两片GPU分别计算。接收的输入图像为 224×224 的RGB三通道图像。可以将通道理解为代表一个像素点的数据维度。例如，RGB图像由红、绿、蓝三原色组成，其通道数就是3。步长为4，卷积后输出 55×55 的图像，具体计算方式为 $(224-1)/4=55.75$ ，第56步卷积的卷积核中心位置超出原图像区域，因此长度和宽度均为55。在卷积之后使用ReLU函数激活，再通过LRN为局部神经元创建活动竞争机制，使较大的激活值变得更大，并且抑制周围激活值较小的神经元，从而增强模型的泛化能力。最大池化的尺寸为 3×3 ，步长为2，因此第一层最终输出的图像大小为 $27\times 27\times 96$ 。

第二层由卷积层、ReLU函数、LRN、最大池化层组成。在图7-1中，卷积层一共包含256个 5×5 的卷积核，分为两部分，每部分有128个卷积核，表示由两片GPU分别计算。接收的输入图像为上一层输出的 27×27 的96通道图像。步长为1，因此卷积不改变图像的形状、大小。在卷积之后使用ReLU函数激活，再通过LRN为局部神经元创建活动竞争机制。最大池化的尺寸为 3×3 ，步长为2，因此第二层输出的图像大小为 $13\times 13\times 256$ 。

第三层和第四层均由卷积层和ReLU函数组成，没有LRN和最大池化层。这两层的卷积层均包含384个 3×3 的卷积核，步长均为1，输出的图像大小均为 $13\times 13\times 384$ 。

第五层由卷积层和最大池化层组成。其中卷积层包含256个 3×3 的卷积核，并且直接通过步长为2的最大池化层，输出的图像大小为 $6\times 6\times 256$ 。

第六层和第七层均由全连接层、ReLU函数和Dropout组成，全连接神经元个数均为4096。

最后一层为Softmax层，包含1000个神经元，输出1000个分类概率，对应ImageNet比赛的总分类个数1000。

7.2 强化学习

7.2.1 起源

强化学习（Reinforcement Learning）又称为增强学习。强化学习的概念首次由Minsky于1954年提出，描述为通过奖惩手段进行学习的基本思路。1957年，Bellman提出了求解最优控制问题的**马尔可夫决策过程**（Markov Decision Process，MDP）的动态规划方法，使MDP成为定义强化学习问题的最普遍形式。1989年，Watkins提出的Q Learning进一步推动了强化学习的应用，给DQN（Deep Q Network）的出现奠定了基础。之后的二十余年，监督学习大放异彩，强化学习就此沉寂。

直到2013年，DeepMind公司发表了一篇利用强化学习玩Atari游戏的论文，由此开启了强化学习新纪元。2015年10月，由DeepMind公司开发的AlphaGo击败了人类专业棋手樊麾，成为第一个无须让子即可在19路棋盘上击败围棋职业棋手的围棋AI程序，描述AlphaGo的论文被发表在当年的国际顶级期刊Science上。时间来到2016年3月，通过自我对弈强化后的AlphaGo约战顶尖职业棋手李世石，最终以4:1的比分将李世石击败。

同年12月，AlphaGo的Master版本开始出现于弈城围棋网和腾讯野狐围棋网，以60连胜的战绩轰动围棋界。在这之后，DeepMind 如约在期刊Nature上公布了最新版AlphaGo论文。随后更新AlphaGo到最强、最新的版本AlphaGo Zero，该版本使用纯强化学习，只训练了3天，就以100:0的比分击败了上一版本的AlphaGo。这些都证明了深度强化学习的发展潜力。

所谓深度强化学习，就是深度学习与强化学习的结合。这里我们需要记住一个人，就是AlphoGo项目的带头人，强化学习界的泰山北斗——David Sliver。感兴趣的读者可以观看David Sliver关于强化学习的视频课程。

7.2.2 流派与分类

强化学习分为Value-based和Policy-based两大流派。前面提到的DeepMind公司就是Value-based流派的代表，另一个在强化学习领域同样造诣颇深的人工智能非营利组织OpenAI则是Policy-based流派的代表。与DeepMind着眼于游戏仿真AI的研究不同，OpenAI着眼于机器人仿真研究。两个流派之间的区别如下：Value-based的算法主要用于学习关于动作的价值函数，然后通过折算相应的概率，形成最终的行动策略，如经典算法DQN；Policy-based的算法主要用于学习动作的概率，无须先学习一个价值函数，如经典算法PG（Policy Gradient）。

强化学习的算法分为Model-free和Model-based两大类。在讲解这两类算法的区别前，先讲解强化学习的基础。和监督学习、无监督学习一样，强化学习也是机器学习的一个研究分支。不过，和监督学习、无监督学习相比，强化学习不一定需要大量的训练数据，它可以通过不断地试错来学习最大化的长期收益，更像是一种从零开始通过“左右互搏”的方式提高模型能力的学习方法。强化学习主要有以下4个特点。

- 没有标签数据，只有奖励信号。
- 奖励信号不一定实时反馈。
- 时间序列是一个重要因素。
- 当前行为影响后续行为。

强化学习中的角色有两种，分别为个体（Agent）和环境（Environment）。个体通过观察环境得到观察结果（Observations），并且根据观察结果做出相应的行为动作（Actions），然后从环境得到奖励信号（Rewards），环境也因此发生了变化。个体与环境之间如此周而复始地持续交互，就形成了强化学习的过程。

训练小狗的过程如图7-2所示。在这个过程中，小狗就是个体，小狗的主人就是环境，小狗观察主人的行为并做出相应的动作，然后根据动作结果从主人那里获得相应的奖励。这个过程可以看作一个强化学习的训练过程。

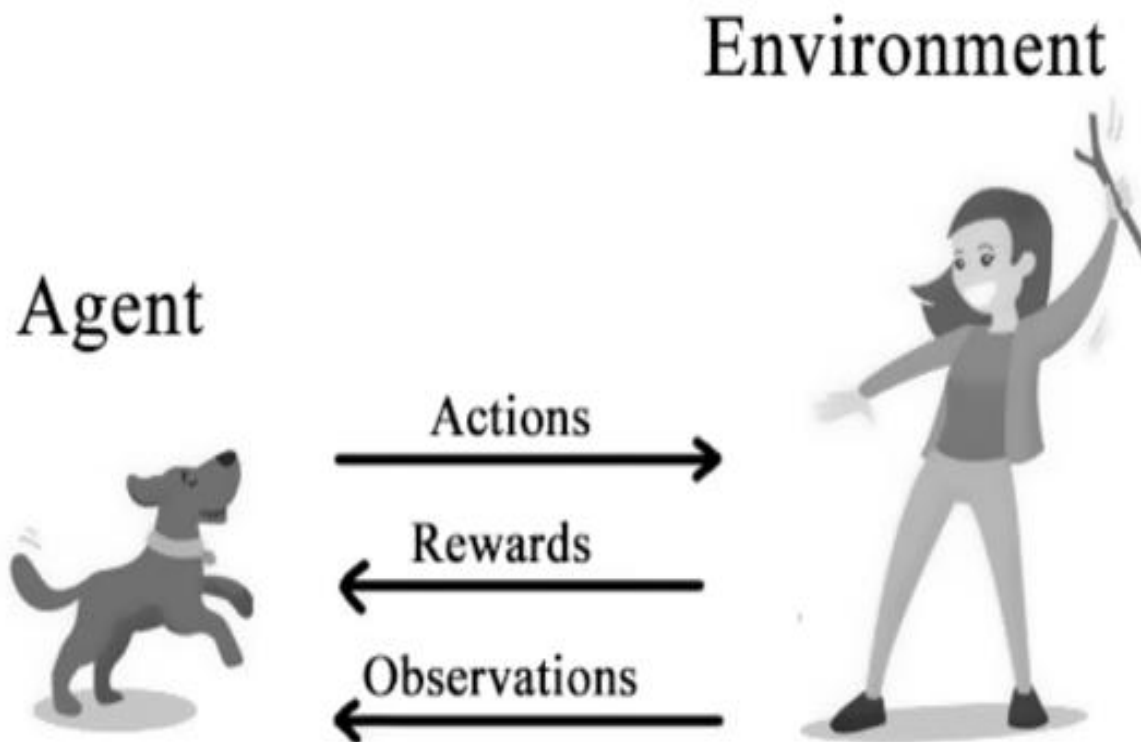


图7-2 训练小狗的过程

有了对强化学习的基本认识，下面介绍Model-free算法和Model-based算法的区别。首先，这两个名字中的“Model”指的是对环境建模，而不是对个体建模。顾名思义，Model-free的意思是不对环境建模，Model-based是对环境建模。对于Model-based，如果对环境信息一清二楚，并且事先掌握环境给的反馈，那么可以直接通过一些规划算法来

找到最优策略，如动态规划和贪婪算法。但是在大部分情况下，环境的表现和规律是十分复杂、令人捉摸不透的，对其进行精准建模是一件非常困难的事情。对于Model-free，我们通过对个体建模，学习动作到奖励的映射模型，也有可能学到最优的策略。

Model-free算法可以分为两类：**策略最优化**（Policy Optimization）类型的算法和基于**Q学习**（Q-Learning）的算法。按照流派划分，策略最优化类型的算法应该归为Policy-based流派，基于Q学习的算法应该归为Value-based流派。其中策略最优化类型的算法有PG算法、A3C（Asynchronous Advantage Actor-Critic）算法、PPO（Proximal Policy Optimization）算法和TRPO（Trust Region Policy Optimization）算法等，基于Q学习的算法有DQN算法、Double DQN算法和Dueling DQN算法等。将策略最优化类型的算法和基于Q学习的算法相结合，比较有名的算法有DDPG（Deep Deterministic Policy Gradient）算法和TD3（Twin Delayed Deep Deterministic Policy Gradient）算法等。

Model-based算法包括大家耳熟能详的围棋顶尖人工智能算法，如AlphaGo和AlphaGo Zero等。AlphaGo系列的围棋AI对整个围棋棋盘进行了建模，下棋规则对模型而言也是透明的，因此模型在进行任何一步之前都能通过蒙特卡罗树搜索近似地估算出奖励信号，即最终的输赢概率。下一节我们将详细讲解AlphaGo这个Model-based算法模型。

总之，区别一个强化学习算法模型是Model-free类型的还是Model-based类型的一个简单方法如下：如果一个算法模型能预测出下一步的环境状态及下一步的奖励信号，那么这个算法模型是Model-based类型的，否则是Model-free类型的。

7.2.3 经典案例：AlphaGo

本节会以AlphaGo为例，讲解强化学习的作用。网上其实已经有很多介绍AlphaGo背后技术的文章，但大部分文章一上来就将AlphaGo分成策略网络（Policy Network）和价值网络（Value Network）两部分来

讲解，未免过于抽象和笼统。本节会从技术角度为大家剖析初版AlphaGo的内部机制。

不妨掀开AlphaGo的神秘外衣，它其实是一个机器学习算法项目，至少包含三个模块：特征提取、模型训练和模型预测。下面我们从这三个方面展开，讲解这个“神秘的”机器学习算法项目。

1. 特征提取

对围棋数据来说，特征提取是指将每一步下棋状态数值化、结构化。AlphaGo所用的原始围棋数据取自KGS围棋服务器中6~9段的对局，共16万盘、近3000万步走棋。根据不同子模型的需求，AlphaGo项目组从这些原始棋局中一共提取了两份特征数据。

首先，我们不难想到最直观的棋盘特征，就是落子情况。围棋棋盘大小为 19×19 ，如图7-3所示，每个位置的可能落子情况有三种：黑子、白子或留白。因此，一个样本的落子特征数据就是3个 19×19 的矩阵，每个矩阵分别描述棋盘上的一种落子情况。

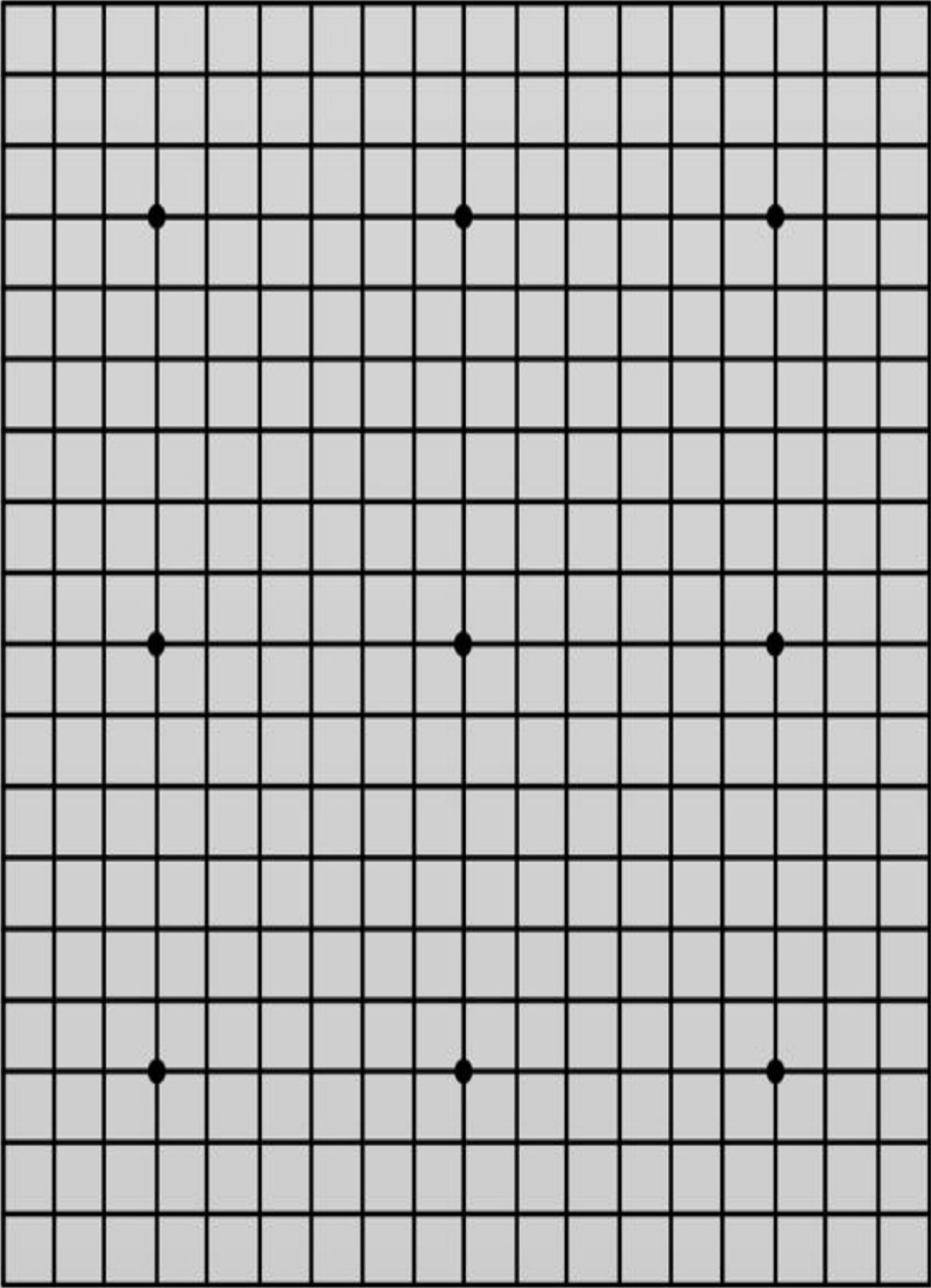


图7-3 围棋棋盘

除了落子情况，当前棋盘本身的状态也可以提取出许多特征，如当前执黑棋还是执白棋、每个落子所在位置经过的轮次、落子后预计吞子数目及落子是否合法等。以“每个落子所在位置经过的轮次”特征为例，一个8位的二进制数足以表示任意一方在整盘棋中所下的轮数，因此该特征可以用28个 19×19 的矩阵表示。落子情况的特征矩阵形式如图7-4所示，其他特征以此类推。

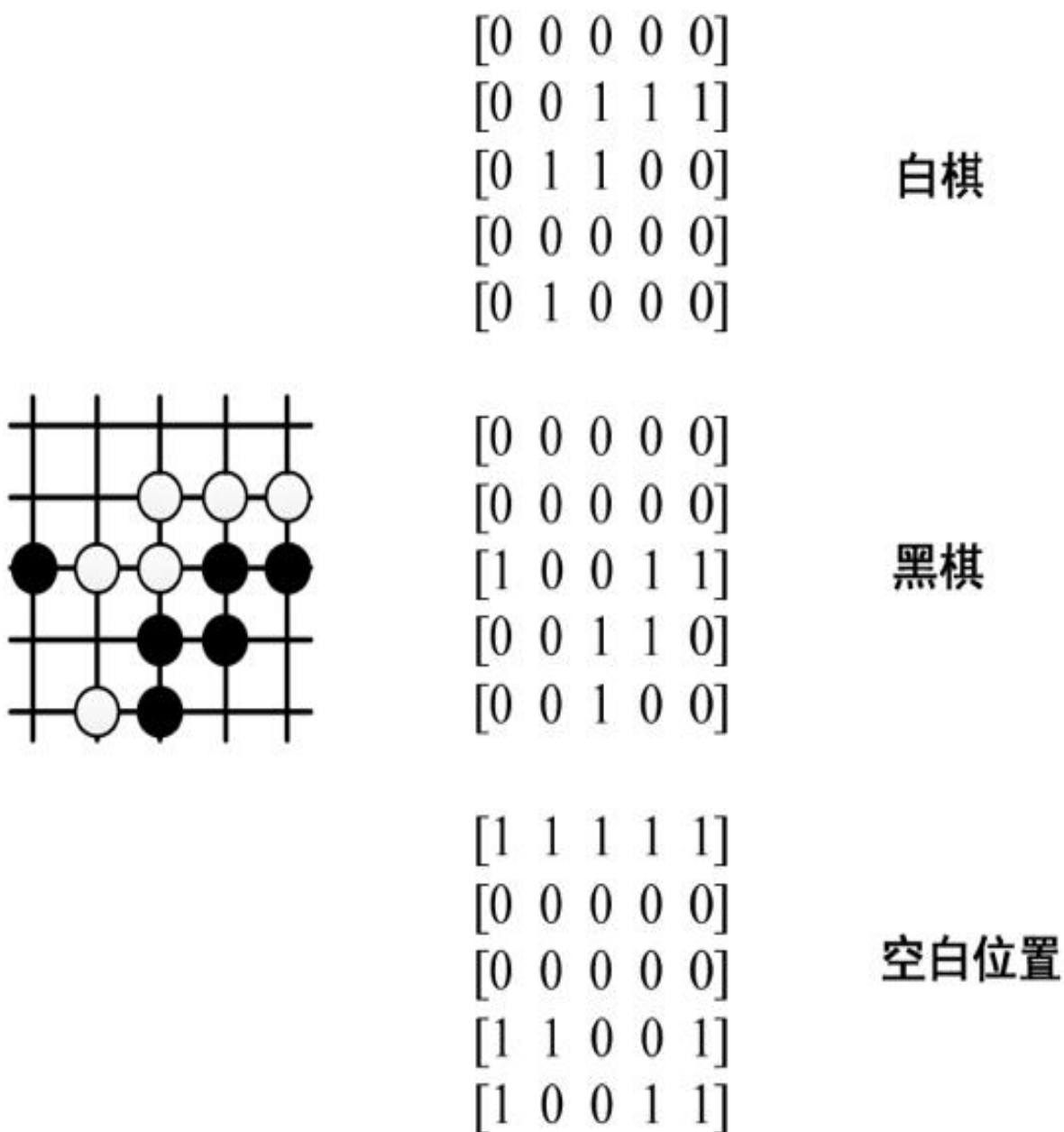


图7-4 落子情况的特征矩阵形式

加上落子情况，AlphaGo一共提取了48种特征，构成一个 $19 \times 19 \times 48$ 的特征空间。为了方便后文叙述，先将这份特征数据集命名为SetA。SetA相当于一份图片数据集，作为卷积神经网络的输入。除了图片数据集，AlphaGo还根据围棋的领域知识提取了另一份数据集，“喂”给Softmax回归网络的局部特征。我们将这份特征数据集命名为SetB。SetB主要用于实现快速走棋，为后面的蒙特卡罗树搜索（MCTS）做准

备。对于这些局部特征，AlphaGo论文中只提供了数目统计，并未给出这些特征的具体提取方法，论文给出的特征数目统计如图7-5所示。

Feature	# of patterns	Description
Response	1	Whether move matches one or more response features
Save atari	1	Move saves stone(s) from capture
Neighbour	8	Move is 8-connected to previous move
Nakade	8192	Move matches a <i>nakade</i> pattern at captured stone
Response pattern	32207	Move matches 12-point diamond pattern near previous move
Non-response pattern	69338	Move matches 3 × 3 pattern around move
Self-atari	1	Move allows stones to be captured
Last move distance	34	Manhattan distance to previous two moves
Non-response pattern	32207	Move matches 12-point diamond pattern centred around move

图7-5 特征数目统计

2. 模型训练

AlphaGo的子模型一共有4个：卷积策略网络、增强策略网络、Softmax回归网络和卷积价值网络。下面逐一讲解这4个网络结构及其训练过程。

1) 卷积策略网络。

这里有必要先铺垫几句，自Hinton在2006年重新掀起深度学习的浪潮以来，伴随着计算机硬件技术的迅猛发展，作为深度学习代表之一的卷积神经网络，在算法界可谓无人不知、无人不晓。卷积神经网络擅长解决图像识别和图片分类的相关问题，因此DeepMind团队选择了

卷积神经网络来训练AlphaGo的base模型。AlphaGo的卷积策略网络模型的网络结构如图7-6所示。

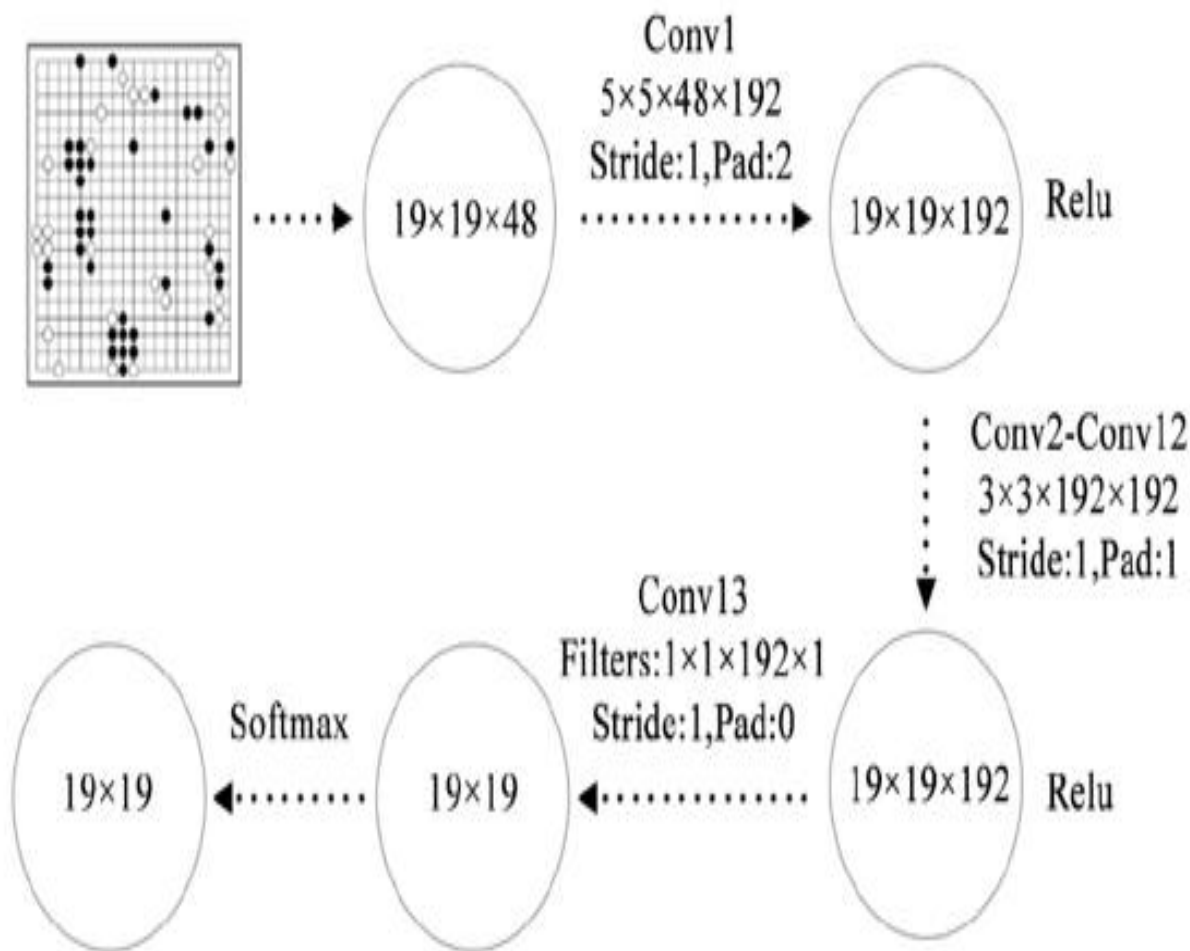


图7-6 AlphaGo的卷积策略网络模型的网络结构

输入数据集为setA，卷积策略网络共有13个卷积层，1个Softmax全连接层。Stride为步长，Pad表示边界补0个数。其中第1个卷积层使用192个5×5的卷积核，在Pad为2的情况下同尺寸ReLU输出；第2~12个卷积层均使用3×3的卷积核同尺寸ReLU输出；第13个卷积层使用1×1的单元核进行了一次线性加权求和，最后对接到一个Softmax全连接层，预测输出下一步落子位置的可能性。这个卷积策略网络能达到的AI围棋水平已经很高了，起码它已经学会了大部分人类高手的下棋方法。但这对AlphaGo来说还远远不够。

2) 增强策略网络。

如果说卷积策略网络是一个学习他人、超越他人的阶段，那么增强策略网络就是一个自我学习、超越自我的阶段，其背后就是强化学习。如前文所说，强化学习是一种通过不断地自我试错，从而找到最佳策略并加以激励的机器学习算法。以下围棋为例，两个完全不会下棋的人，一开始都随机落子，直到一方获胜。获胜的一方根据走法来优化自己的落子策略，然后两人继续不停地下棋、优化、下棋，重复这样的过程，直到满足某个条件。最终，不会下棋的人也能知道哪一步是好棋，哪一步是坏棋。这就是增强学习的一种通俗解释。

增强策略网络使用强化学习进行训练，这里不是随机落子，而是使用卷积策略网络这个基础AI进行落子。具体方法是先复制一份卷积策略网络参数给增强策略网络进行初始化，最终输赢标签定为和棋为0、赢棋为1、输棋为-1，然后用增强策略网络反复和自己对弈，根据输赢结果反向传播更新网络参数。在经过若干次迭代后，更新一份参数至对手网络，在满足一定的条件后结束。单凭该网络的下棋水准就已经达到了KGS的3d水平。不过，AlphaGo并没有在最后的系统中使用这个增强策略网络，理由是增强策略网络的走棋风格缺乏变化，对后面的蒙特卡罗树搜索不利。

3) Softmax回归网络。

大家或许会问，上面已经有了一个很强的走棋网络，为什么还需要一个Softmax回归网络来走棋呢？笔者认为有如下两个原因。

- Softmax回归网络结构只有一个隐层，与卷积策略网络的13层结构相比，其落子的速度要快得多，根本不在一个数量级上，并且准确率很高，而后面要讲的蒙特卡罗树搜索算法对搜索时间的要求很高，卷积策略网络无法满足要求。

- Softmax回归网络的数据集是setB，与setA处于完全不同的特征空间，因此能提高走棋策略的多样性，增加不同落子情况的可能性。事实证明，用Softmax回归网络单独落子，也可以达到KGS的3d水平。

4) 卷积价值网络。

这是AlphaGo的又一个厉害之处。也就是说，AlphaGo不仅有高效的落子方案，还会使用卷积价值网络评估当前棋局的获胜概率，从而双重保证当前的落子方案是最优方案。卷积价值网络的算法思路如下：随机棋局中的轮次为T，棋局的第1~T-1轮落子方案由卷积策略网络生成；然后第T轮采用随机落子方案，第T+1轮到该局结束使用增强策略网络生成落子方案；最后计算输赢结果，赢棋为1，输棋为0，用反向传播误差调整网络参数。卷积价值网络的结构如图7-7所示。

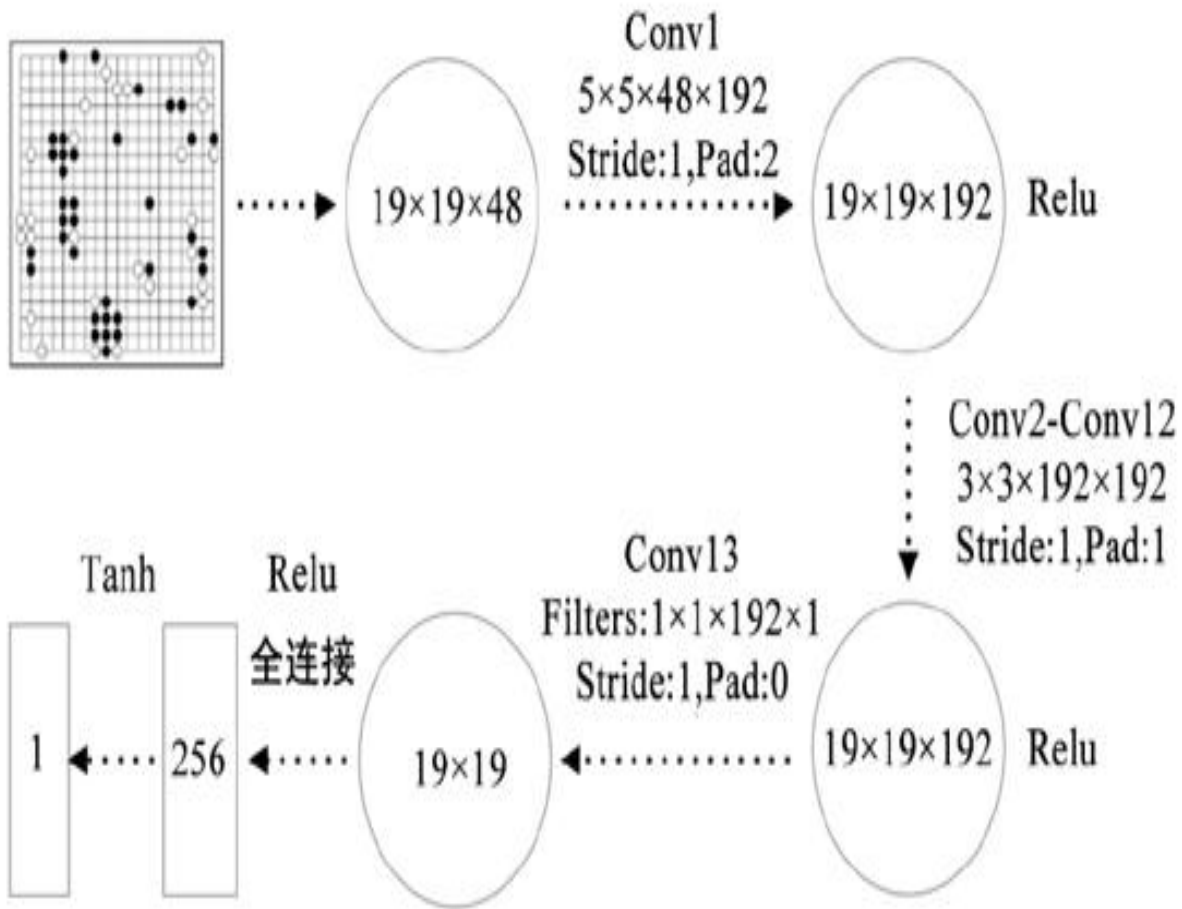


图7-7 卷积价值网络的结构

根据图7-7可知，输出层将Softmax全连接层替换成了256个全连接层和1个Tanh激活层，其他层的结构与卷积策略网络一模一样。需要注意的是，这里的输入数据集全部是由卷积策略网络和增强策略网络共

同生成的。而且为了避免在相同对局下，输入不同但结局相同所带来的过拟合现象，每轮对局只会取一个样本进行训练，这也是单独采集近3000万个对局样本作为训练集的原因。

3. 模型预测

打造好了上述四件“兵器”——卷积策略网络、增强策略网络、Softmax回归网络和卷积价值网络，AlphaGo“披挂上阵”。它走进了人类顶级围棋道场，与顶级棋手进行较量，采用的战术策略就是前面提到的蒙特卡罗树搜索算法，其示意图如图7-8所示。

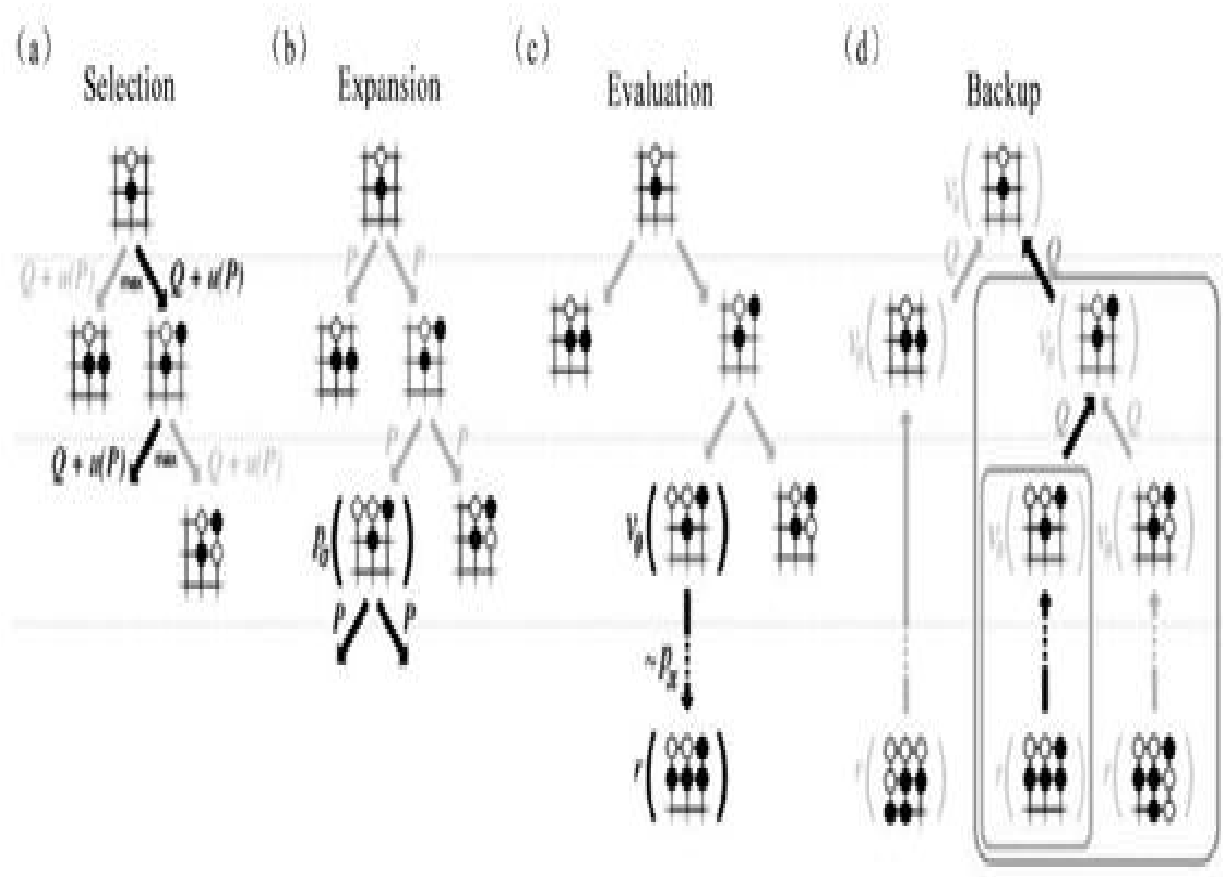


图7-8 蒙特卡罗树搜索算法示意图

蒙特卡罗树搜索算法的基本思路如下：多次模拟未来棋局，然后选择模拟次数最多的落子方案作为当前走棋方案。

具体步骤如下：首先，使用卷积策略网络生成未来T轮的落子方案；然后，使用卷积价值网络评估该落子方案的胜率，使用Softmax回归网络基于该落子方案继续快速走棋，直至棋局结束得到输赢结果，综合两个结果算出这T轮落子方案的胜率；接着，结合该胜率，使用卷积策略网络重新生成未来T轮的落子方案，如果出现相同走棋方法，则胜率取平均值；最后，循环上述过程若干次，选择模拟次数最多、胜率最高的落子方案作为当前走棋方案。

如果单纯将AlphaGo分成策略网络和价值网络，那么策略网络指的是上面所说的卷积策略网络，而价值网络则是卷积价值网络与Softmax回归网络的结合。其中，策略网络负责计算当前的落子位置，价值网络则负责评估落子该位置的最终胜率，蒙特卡罗树搜索算法则利用这两个结果进行搜索，给出最终的落子方案。

对战柯洁的AlphaGo Master版本要胜过战胜李世石的版本，与柯洁对战三局下来也没有出现像对战李世石第四局那样因“昏招”而落败的情况。推测原因，一是进行了优化，结合快速走棋搜索节点，而不是只利用卷积策略网络搜索节点，这样可以避免短期内落入局部思维而完全忽略人类的“神来之笔”；二是数据集已经完全弃用人类棋局，它由从AlphaGo自我对弈中挑选出的样本数据构成，因此策略网络更加强大。之所以只用卷积策略网络搜索节点存在弊端，是因为卷积核一般都很小，导致棋盘不同位置的局部视野需要在若干轮后才能互相影响，因此才出现了所谓的“昏招”。

7.3 迁移学习

7.3.1 简介

顾名思义，**迁移学习**（Transfer Learning）就是将学过的知识用于解决新问题，从而达到快速解决新问题的目的。迁移学习和强化学习一样，也是机器学习的一个研究分支。从20世纪90年代开始，迁移学

习吸引了越来越多研究者的目光，其初衷是节省人工标记样本的时间，让模型通过对已有的标记样本进行学习，将预测能力迁移到未标记的样本空间。随着机器学习在实际业务中的普及，新业务在早期通常缺乏足够的训练样本，而具有一定相似性的老业务拥有大量的可训练样本，只是样本特征分布可能与新业务的样本特征分布不同，迁移学习就是利用老业务场景中的样本数据快速训练出一个适应新业务场景的模型的过程。也就是说，迁移学习是从**源领域**（Source Domain）中学习知识和经验，然后应用到**目标领域**（Target Domain）中的一个过程。

7.3.2 方法与研究方向

迁移学习主要有以下4种迁移方法。

1. 样本迁移

如果直接将源领域中的所有样本数据用于目标领域中的模型训练，那么效果会比较差。利用样本相似度衡量，我们可以在源领域中找到很多可以用于目标领域中的相似样本数据。例如，源领域中有大量的动物图片，目标领域的任务是识别小狗，但只有少量的样本图片，我们可以从大量的动物图片中找出特征最相似的小狗图片，从而解决小狗图片样本短缺的问题。

2. 特征迁移

挑选源领域中具有代表性的特征，通过特征变换将源领域和目标领域中的特征映射到相同的空间中，使这个空间中源领域和目标领域中的样本数据具有相同的特征分布。

3. 模型迁移

模型迁移又称为参数迁移。假设源领域和目标领域之间共享一些参数，或者共享模型超参数的先验分布，那么在目标领域中使用迁移源领域的老模型也可以达到很高的精度。预训练（Pre-training）加微调（Fine-tuning）的迁移学习方法就是一种模型迁移，这种方法也是最常用的迁移学习方法之一。

4. 关系迁移

关系迁移类似的数据关系是可以拿来迁移的，如师生关系和上下级关系、生物病毒的传播与网络病毒的传播等。

迁移学习领域比较有名的两个研究方向分别为**多任务学习**（Multi-task Learning）和**领域自适应**（Domain Adaptation）。

大部分机器学习算法的训练目标只有一个，而多任务学习的训练目标有多个。从广义上来说，损失函数包含多个目标（不含正则惩罚项）的计算，是多任务学习。多任务学习虽然属于迁移学习的范畴，但与通常意义上的迁移学习存在一定的区别。首先，迁移学习的“迁移”，是有学习的先后顺序的，而多任务学习是多个任务一起学习，没有先后顺序；其次，多任务学习的目标是提升所有任务而非单一任务的学习效果。比较有名的多任务学习应用有Google公司的MMoE（Multi-gate Mixture-of-Experts）模型、阿里巴巴公司的ESMM（Entire Space Multi-Task Model）模型等。

机器学习算法能在各大场景中取得不错的效果，其前提是假设源领域和目标领域中的样本数据均服从**独立同分布**（Independently Identically Distribution，IID）。当源领域和目标领域中的样本数据不服从独立同分布时，机器学习算法会出现过拟合问题，导致分类器的性能很差。领域自适应就是解决这个问题的研究课题之一。领域自适应问题作为迁移学习的一个子问题，在相似的不同任务之间，以及同一个任务下不同类别的样本（如不同的人群和商品类别）之间，都经常出现。在同一个任务下的领域自适应问题，直观的表现就是将数据的相关性错误地当成了因果性，这一类问题有一个专门的研究方向，称

为**因果推断**（Causal Inference）。比较流行的领域自适应算法有 DaNN（Domain Adaptive Neural Network）算法、DDC（Deep Domain Confusion）算法和DAN（Deep Adaptation Network）算法等。

7.3.3 经典模型：TrAdaBoost

在进行算法模型训练前，通常会假设划分的训练集和测试集的分布相同。但是在许多实际情况中，这种服从单一分布的假设并不成立。如果新数据集分布与训练所用的旧数据集分布不一样，那么算法的准确率就会大打折扣。但是完全使用新数据集重新进行训练，又会存在两个问题：其一，新数据集的样本数量可能不够；其二，旧数据集并非所有的样本数据都没有价值，完全不用又比较浪费。由戴文渊于2007年提出的TrAdaBoost算法，是一种经典的解决上述问题的机器学习算法。TrAdaBoost算法可以从旧数据集中提取出有价值的样本数据，并且结合收集到的少量新样本数据，学习出比单纯使用新样本数据训练更精确的结果。假设有 n 个数据来自旧数据集 d ，有 m 个数据来自新数据集 s ，TrAdaBoost算法的整体步骤如下。

（1）计算每个样本权重。

（2）调用弱分类器，将旧数据集和新数据集合在一起作为训练数据，过程和AdaBoost训练弱分类器一样。

（3）计算新数据集中所有样本数据的错误率。

（4）分别计算新数据集和旧数据集中样本权重的调整速率。

（5）更新样本权重。如果新数据集中的样本分类错误，则提高该样本的权重值；如果旧数据集中的样本分类错误，则降低该样本的权重值，这是因为分类错误意味着旧数据集中的这个样本与新数据集中的这个样本之间的差距太大。最后，将后面50%弱分类器的投票结果作为模型的最终结果输出。

7.4 动脑时刻

1. 深度学习、强化学习、迁移学习三者之间有什么关系？
2. 在深度学习模型中加入Dropout的作用是什么？
3. 第4章中提到的梯度消失问题在深度学习中经常出现，有哪些解决方法？

7.5 本章小结

思维与创造能力、交互学习能力、知识迁移能力，三者共同构建了人类智能的闭环。本章分别介绍了AI算法中代表这三大能力的深度学习、强化学习和迁移学习，从起源讲到其各自具有代表性的算法模型，旨在起到抛砖引玉的作用，帮助感兴趣的读者对这三个研究方向有一个初步的认识，方便后续深入研习学习。

关键词回顾

深度学习

强化学习

迁移学习

浅层学习

反向传播

感知机

梯度消失

梯度爆炸

梯度裁剪

批规范化

残差网络

马尔可夫决策过程

动态规划

Value-based

Policy-based

DQN

Policy Gradient

Model-free

Model-based

Agent

奖励信号

策略最优化

Q-Learning

AlphaGo

源领域

目标领域

样本迁移

特征迁移

模型迁移

关系迁移

多任务学习

领域自适应

独立同分布

因果推断

第8章

思考与展望

对想从事AI算法行业或对AI算法抱有感兴趣的读者而言，在各种算法培训、学习资料应接不暇的大环境下，本书向读者较为简明、完整地描绘了一条算法入门之路。或许有不少读者对第3章中的内容心存疑惑，入门学习AI算法是否有必要学习这些基础算法技能？笔者认为是有必要的，理由有二。其一，对于算法工程前期的数据统计验证工作，适当掌握一些基础算法技能，有助于我们写出更高效的数据分析程序；其二，掌握一些常见的基础算法思路，可以帮助我们培养更好的逻辑思维，这一点在模型实践过程中的作用不言自明。当然，本章的目的不是要阐述前面每章节的必要性，而是谈一谈笔者对AI算法行业的思考与展望。

8.1 思考

8.1.1 人工智能感悟

人工智能是研究、开发用于模拟、延伸和扩展人的智能的理论、方法、技术及应用系统的一门新的科学技术。算法、数据和算力被称为推动人工智能发展的三个重要基础，而决定智能程度高低的关键因素就是算法。如果将数据比作知识，将算力比作精力，那么算法相当于学习方法。如果一个人不花费精力使用合适的方法学习，那么这个

人很难达到较高的智能程度。从这个角度来看，算法对人工智能的发展起着重要作用。

然而，现阶段的人工智能水平距离实现真正的智能还有相当长的一段距离。一句流行语是这么说的，“有多少人工，就有多少智能”。当然，这个讽刺的主要是那些虚假的同声传译和人机交互。即便强如AlphaGo，在人类围棋界一骑绝尘的情况下，也远没有做到真正的智能。AlphaGo的出现只是说明了人类的围棋水平还远远不够罢了。我在《AlphaGo技术剖析：揭开围棋大脑的神秘面纱》一文中写道：智能所体现的思维与认知，没有标准。真正的智能体至少应该具备两种能力——想象力和创造力。由此观之，AlphaGo做到的也只是对既定目标的学习训练，如果目标更换，则会束手无策，需要重新建模训练。

怎样才能实现不那么人工的智能？不妨从人的角度分析这个问题，看看一个人到底是如何学习新鲜事物的。假如有两个运动天赋相同的人A和B，他们都想学习网球，其中A会羽毛球，B什么运动都不会，问谁学习网球的速度更快一些，谁最终的网球水平更好一些？对于第一个问题，大家自然想到是A，因为A会羽毛球，B不会，而羽毛球和网球有不少相似之处。对于第二个问题，由于运动天赋相同，因此得看两人的学习程度。这两个问题的答案放到人工智能的算法世界，对应的是迁移学习和强化学习。迁移学习能够举一反三，从而快速学习未知领域的知识；强化学习能够通过不断地试错练习，深度掌握某个领域的知识。

从这个角度来看，迁移学习和强化学习或许是实现真正人工智能的重要手段。但是要让机器像人一样思考，拥有自主意识，具备想象力和创造力，除了需要结合算法、数据和算力三方面因素，对人脑神经元的深入研究也必不可少。总而言之，算法是实现人工智能的重要因素，但是要实现真正的智能，只提升算法能力仍然远远不够。

8.1.2 万物数据化

在影片《黑客帝国》中，主角尼奥在影片最后阶段与史密斯的决斗过程中，看到身边的世界全部都是绿色的数字信号，如图8-1所示。笔者对这个场景记忆犹新，虽然现实世界并非如电影中的母体一般，所有事物都由程序构建，但是作为人工智能发展的重要基础之一，万物数据化却是实实在在逐渐向前推进着的。



图8-1 影片《黑客帝国》片段

我们知道，只要有数据，就可以训练出一个机器学习算法模型。如果世间万物都被转换成数据形态存储，那么建立在这之上的认知、

理解、推断等智慧生物才能拥有的行为和意识，是否也有可能发生在所谓的人工智能身上呢？首先，我们来讨论一下万物数据化背后的意义。我们不妨从人的角度，以万物数据化的方式模拟这件事情。

早晨上班出门，你看了看窗外的天空，晴空万里，天气特征下雨那一项的值为0，这一项对于“出门是否带伞”这个目标是强相关的，因此大概率你不会带伞。当然，之所以不是100%不带伞出门，是因为有可能同时存在相反方向的强相关事件，如“归还同事的伞”或“担忧下班回家下雨”。不管是还伞还是担心下班回家下雨，不同的人都有一个实时变化的触发概率，毕竟什么事都有可能忘记。

出了单元门，你的下一个动作暂时是以100%的概率持续朝前走，朝前走这个过程会持续很长一段时间。在这期间至少包含两部分数据。一部分是你本身的数据，包括如何到达目的地的路径规划数据、各种情绪值及各种定时任务的数据。例如，在九点之前在家庭群里发送早安问候，在疫情期间十点之前完成健康打卡，等等。当然这些都是以概率的形式触发的，也就是说有可能延期，不同任务在延期之后都有对应的处理规则。另一部分是实时接收的数据，包括自身出门前未注意到的衣着形体数据、当前道路数据，以及周围的各种人、事、物的数据。根据这两部分数据，你会实时调整下一刻若干动作发生的概率。例如，在走向地铁站的这段路中，在通常情况下，“沿着路径规划往前行走”这个动作的概率是最高的。此时，你的眼睛捕捉到路边有人买卖小动物的信息，CNN图像识别告诉你是在卖小猫小狗。然后你迅速匹配到自己的动物兴趣库，发现小猫的得分比较高。这时，“沿着路径规划往前行走”暂时就不是你最高概率的下一个动作了，而“驻足撸猫”这一动作的概率迅速拔高。你依据目标的位置，停留在小猫的旁边抚摸、拍照。这时，你的兴趣指数从高点缓慢降低，上班迟到指数由低点缓慢上升，“沿着路径规划往前行走”的概率也随之慢慢增大，“驻足撸猫”的概率则慢慢减小。在超过一定阈值之后，你恢复了“沿着路径规划往前行走”的动作，由于“驻足撸猫”的动作加成，此时情绪值里的开心指数也有所提高。

进入地铁进行安检，你会触发判断“是否有背包”事件，过了安检，来到刷卡闸机口，触发最高概率的刷卡动作，如采取“手机刷

卡”的动作进入地铁站等候。高峰期地铁人多，你会以一定概率忍受人潮的拥挤钻进车厢，各项情绪值并没有为此产生较大波动，那是由于你早已装载了稳定版本的北京早高峰模型。在上车后，你以一定概率拿出耳机，打开手机中的某音频App开始打发地铁时光。此时，一个临时的倒数计数任务在后台生成，即下车站数或换乘所需乘坐站数。

在到站后，临时任务触发下车动作。你走出地铁站朝公司大楼走去，一路上无突发事件，你以高概率“沿着路径规划往前行走”的动作来到公司，开启了新一天的工作。

以上就是一段简易的万物数据化的模拟过程。对于不同的数据，会有一个或多个目标事件，结合不同人本身的特征数据，每个目标事件或动作都有一个可能发生的概率，不同概率触发的事件或动作又会实时改变本身的部分特征值，进而实时改变下一步不同动作发生的概率，从而形成一连串序列行为的表现。

由此观之，人的行为是一个极其复杂的序列建模过程，这比任何一个已知的机器学习序列行为建模都要复杂得多。难点主要有两点：第一，非结构化的数据太多，而且一直在实时发生变化，这使结构化时间维度下的万物数据化成为几乎不可能完成的任务；第二，常规的样本收集方法完全行不通，目前还不存在能将所见、所得像日志一样完整地记录下来的人脑芯片。即便最终实现了万物数据化并训练出了这样的智能模型，这个模型做的也只是按照概率不断地完成一个又一个目标，仍然没有自主意识，仍然不能去想象、去创造。

所以，笔者理解的万物数据化，最终也只能帮助我们更加便捷地与这个世界相处，无法做到真正的智能。好比科幻电影中的智能眼镜，在戴上后看到什么事物，就能将该事物对应的详细信息传到我们的大脑，方便我们利用这些数据进行思考、想象，然后做出反应。

8.2 展望

8.2.1 人工智能最终能做什么

在未来，人工智能无论发展到何种程度，都一定是以服务人类为宗旨的存在。那么人工智能最终能做什么？既然是以服务人类为宗旨的，不妨换一个角度思考这个问题，想想人类需要什么。

人类需要安全感。对于任何其他方面的需求，其前提条件都是安全。人工智能可以承担人类从事的不安全工作，如消防、矿采等高危工作。

人类需要时间。生命的长度是有限的，很多浪费时间的家务都可以交给人工智能来做，如做饭、洗碗、洗衣、清扫房屋等。

人类需要享受。在这方面，如今的人工智能已经做得比较好了，如可穿戴设备、虚拟现实、增强现实等相关娱乐设施、电子游戏等。所谓科技造福人类社会，人工智能的终极奥义莫过于此。

8.2.2 人类最终能做什么

人类最终能做什么？同样地，我们也可以换一个角度思考这个问题，想想人工智能做不了什么。

人工智能没有感情。这是人之所以区别于机器的最大原因，也是人工智能无法代替人的根本原因。人的同情心、共情能力、亲情、爱情、友情等将一代一代传递下去。

人工智能无法代替人类运动。所谓“流水不腐，户枢不蠹”，生命在于运动，运动是生物的本能，而且运动能让身体保持健康的状态，也只有健康的身体状态，才能驾驭更先进的人工智能。

8.3 本章小结

本章对AI算法的未来展开了思考和展望，阐述了人工智能发展的三个重要基础。一方面，笔者对人工智能充满了希望，相信人工智能会在更多领域普及和服务人类；另一方面，笔者也强调对待人工智能的理性态度，即使在未来，人工智能也并非无所不能，人类需要保持作为人类的坚定姿态。希望本章讨论的内容，能激发读者对算法与人工智能更深层次的理解。谢谢大家的阅读，再见！

反侵权盗版声明

电子工业出版社依法对本作品享有专有出版权。任何未经权利人书面许可，复制、销售或通过信息网络传播本作品的行为；歪曲、篡改、剽窃本作品的行为，均违反《中华人民共和国著作权法》，其行为人应承担相应的民事责任和行政责任，构成犯罪的，将被依法追究刑事责任。

为了维护市场秩序，保护权利人的合法权益，我社将依法查处和打击侵权盗版的单位和个人。欢迎社会各界人士积极举报侵权盗版行为，本社将奖励举报有功人员，并保证举报人的信息不被泄露。

举报电话：（010）88254396；（010）88258888

传 真：（010）88254397

E-mail：dbqq@phei.com.cn

通信地址：北京市万寿路173信箱

电子工业出版社总编办公室

邮 编：100036

好书分享



Broadview
www.broadview.com.cn

博文视点·IT出版旗舰品牌
技术凝聚实力·专业创新出版

深入浅出AI算法 基础概览

专家力荐



AI是一项具有历史性意义的革命技术，了解人工智能，了解新时代计算机技术发展趋势对每一位时代新人都具有深远意义。在作者的缜密逻辑下，本书深入浅出话AI，将看似高深莫测的AI技术抽丝剥茧、化繁为简，可帮助读者破解AI奥秘。

黄山

Mathlab联合创始人

在人工智能愈发被普通大众所了解，各国政府大力投入人工智能领域研究的今天，《深入浅出AI算法：基础概览》作者结合自身从业经验，归纳和总结出人工智能基础学习路径，引导读者从理论到实践，让感兴趣的“门外汉”学习人工智能是什么、做什么及怎么做。本书是一本介于科普图书与专业图书之间的上乘之作。

张亚雄

互联网连续创业者

我与作者熟识多年，作者无疑是喜欢且善于思考和总结的。本书立意新颖，以算法为主线，深入浅出、循序渐进地介绍了算法的相关基础知识并拓展至AI算法，同时融入了作者的思考和多年互联网一线工程实践的经验，值得一读。

王雷岩

清华大学博士，北京航空航天大学“卓越百人”博士后

能在繁忙的工作之余写出书的人不多，《深入浅出AI算法：基础概览》一书展现了作者对算法世界清晰、完整的见解，值得同行借鉴。

郭乐

阿里巴巴高级算法专家

《深入浅出AI算法：基础概览》一书用浅显易懂的语言为读者构建了AI算法的完整知识体系，同时结合了作者多年的一线算法工作经验，将理论和实践相结合，可以给入门者起到很好的引导作用，非常值得阅读。

魏凯峰

《MXNet深度学习实战》图书作者，网易资深算法工程师



读者服务

微信扫码回复：41551

- 获取章后习题答案。
- 加入“算法”读者交流群，与更多同道中人互动。
- 获取《百场业界大咖直播合集》（永久更新），仅需1元。



责任编辑：宋亚东
封面设计：李玲

欢迎
投稿

微信：qie_yd
邮箱：syd@phei.com.cn

上架建议：人工智能

ISBN 978-7-121-41551-7



定价：89.00元